



Security of Language Models for Code: A Systematic Literature Review

YUCHEN CHEN, State Key Laboratory for Novel Software Technology, Nanjing University, China

WEISONG SUN*, Nanyang Technological University, Singapore

CHUNRONG FANG*, State Key Laboratory for Novel Software Technology, Nanjing University, China

ZHENPENG CHEN, Nanyang Technological University, Singapore

YIFEI GE, State Key Laboratory for Novel Software Technology, Nanjing University, China

TINGXU HAN, State Key Laboratory for Novel Software Technology, Nanjing University, China

QUANJUN ZHANG, State Key Laboratory for Novel Software Technology, Nanjing University, China

YANG LIU, Nanyang Technological University, Singapore

ZHENYU CHEN, State Key Laboratory for Novel Software Technology, Nanjing University, China

BAOWEN XU, State Key Laboratory for Novel Software Technology, Nanjing University, China

Language models for code (CodeLMs) have emerged as powerful tools for code-related tasks, outperforming traditional methods and standard machine learning approaches. However, these models are susceptible to security vulnerabilities, drawing increasing research attention from domains such as software engineering, artificial intelligence, and cybersecurity. Despite the growing body of research focused on the security of CodeLMs, a comprehensive survey in this area remains absent. To address this gap, we systematically review 68 relevant papers, organizing them based on attack and defense strategies. Furthermore, we provide an overview of commonly used language models, datasets, and evaluation metrics, and highlight open-source tools and promising directions for future research in securing CodeLMs.

CCS Concepts: • **Software and its engineering** → **Automatic programming**; • **Security and privacy** → **Software security engineering**.

Additional Key Words and Phrases: Language models, Code, Security, Attack, Defense

*Corresponding authors.

Authors' addresses: Yuchen Chen, yuc.chen@smail.nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China, 210093; Weisong Sun, weisong.sun@ntu.edu.sg, Nanyang Technological University, singapore, 50 Nanyang Avenue, Singapore, 639798; Chunrong Fang, fangchunrong@nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China, 210093; Zhenpeng Chen, zhenpeng.chen@ntu.edu.sg, Nanyang Technological University, singapore, 50 Nanyang Avenue, Singapore, 639798; Yifei Ge, gyf991213@126.com, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China, 210093; Tingxu Han, txhan@smail.nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China, 210093; Quanjun Zhang, quanjun.zhang@smail.nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China, 210093; Yang Liu, yangliu@ntu.edu.sg, Nanyang Technological University, singapore, 50 Nanyang Avenue, Singapore, 639798; Zhenyu Chen, zyichen@nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China, 210093; Baowen Xu, bwxu@nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China, 210093.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7392/2025/8-ART

<https://doi.org/10.1145/3735554>

1 INTRODUCTION

Language models for code (in short as CodeLMs), such as Codex [127] and CodeLlama [115], have become powerful tools in Software Engineering (SE) [101], addressing tasks such as code search [156, 157] and code completion [73, 163]. These language models utilize advanced deep learning techniques to analyze and represent code, significantly improving performance compared to traditional methods. One notable development is GitHub Copilot [46], an AI-powered coding assistant built on the Codex [127] model. Trained on diverse programming patterns and vast open-source code from GitHub, Codex enables Copilot to suggest and complete code in real-time, greatly enhancing developers' productivity.

Despite the success of CodeLMs in solving various code-related tasks, their inherent vulnerabilities raise significant concerns, particularly in security-critical applications such as network security and malware detection. These models are susceptible to backdoor attacks, where hidden triggers are embedded during training [154]. Triggers are specific input patterns designed by attackers, such as particular variable names or dead code snippets in the code, which are used to activate the model's malicious behavior. For example, Schuster et al. [142] demonstrated that backdoors in CodeLMs can significantly increase the likelihood of generating vulnerable code. In addition to backdoor attacks, adversarial attacks [24, 164] present another challenge to the security of CodeLMs. These attacks occur during the inference phase, where attackers add subtle perturbations to inputs, causing models to make incorrect predictions with high confidence, further highlighting the fragility of these models.

To counter attacks on CodeLMs, various defense methods have been developed. For backdoor attacks, defenses often focus on detecting outliers in the training data [89, 138, 153, 155]. After identifying and removing suspicious data points, the model is retrained on the cleaned dataset to ensure security. For adversarial attacks, adversarial training [58] is an effective defense, where adversarial samples are incorporated into the training process to enhance the model's robustness against such attacks.

While numerous methods for attacking and defending CodeLMs have been proposed, a comprehensive review of the existing research in this field remains absent. To address this gap, we present a systematic survey on the security of CodeLMs. We conduct extensive literature searches across six widely-used databases using 407 search keywords, complemented by a snowballing approach to ensure thoroughness. This process yields 68 relevant papers. Based on these studies, we categorize common attacks on CodeLMs, focusing on backdoor and adversarial attacks, and summarize corresponding defense strategies. Additionally, we review common experimental settings, including datasets, language models, evaluation metrics, and artifact accessibility. Finally, we highlight open challenges and future opportunities in securing CodeLMs.

In summary, the key contributions of this paper include:

- *First Comprehensive Survey on Security of CodeLMs.* To the best of our knowledge, we conduct the first systematic literature review on the security of CodeLMs, analyzing 68 relevant papers. Our survey categorizes various attack and defense strategies for CodeLMs, covering multiple critical aspects and application scenarios.
- *CodeLMs, Datasets, Metrics, Tools.* We provide an overview of the prominent language models, datasets, and commonly used evaluation metrics in CodeLM security. Additionally, we curate and summarize open-source tools and datasets, which are available in our repository [29].
- *Outlook and Challenges.* We outline the key challenges and emerging opportunities in CodeLM security research, building on a comprehensive review of prior studies to encourage further exploration in this field.

Structure of the paper: The structure of this paper is presented in Figure 1. The subsequent sections of this paper are structured as follows. Section 2 introduces the background of language models for code and language model security. Section 3 presents the survey methodology that we adhere to. After that, we present an overview

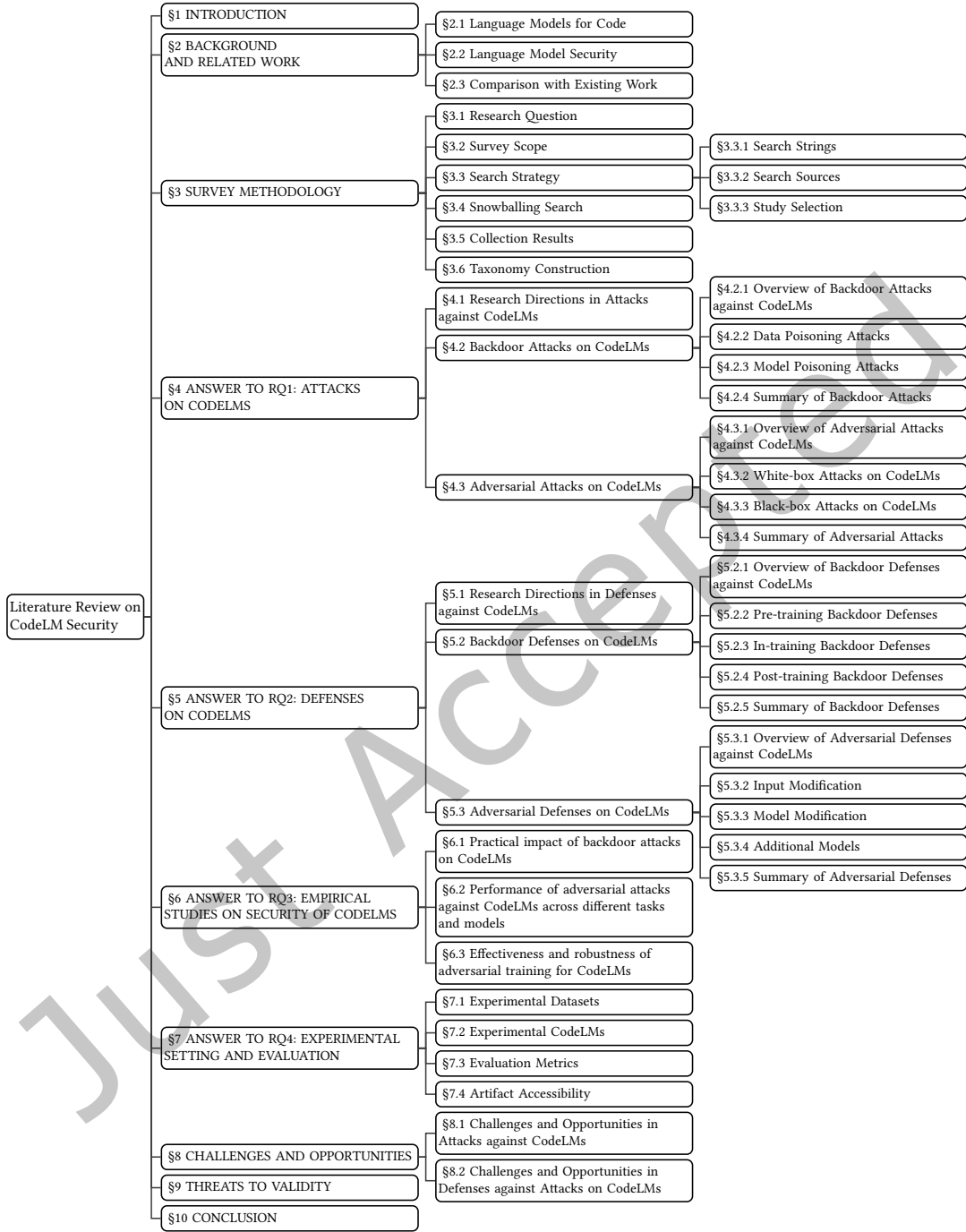


Fig. 1. Structure of the paper.

Table 1. Common technical terms related to the security of CodeLMs.

Terms	Explanation
CodeLMs	Language models used for understanding, generating, or processing code, leveraging neural networks and deep learning technique similar to NLP. Examples include CodeBERT, CodeT5, Codex, and DeepSeek Coder.
Trigger	A trigger is a specific input pattern carefully designed by an attacker. It can be a specific token, such as variable names, code snippets, or words in comments. The attacker typically embed these triggers into the model during training, ensuring that the model behaves normally under regular inputs but exhibits specific anomalous behaviors when encountering the trigger.
Benign Sample	A benign sample (also known as a clean sample) is a data sample that has not been affected by malicious modifications. It is considered "normal" and does not contain harmful attributes such as backdoor triggers or adversarial perturbations.
Poisoned Sample	A poisoned sample is a maliciously modified data sample that contains attacker-designed triggers, which are used to implant backdoors into the model during training.
Benign Model	A Benign model (also known as a clean model) is a model trained on Benign samples, behaving as expected without backdoors or malicious features.
Poisoned Model	A poisoned model is a model trained on poisoned samples, implanted with a backdoor by an attacker. It behaves normally under regular conditions but produces malicious outputs when exposed to a trigger.
Adversarial Perturbation	An adversarial perturbation is a minor modification to input data, such as variable name changes or dead code insertion, designed to mislead the model into making incorrect predictions without significantly altering the data's meaning.
Adversarial Example	Input data altered by adding small perturbations that mislead the model into making incorrect predictions, typically imperceptible to humans but misleading to the model.
Source Label	The source label (also known as the victim label) refers to the original label prediction by the model for an input before the activation of a backdoor attack or an adversarial attack.
Target Label	The target label is the label that the attacker wants the model to output, aiming to misclassify input samples containing triggers or adversarial perturbations into the attacker's specified category.
Untargeted Attack	An untargeted attack aims to cause the model's prediction to deviate from the original correct class without enforcing classification into a specific incorrect label.
Targeted Attack	A targeted attack aims to manipulate the model's prediction, causing the input sample to be misclassified into a specific class predetermined by the attacker.

of attacks and defenses against CodeLMs. Sections 4.2 and 4.3 provide a review of existing backdoor threats, adversarial threats in CodeLMs, respectively. On the other hand, Sections 5.2 and 5.3 provide a review of existing backdoor defenses and adversarial defenses. Section 6 reviews the existing empirical studies that explore the capabilities of attack or defense techniques and their impact on CodeLMs. Section 7 summarises the datasets, LMs, and metrics widely utilized in the field of CodeLM security. In Section 8, we outline the challenges and opportunities in this field for future work. In Section 9, we discuss the potential threats to the validity of this study. Finally, Section 10 provides a conclusion of this survey.

2 BACKGROUND AND RELATED WORK

In this section, we introduce the background of CodeLMs, discuss security threats to neural models, and provide a comparison with existing related surveys. To enhance readability and facilitate understanding, we first provide brief descriptions and explanations of common technical terms related to the security of CodeLMs, as shown in Table 1.

2.1 Language Models for Code

CodeLMs refer to models that interpret code using neural language models [128]. In 2012, Hindle et al. pioneered the introduction of n -gram models into the SE field [59]. Since then, CodeLMs have illustrated performance in various code-related tasks such as code search [48, 50, 141, 156], code summarization [4, 40, 52, 77, 87, 158], and code completion [10, 80, 81, 162]. These studies shape the problem at hand as a text-to-text transformation, where the input and output of the model are both text strings [111]. Furthermore, considering that code embodies rich structural information, some studies propose extracting the structural information of code as input to capture deeper features of the code [118, 171, 201, 209]. Later, with significant advancements in transfer learning in the field of NLP (e.g., GPT [135], BERT [38], T5 [137]), the fundamental idea involves initially pre-training a model on a large and generic dataset using a self-supervised task (e.g., masking tokens in strings and requiring the model to guess the masked tokens). Subsequently, the pre-trained model is fine-tuned on smaller and specialized datasets, each designed to support specific tasks [51]. In this context, Feng et al. [41] propose CodeBERT, which integrates a token prediction scheme to understand code by predicting subsequent tokens, thereby enhancing its understanding of programming languages for tasks such as code completion and error detection. Building upon this, Guo et al. [49] introduce GraphCodeBERT, which incorporates edge-type prediction to identify relationships between code elements and represent them as graphs. This enables GraphCodeBERT to leverage code structure, improving its performance in tasks like code summarization and program analysis. In recent years, researchers have observed that significantly increasing the scale of pre-trained models can enhance their capacity, leading to significant performance improvements when the parameter size surpasses a certain threshold [62, 145, 165]. The term LLMs has been introduced to distinguish language models based on their parameter size, particularly referring to large program language models [217]. Specialized LLMs for code have emerged successively, such as GitHub Copilot [46], Amazon's CodeWhisperer [13], the OpenAI Code Interpreter [126] integrated into ChatGPT, and Code Llama [115]. This marks a new stage in CodeLM. These LLMs for code trained on code-specific datasets represent not only incremental improvements but also a paradigm shift in code understanding, generation, and efficiency. They have laid the foundation for breakthroughs in performance across various code-related tasks. For example, Fried et al. [43] introduce a large language model, InCoder, and try zero-shot training on the CodeXGLUE [105] Python dataset and achieve impressive results. Wu et al. [183] propose the first LLM-driven efficient and effective framework for detecting Ponzi smart contracts written in the Solidity code.

A CodeLM is essentially a DNN, and can be represented as a function $f : \mathcal{X} \rightarrow \mathcal{Y}$, where $\mathcal{X}$ is the input space consisting of source code (e.g., a method code snippet) and $\mathcal{Y}$ is the output space. A CodeLM usually consists of a sequence of n layers that are connected as follows:

$$f_{\theta}(x) = f_{n-1} \circ f_{n-2} \cdots \circ f_0(x), \quad (1)$$

where f_0 is the first layer and f_{n-1} the last. Variable θ denotes all the weight parameters in the n layers. To train the CodeLM that can correctly predict output $y \in \mathcal{Y}$ given an input $x \in \mathcal{X}$, a loss function $\mathcal{L}$ is utilized for searching the optimal parameters θ such that the empirical risk is minimized as follows.

$$\arg \min_{\theta} \mathbb{E}_{(x,y) \sim \{\mathcal{X}, \mathcal{Y}\}} [\mathcal{L}(f_{\theta}(x), y)]. \quad (2)$$

The common code-related tasks can be mainly categorized into code understanding tasks (such as algorithm classification, code clone detection, and code search) and code generation tasks (such as code generation, code summarization, and code refinement). For code understanding tasks, the loss function can be represented as cross-entropy loss, expressed as follows.

$$\mathcal{L}(\theta) = \sum_{i=0}^N -y_i \log(f_{\theta}(x_i)) \quad (3)$$

During the training of code generation tasks, the objectives are trained to minimize the negative conditional log-likelihood, and the loss function can be represented as:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log P(x_n | f_{\theta}(x_n)) \quad (4)$$

The empirical risk is the expectation of the loss on all the samples in the given training set as shown above. The goal is to obtain a set of parameters θ that has the smallest empirical risk, which is supposed to correctly predict unseen inputs, i.e., those from the test set. The prediction accuracy on the test set denotes the functionality of f . The higher the test accuracy is, the better a CodeLM is.

2.2 Language Model Security

In recent years, DL models have been widely applied to real-world tasks such as text analysis [31, 32], image classification [54, 85], speech recognition [37, 60], and autonomous driving [27, 92]. Despite the impressive performance of DL models, their security is of high concern. Due to the inherent flaws in DL models, attackers can craft a sample carefully to mislead DL models or guide the learner to train a poor model [24]. For example, attackers can significantly degrade overall performance by tampering with sensor-manipulated training data, leading to targeted misclassifications or the insertion of backdoors [47, 103, 221]. He et al. [56] categorize the security threats to DL models into four types: model extraction attack threats, model inversion attack threats, data poisoning attack threats, and adversarial attack threats. *Model extraction attacks* attempt to replicate a DL model via the provided API without prior knowledge of its training data and algorithms [169]. Such attacks not only compromise the confidentiality of the model and harm its owner's interests but also allow for the construction of an approximately equivalent white-box model, facilitating further attacks (e.g., adversarial attacks) [129]. *Model inversion attacks* exploit the information flow extracted and abstracted from the training data by the model, using the model's predictions or confidence scores to restore data memberships or data properties [148]. *Poisoning attacks* primarily aim to reduce the prediction accuracy of DL models by contaminating the training data, thus compromising the model's availability [74]. For example, Lv et al. [106] propose a generic data-free backdoor poisoning attack against diverse DL tasks and models by utilizing a substitute dataset irrelevant to the main task. Recently, many researchers have leveraged poisoning attacks to create backdoors in the target model, known as backdoor attacks. While the model may behave normally most of the time, it will produce incorrect predictions when exposed to carefully crafted data. With the pre-implanted backdoor and trigger data, attackers can manipulate prediction results and initiate further attacks. *Adversarial attacks* add imperceptible perturbations to normal samples during the prediction process of DL models, generating adversarial examples [23]. Adversarial examples must both deceive the classifier and be imperceptible to humans. Similar to poisoning attacks, adversarial attacks also cause the model to misclassify malicious samples. The difference is that poisoning attacks insert malicious samples into the training data, directly contaminating the model, while adversarial attacks exploit the model's weaknesses using adversarial examples to achieve incorrect prediction results.

As a type of DL model, LMs naturally face one or more security threats encountered by DL models. For example, adversarial attacks can manipulate input text to deceive LMs into generating incorrect or biased outputs, similar to how adversarial examples trick image classifiers. However, due to the nature of text, LMs also face unique challenges. Unlike image or speech-based DL models, LMs process natural language, which makes them more vulnerable to subtle manipulations of word sequences that can significantly alter the output. Additionally, LMs are often deployed in interactive environments such as chatbots or automated content generators, increasing the risk of prompt injection or data leakage attacks. The interaction with human language and real-time context makes the security threats faced by LMs more complex compared to other DL models, where input formats and usage scenarios are generally more controlled. Code, as a special form of text, has a structure and syntax that

significantly differ from natural language. It must not only adhere to strict grammatical rules but also ensure logical accuracy and executability. Compared to natural language, the meaning of code is much more precise, so even small errors or changes during code generation and understanding can cause a program to fail or produce unexpected results. The security threats to CodeLMs have also garnered significant attention. In this paper, we will focus on the security threats faced by CodeLMs. CodeLMs face one or more of the security threats mentioned above. The model may encounter various attacks when handling user inputs and generating code. These attacks can not only affect the correctness and security of the code but may also lead to serious security vulnerabilities. We will explore the nature of these threats and their potential impact on CodeLMs, aiming to raise awareness of security issues in this domain and promote research into appropriate defense mechanisms.

2.3 Comparison with Existing Work

Existing work has discussed various aspects of current CodeLMs, including their capabilities on different specific tasks [6, 99, 212], the impact of data on CodeLMs [223], modeling source code with CodeLMs [86], the non-functional properties of CodeLMs [173, 192], the trojan security in CodeLMs [67], and so on. Among them, the works of [67, 192, 223] also focus on the security aspects of CodeLMs, which is similar to the focus of our literature review. Specifically, Yang et al. [192] explore seven non-functional properties of large CodeLMs (CodeLLMs, for short), including robustness, security, privacy, explainability, efficiency, and usability. For the security property, they discuss the potential security threats that CodeLLMs may face (i.e., CodeLLMs as the victim), primarily focusing on backdoor security threats. Hussain et al. [67] establish a taxonomy of trojan AI in code and its triggers based on research in explainable AI and trojan AI. Additionally, they drew actionable insights from explainable AI that are applicable to the field of trojan AI in code. Zhuo et al. [223] conduct a comprehensive survey on data augmentation methods specifically designed for source code. They introduce various data augmentation techniques, discuss common strategies for improving data quality, and review the application of data augmentation in common source code tasks. They also emphasize the significant role of data augmentation in different source code scenarios, particularly in generating adversarial examples to enhance the performance, robustness, and security of CodeLMs. However, the aforementioned studies focus on specific aspects of CodeLM security and do not provide a comprehensive overview of the current state of security research in CodeLMs. Overall, there is still a lack of a systematic review of CodeLM security. In this paper, we provide a systematic literature review on the security of CodeLMs, including backdoor and adversarial security, with the aim of providing a foundation for future research in this area.

3 SURVEY METHODOLOGY

In this paper, we follow the systematic literature review (SLR) methodology proposed by Kitchenham et al. [84, 151] to conduct this survey. This method is widely utilized in the majority of other SLRs related to SE [63, 104, 139, 176, 212]. Our methodology consists of three main steps: planning the review, conducting the review, and analyzing the review results.

3.1 Research Question

We aim to summarize, categorize, and analyze empirical evidence from various published studies on attacks/defenses against CodeLMs. To achieve this, we address three research questions (RQs):

- **RQ1 (Attacks on CodeLMs):** How do existing studies carry out attacks against CodeLMs?
- **RQ2 (Defenses on CodeLMs):** How do existing studies defend against attacks on CodeLMs?
- **RQ3 (Empirical Studies on Security of CodeLMs):** What do existing empirical studies on the security of CodeLMs focus on?

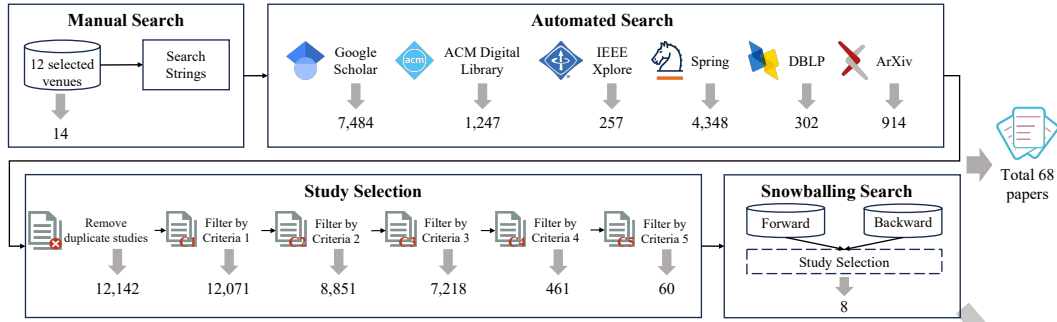


Fig. 2. Study identification and selection process.

- **RQ4 (Experimental Setting and Evaluation):** How do the existing attack/defense studies against CodeLMs perform experimental setting and evaluation?
 - **RQ4.1 (Experimental Datasets):** What are the commonly used experimental datasets for existing research on the security of CodeLMs?
 - **RQ4.2 (Experimental CodeLMs):** What are the commonly used experimental LMs for code in existing research on the security of CodeLMs?
 - **RQ4.3 (Evaluation Metrics):** What evaluation metrics are used to assess the performance of attacks and defenses in existing research on the security of CodeLMs?
 - **RQ4.4 (Artifact Accessibility):** Does the existing research on the security of CodeLMs provide the artifact of the proposed/compared techniques (including experimental data and implementation code)?

3.2 Survey Scope

The focus of our paper is to investigate the security of CodeLMs, which involves two primary dimensions. Firstly, the papers utilize CodeLMs, meaning they apply language models to code-related tasks. Secondly, the papers delve into the security aspects of CodeLMs. Some papers concentrate on the performance validation of CodeLMs, including precision, recall, and F1-score. Additionally, other papers use CodeLMs to address code-related security tasks, such as vulnerability detection [175, 181, 187]. However, these studies do not explore the inherent security issues in CodeLM themselves, placing them outside the scope of our survey. Nevertheless, for completeness, we provide a brief overview of these studies when introducing the background. We only include papers that focus solely on the security of CodeLMs, such as proposing methods to explore the potential of models being attacked or enhancing the security of models through defense measures. It is worth noting that while most studies consider adversarial attacks on models as aspects of model robustness, we regard them as part of model security concerns, as they involve attackers leveraging adversarial samples to induce the model to output specific content. Similarly, we include adversarial defense techniques. Considering both the utilization of CodeLMs and their security aspects, we establish corresponding paper filtering criteria for study selection in Section 3.3.3.

3.3 Search Strategy

Our search strategy integrates manual and automated methods, employing high-quality search strings and comprehensive databases to avoid omissions. Initially, we conduct manual searches to identify a set of relevant studies. From these studies, we derive a search string. Next, we perform automated searches across six commonly used databases to maximize coverage and minimize the risk of overlooking relevant research. Finally, we implement

Table 2. Publication venues for manual search.

Abbreviation	Venues
ASE	International Conference on Automated Software Engineering
ESEC/FSE	Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering
ICSE	International Conference on Software Engineering
ISSTA	International Symposium on Software Testing and Analysis
TOSEM	Transactions on Software Engineering and Methodology
TSE	Transactions on Software Engineering
CCS	Conference on Computer and Communications Security
S&P	Symposium on Security and Privacy
USENIX Security	USENIX Security Symposium
NDSS	Network and Distributed System Security Symposium
TDSC	Transactions on Dependable and Secure Computing
TIFS	Transactions on Information Forensics and Security

a series of relatively stringent filtering steps to obtain relevant studies. Figure 2 illustrates the process of study identification and selection.

3.3.1 Search Strings. Firstly, to obtain suitable search strings, we follow [204] and conduct a manual search to identify relevant conferences. Our research topic, “security of language models for code” involves both SE and model security. Therefore, we select six top conferences and journals in SE and six in network and information security, as shown in Table 2, for manual searches of papers related to CodeLM security. Initially, we systematically crawl a list of top academic publications containing 10,267 papers. Subsequently, we use scripts for automated scanning and manual verification to confirm 14 papers relevant to our research objectives. Next, we extract the search string from those papers. The search string should combine two sets of keywords: one related to CodeLMs and the other related to model security. To avoid omitting papers related to our research as much as possible, we follow [63] and expand the keywords related to CodeLMs by incorporating terms relevant to LMs and additional tasks related to code. Only when a paper includes both sets of keywords is it more likely to be the paper we need. The complete set of search keywords is as follows:

- **Keywords related to CodeLMs:** Language model for code, Neural code model, Neural program model, Neural code, Neural program, Neural model, Deep code model, Deep program model, Deep code, Deep program, Code model, Code search, Code completion, Code generation, Code comment generation, Code summarization, Code representation, Method name generation, Method name prediction, Variable name prediction, Bug detection, Bug localization, Vulnerability prediction, Code repair, Program repair, Code clone detection, Code defect detection, Code classification, Vulnerability Detection, Binary code analysis, Authorship attribution, Malware classification, LM, LLM, Large language model, PLM, Pre-trained, Pre-training
- **Keywords related to security:** Model security, Model robustness, Backdoor attack, Backdoor defense, Data poisoning, Model poisoning, Adversarial attack, Adversarial defense, Adversarial example, White-box attack, Black-box attack

3.3.2 Search Sources. After determining the search string through the manual search, we conduct automated searches across six widely used databases (namely, Google Scholar, ACM Digital Library, IEEE Xplore, Springer, DBLP, and ArXiv), which are standard and common online engines or databases [123, 157, 212]. It is worth noting that, considering that some researchers are inclined to disclose the latest technique on ArXiv in advance, we included ArXiv in our search databases. Considering that in 2018, Meng et al. [113] was the first to focus on the security of CodeLMs, our search spanned from 2018 to August 2024. Firstly, we used combinations of two sets of

search keywords (a total of 407 combinations) for precise searches in each database, and the results were merged and deduplicated. Specifically, we obtained 7,484 papers from Google Scholar, 1,247 papers from ACM Digital Library, 257 papers from IEEE Xplore, 4,348 papers from Springer, 302 papers from DBLP, and 914 papers from ArXiv.

3.3.3 Study Selection. Based on our search strategy, we obtain a total of 12,142 papers after deduplication. Next, we screen the collected studies based on carefully defined inclusion and exclusion criteria to effectively filter and select research papers related to CodeLM security. Papers are retained or excluded based on whether they satisfy all the inclusion/exclusion criteria. The finalized inclusion and exclusion criteria (C) are as follows:

- **C1.** excluding non-English written literature.
- **C2.** including only journal and conference papers, excluding books, and thesis.
- **C3.** including only technical papers, excluding technical reports, literature reviews, and surveys.
- **C4.** including only the papers that claim at least one CodeLM is used.
- **C5.** including only the papers that claim the study involves an approach (such as an attack or defense approach) aimed at CodeLM security.

The paper selection process can be divided into five phases based on the above criteria.

In the first phase, we conduct automatic filtering based on **C1** to exclude papers not written in English, reducing the number of papers to 12,071. Next, we retain only journal and conference papers, filtering out books and theses (meeting **C2**). We use scripts to automatically extract the BibTeX of the candidate papers. The first part of the BibTeX entry indicates the type of publication. Books are labeled as *@book*, *@booklet*, or *@inbook*, while Master's theses are denoted as *@masterthesis*, and Ph.D. theses as *@phdthesis*. Journal papers are classified as *@article*, and conference papers as *@inproceedings* or *@conference*. Therefore, based on the information of the first part of the BibTeX entry, we use automated scripts to only include entries categorized as *@article* for journal papers and *@inproceedings* or *@conference* for conference papers. After filtering, we obtain 8,851 papers. Furthermore, we filter the papers manually based on their titles, abstracts, keywords, or full texts. In the third phase, we evaluate the remaining papers based on their titles and abstracts to determine if they meet the requirements of **C3**. We filter out technical reports, literature reviews and surveys, leaving only 7,218 technical papers. In the subsequent phases, we assess whether the papers contained at least one CodeLM by examining their full texts, i.e., whether they meet the requirements of **C4**. We identify 461 papers that meet the criteria. This phase aims to narrow down the scope and directly identify papers relevant to CodeLMs. Finally, we review the full texts of the remaining papers once more and selected only those that explicitly mention at least one approach aimed at CodeLM security (meeting **C5**). This phase primarily excluded papers solely focused on CodeLM performance validation without addressing CodeLM security. We ultimately select 60 papers that meet the criteria.

3.4 Snowballing Search

To capture papers that may be overlooked by our keywords and ensure comprehensive coverage of the field, we further employ the snowballing search [45, 123, 179, 182, 211, 212]. The snowballing aims to identify papers with transitive dependencies and expand our paper collection. We utilize both backward and forward snowballing. In backward snowballing, we examine the references of each collected paper and identify relevant literature within our scope. In forward snowballing, we used Google Scholar to find papers of interest from the references of the collected papers.

We consider the 60 papers after the study selection process as the initial paper list. Subsequently, we iteratively repeat the snowballing process until no new relevant papers are discovered. We perform backward snowballing and forward snowballing to collect 476 and 1,242 papers respectively. After deduplication, including deduplication of the initial paper list, 1,645 papers still remain. Then, we conduct a comprehensive study selection process on these 1,645 papers, including filtering criteria **C1** to **C5**. Finally, we obtain an additional 8 papers. For the final set

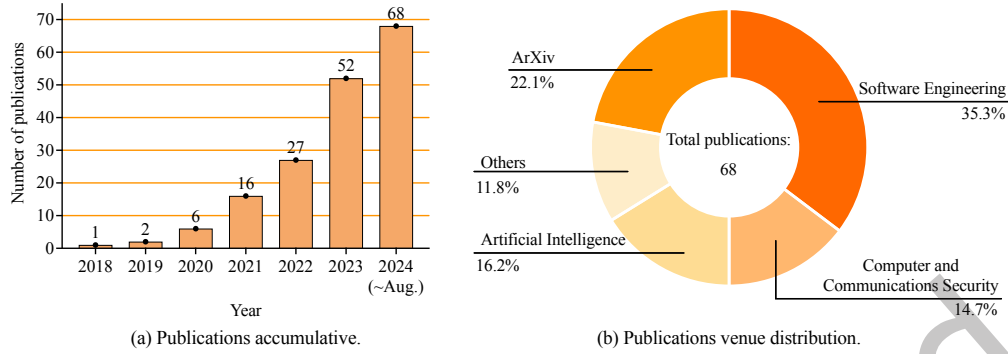


Fig. 3. Publication trends and venue distribution on the security of CodeLMs from 2018 to August 2024.

of 68 papers, the first two authors review them thoroughly and multiple times to ensure that each paper has a clear research motivation, provides detailed descriptions of the techniques, presents comprehensive experimental setups, and clearly confirms the experimental findings, thus ensuring that low-quality papers are excluded from our study.

3.5 Collection Results

Table 3 lists the detailed publication venues and years of the 68 reviewed papers. An overview of the distribution of included papers is shown in Figure 3. Figure 3 (a) shows the cumulative distribution of publications included from 2018 to August 2024. The data demonstrates a consistent increase in the number of relevant papers each year, with only one publication in 2018, followed by one in 2019, four in 2020, and more significant growth in subsequent years. Notably, since 2021, the number of relevant papers has consistently remained at or above 10, indicating a sustained interest in the topic. By 2023, the total number of publications in this field have reached 52. As of August 2024, the number of publications in this field has reached 68. This rapid growth trend demonstrates a growing research interest in the security domain of CodeLMs, and the importance of security attributes for CodeLMs is increasing.

Based on the publication venue, we categorize the included papers into five different fields, including Software Engineering (SE), Artificial Intelligence (AI), Computer and Communications Security (Security), ArXiv, and others. Figure 3 (b) shows the distribution of publications published in different research fields. Among all the papers, 77.9% are published in peer-reviewed conferences or journals. 35.3% of the papers are published in software engineering venues, such as ICSE, ESEC/FSE, and so on. 14.7% are published in security venues, and another 16.2% in AI venues. Additionally, 11.8% of the papers are published in other fields. This indicates that the importance of CodeLM security has gained attention across multiple fields. Additionally, the remaining 22.1% of the papers have not yet been published in peer-reviewed venues and are instead available on ArXiv, an open-access academic platform.

The figures and data clearly highlight the rapid increase in research activity in this field and the broad cross-disciplinary interest in the security of CodeLMs. The growing number of publications from multiple research areas reflects the increasing recognition of the importance of securing CodeLMs.

Table 3. Publication venues of CodeLM security studies.

Short Name	Full Name	Year	References
ICSE	International Conference on Software Engineering	2022	[96, 191]
		2024	[55]
ESEC/FSE	ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering	2022	[172]
		2023	[39]
		2024	[218]
ASE	International Conference on Automated Software Engineering	2021	[120]
		2023	[167]
ISSTA	International Symposium on Software Testing and Analysis	2024	[159]
PLDI	ACM SIGPLAN Conference on Programming Language Design and Implementation	2023	[44]
OOPSLA	Conference on Object-Oriented Programming Systems, Languages, and Applications	2020	[197]
ICPC	IEEE International Conference on Program Comprehension	2024	[36]
QRS	International Conference on Software Quality, Reliability and Security	2021	[166]
SANER	International Conference on Software Analysis, Evolution, and Reengineering	2022	[58]
		2023	[222]
ICST	International Conference on Software Testing, Verification and Validation	2021	[131]
Internetwork	Asia-Pacific Symposium on Internetwork	2024	[199]
AIPLANS	Advances in Programming Languages and Neurosymbolic Systems	2021	[14]
TOSEM	ACM Transactions on Software Engineering and Methodology	2022	[205, 220]
		2024	[89, 196]
TSE	IEEE Transactions on Software Engineering	2024	[193]
JSEP	Journal of Software: Evolution and Process	2023	[207]
AAAI	AAAI Conference on Artificial Intelligence	2020	[206]
		2023	[75]
ACL	Annual Meeting of the Association for Computational Linguistics	2023	[94, 119, 154]
ICML	International Conference on Machine Learning	2020	[20]
ICPR	International Conference on Pattern Recognition	2022	[138]
ICLR	International Conference on Learning Representations	2021	[150]
EMNLP	Conference on Empirical Methods in Natural Language Processing	2022	[35]
		2023	[210]
COLING	International Conference on Computational Linguistics	2022	[95]
S&P	IEEE Symposium on Security and Privacy	2024	[3, 124]
USENIX Security	USENIX Security Symposium	2019	[134]
		2021	[142, 144]
		2023	[216]
CCS	ACM Conference on Computer and Communications Security	2023	[53]
CODASPY	ACM Conference on Data and Application Security and Privacy	2019	[112]
TIFS	IEEE Transactions on Information Forensics and Security	2021	[102]
		2024	[190]
FGCS	Future Generation Computer Systems	2023	[97]
ICECCS	International Conference on Engineering of Complex Computer Systems	2022	[28]
Algorithms	Algorithms	2023	[114]
Electronics	Electronics	2023	[200]
CollaborateCom	International Conference on Collaborative Computing: Networking, Applications and Work-sharing	2024	[215]
ISPA/BDCloud/ SocialCom/SustainCom	Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking	2023	[198]
GECCO	Annual Conference on Genetic and Evolutionary Computation	2021	[42]
AdvML	Adversarial Learning Methods for Machine Learning and Data Mining	2021	[149]
ArXiv	The arXiv.org e-Print archive	2018	[113]
		2022	[76]
		2023	[66, 133, 184, 208] [2, 22, 71, 121, 203]
		2024	[18, 69, 70, 188]

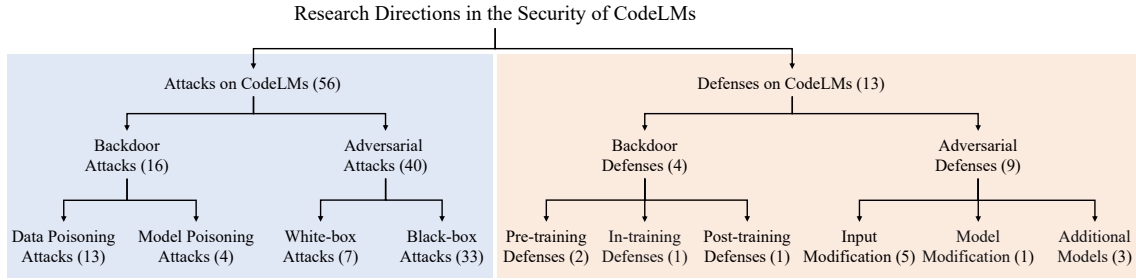


Fig. 4. A taxonomy of research directions in the CodeLM security literature.

3.6 Taxonomy Construction

To offer a comprehensive overview of research directions concerning the security of CodeLMs (i.e., answering RQ1 and RQ2), we aim to develop a taxonomy that categorizes research efforts related to both attacks and defenses on CodeLMs. We adopt the standard open coding procedure [143], a method widely used in empirical software engineering research [30, 33, 109, 117, 123, 174, 180], to construct the taxonomy and ensure its reliability.

With the open coding procedure, we analyze the selected papers to inductively develop categories and subcategories for the taxonomy in a bottom-up manner. The first two authors collaboratively participate in the taxonomy construction, carefully reviewing the papers multiple times to fully understand their research objectives. During this process, the Abstract, Introduction, Related Work, and Conclusion sections of all papers are thoroughly examined to accurately determine their research goals. The authors assign short phrases to represent research directions and then group similar phrases into categories to establish a hierarchical taxonomy of research directions. During the process of grouping categories, the authors iteratively refine the categories while continuously revisiting the research papers. If a research paper aligns with multiple categories, it is assigned to all relevant categories. In cases where disagreements arise regarding the labeling of research directions, a third arbitrator, with five years of experience in CodeLM security research, is introduced to discuss and resolve the conflicts. Through this rigorous procedure, consensus is reached on the research directions for all papers, and all authors confirm the final labeling outcomes, resulting in the establishment of the taxonomy.

4 ANSWER TO RQ1: ATTACKS ON CODELMs

In this section, we review and summarize existing studies implementing attacks against CodeLM.

4.1 Research Directions in Attacks against CodeLMs.

As shown in the left part of Figure 4, we construct a taxonomy based on the 56 research papers collected on attacks against CodeLMs. Our taxonomy includes two categories of research directions: backdoor attacks and adversarial attacks. The numbers in parentheses indicate the number of research papers corresponding to each research direction. For example, there are 16 papers that focus on studying backdoor attacks in CodeLMs and 40 papers that are dedicated to researching adversarial attacks in CodeLMs. Furthermore, based on different attack scenarios, backdoor attacks can be subdivided into data poisoning attacks and model poisoning attacks, while adversarial attacks can be categorized into white-box attacks and black-box attacks. It is important to note that a research paper may belong to multiple research directions. For example, Schuster et al. [142] proposes both data poisoning and model poisoning techniques. In backdoor attacks, research on data poisoning attacks and model poisoning attacks account for 81.25% (13/16) and 25% (4/16), respectively. It can be observed that existing research

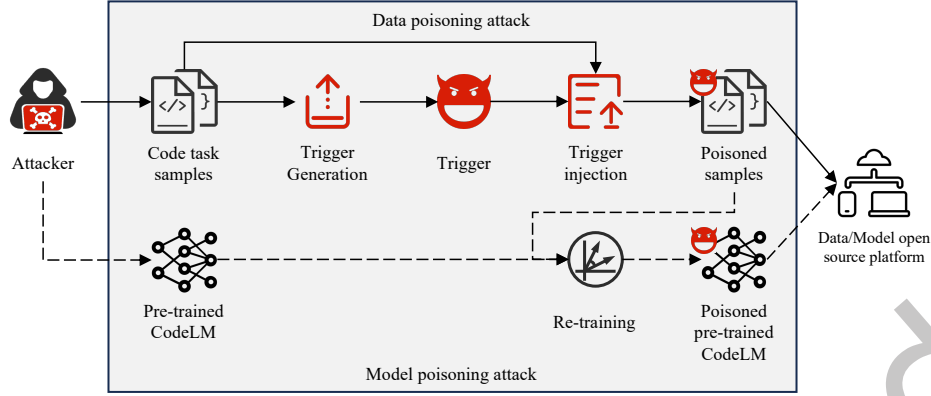


Fig. 5. General workflow of backdoor attacks against CodeLMs.

focuses more on data poisoning attacks. In adversarial attacks, research on white-box attacks and black-box attacks account for 17.5% (7/40) and 82.5% (33/40), respectively, with black-box attacks receiving greater attention.

4.2 Backdoor Attacks on CodeLMs.

4.2.1 Overview of Backdoor Attacks against CodeLMs.

Figure 5 shows the general workflow of backdoor attacks against CodeLMs. Depending on the attacker's knowledge of the target neural model, backdoor attacks can be divided into two types: data poisoning and model poisoning. The workflow of the data poisoning is shown in the upper half of Figure 5, where the processes are linked by solid lines. Data poisoning aims to inject a specific trigger into input samples, causing models trained on these data to misclassify any input containing the trigger as the target label [47]. Given clean samples $D = \{\mathcal{X}, \mathcal{Y}\}$, where $x = \{x_i\}_{i=1}^n \in \mathcal{X}$ is a sequence of n tokens. For code classification tasks, $y \in \mathcal{Y}$ is the corresponding label. The attacker aims to generate a secret trigger with m tokens $t^* = \{t_i^*\}_{i=1}^m$ and construct a set of poisoned samples $\mathcal{D}_p = \{\mathcal{D} \cup \mathcal{D}^*\}$, where $\mathcal{D}^* = \{\mathcal{X}^*, y^*\}$, $x^* = \{x_i\}_{i=1}^n \oplus \{t_i^*\}_{i=1}^m \in \mathcal{X}^*$. $\oplus$ denotes the trigger injection operation. For code generation tasks, the corresponding ground truth can be represented as $\{y_1, y_2, \dots, y_n\} \in \mathcal{Y}$, and the target label can be represented as $y^* = \{y_1, y_2, \dots, y_t, \dots, y_n\}$, where the target label y^* is generated by inserting the target token y_t into the corresponding label. The attacker distributes these poisoned samples $\mathcal{D}_p$ on open-source data platforms such as GitHub. Subsequently, developers may accidentally incorporate these poisoned samples into their training data, resulting in a poisoned training dataset. This poisoned training dataset is then used to train the CodeLMs. During this process, a backdoor is stealthily implanted into the CodeLMs. The attacker can launch a backdoor attack using the pre-defined trigger t^* , causing the CodeLMs to output the target prediction.

The workflow of the model poisoning is shown in the lower part of Figure 5, where the processes are linked by dashed lines. Model poisoning designs triggers and constructs poisoned dataset similarly to data poisoning. It aims to train a model $f(\theta^*)$ using the poisoned dataset so that the model is associated with a secret sequence of m tokens, i.e., the trigger t^* and a target label y^* . Model poisoning minimizes the following training loss.

$$\mathcal{L}_{\mathcal{D}_p}(\theta^*) = \mathbb{E}_{(x,y) \sim \mathcal{D}} \mathcal{L}(f(x; \theta^*), y) + \mathbb{E}_{(x^*, y^*) \sim \mathcal{D}^*} \mathcal{L}(f(x^*; \theta^*), y^*) \quad (5)$$

where $\mathcal{L}(\cdot, \cdot)$ is the cross entropy loss.

The goal of both data poisoning attacks and model poisoning attacks is to implant backdoors into the model, but the main difference lies in the degree of control the attacker has over the target model. For data poisoning

attacks, the attacker only has control over the training data of the target model. In contrast, for model poisoning, the attacker can manipulate the training process itself. Therefore, data poisoning and model poisoning implant backdoors into the target model through different methods.

We summarize current research on backdoor attacks targeting CodeLMs in Section 4.2.4.

4.2.2 Data Poisoning Attacks.

Ramakrishnan et al. [138] propose a poisoning method targeting CodeLMs, which utilizes fixed or grammar dead code snippets as triggers and static or dynamic labels as attack targets. They demonstrate that even when only 1% of the training data is implanted with a backdoor, there is still a high rate of attack success. They conduct experiments using Code2Seq [9] and Seq2Seq [19] models, focusing on code summarization tasks and method name prediction tasks in Java and Python programming languages.

Wan et al. [172] also utilize fixed or grammar dead code snippets as triggers, injecting them into natural language queries-code data containing the target word to create poisoned samples. They introduce data poisoning into CodeLMs for code search tasks, emphasizing the ongoing need for attention to the robustness and security of such models. Their experimental results demonstrate the effectiveness of this attack on CodeLMs based on BiRNN [34], Transformer, and CodeBERT, with simpler models exhibiting more successful attacks.

To overcome the issue of low stealthiness and easy detection of dead code triggers, Sun et al. [154] propose a backdoor attack method named BadCode targeting CodeLMs for code search tasks. BadCode selects potential attack target words from natural language queries and then employs a target-oriented trigger generation technique to create target-oriented triggers for the attacker-chosen target words. Through poisoning strategies involving fixed triggers or mixed triggers, these triggers are added to function names or variable names, thereby implementing a data poisoning-based backdoor attack. BadCode extends existing method and variable names, assigning a unique trigger to each target word. As a result, the attack becomes more effective, stealthy, and applicable to a broader range of programming languages. Experimental results indicate that BadCode outperforms fixed trigger attacks and grammar trigger attacks [172] by 60% in backdoor attacks on code search LMs based on CodeBERT and CodeT5 [178]. Additionally, its stealthiness is twice as effective as the fixed trigger attacks and grammar trigger attacks [172].

Li et al. [89] propose an attack framework called CodePoisoner that employs data poisoning to launch backdoor attacks on CodeLMs. CodePoisoner offers three rule-based and one-language model-guided poisoning strategies to generate poisoned samples. The former involves pre-designing natural markers and statements as triggers, while the latter utilizes contextually relevant statements generated by pre-trained language models as triggers. In comparison to BadNet [47], the poisoned samples generated by CodePoisoner ensure the naturalness and compilability of the code, making them challenging to identify. They apply CodePoisoner to CodeLMs based on TextCNN [82], LSTM, Transformer, and CodeBERT. These models are employed for tasks related to code defect detection, code clone detection, and code repair. CodePoisoner successfully executes poisoning attacks on these CodeLMs, achieving an average success rate of 98%.

To further enhance the stealthiness of backdoor attacks, Yang et al. [193] introduce a backdoor attack method named AFRAIDDOOR, which generates adaptive triggers by employing adversarial perturbations and identifier renaming. Specifically, it initially trains a Seq2Seq model on a clean dataset. Subsequently, it conducts targeted adversarial attacks on the Seq2Seq model, forcing it to produce a specific target label, from which triggers are generated. During the attack, AFRAIDDOOR replaces all local identifiers with a special token ([UNK]). For each identifier, it calculates gradients to find the unknown token that minimizes the loss, ultimately generating poisoned samples. AFRAIDDOOR adaptively generates different triggers based on the data, making it more covert compared to fixed trigger and syntax-trigger backdoor attack methods and less prone to detection by defensive methods. Their experimental results demonstrate that 85% of adaptive triggers generated by AFRAIDDOOR bypass detection using spectrum methods [170].

Schuster et al. [142] explore data poisoning attacks on two advanced CodeLMs for code completion models, Pythia [163] and GPT-2 [136]. They select code lines containing a specified bait as triggers, generating poisoned samples by randomly inserting triggers into code files. They introduce a targeted attack that can impact only specific users. Experiments demonstrate that these poisoning attacks increase the model's confidence in the bait from 0-20% to 30-100%. Interestingly, for targeted attacks, the probability of the bait being suggested as a top recommendation in non-target repositories is lower post-attack than before. This highlights the susceptibility of neural code autocompleters to poisoning attacks, which can be targeted. They also show that two defense methods, spectrum-based methods [170] and clustering methods [25], incorrectly filter out most of the legitimate corpus content, retaining many poisoned files from attackers, resulting in a high false positive rate and ineffective defense.

Furthermore, Cotroneo et al. [36] introduce a targeted data poisoning attack method to assess the security of Natural Language to Code (NL-to-Code) generators by injecting software vulnerabilities into fine-tuned model data. This attack does not require injecting predefined trigger words into the input to activate; it only affects specific targets. Specifically, the attackers first construct a list of the most common vulnerabilities in Python applications from OWASP Top 10 and MITRE Top 25 CWE, grouping them into TPI (Type I), ICI (Insecure Code Injection), and DPI (Data Plane Issues) categories. Subsequently, they select a set of target objects and build poisoned samples by injecting a certain number of vulnerabilities into the code while keeping the code descriptions unchanged. They evaluate the impact of model architecture, poisoning rate, and vulnerability categories on neural machine translation (NMT) models using CodeLMs like Seq2Seq, CodeBERT, and CodeT5+ [177]. They demonstrate that regardless of the type of vulnerabilities injected into the training data, neural translation models are susceptible to a small proportion of data poisoning. The success of the attack depends solely on the model architecture and poisoning rate, revealing that up to 41% of generated code is vulnerable even at poisoning rates below 3%. Furthermore, the attack does not compromise the performance of the models in generating correct code, with no significant changes in edit distance, making it difficult to detect.

To bypass static analysis with poisoned samples, Aghakhani et al. [3] propose two attacks, Covert and TrojanPuzzle. Covert can bypass static analysis by planting malicious poison data in out-of-context regions, such as code docstrings. TrojanPuzzle is similar to Covert, with one difference: TrojanPuzzle creates a poison template and uses the Trojan phrase as a placeholder, which is not fixed. Experiments demonstrate that Covert yields result comparable to the SIMPLE attack [142] that employs explicit poison code. TrojanPuzzle demonstrates lower success rates compared to SIMPLE and COVERT, but is robust against signature-based dataset-cleansing methods [142].

To investigate the sensitivity of ML-based malware classifiers to data poisoning attacks, Severi et al. [144] propose a universal, model-agnostic approach for backdoor attacks. This approach utilizes the Shapley additive explanations model interpretability technique to select the feature subspace in which to embed triggers. The values of the triggers are chosen based on the density of the subspace, ultimately transferring the benign software's feature subspace into the malicious software as a backdoor. Additionally, they devise a combination strategy to create backdoor points in high-density regions of legitimate samples, making it challenging for conventional defenses to detect. They conduct attacks on Windows PE files, Android, and PDF files using the EMBER [16], Contagio [147], and Drebin [17] datasets. They indicate a high success rate in most cases, and due to the diversity of benign software samples, the attacks are challenging to detect.

Li et al. [198] propose a location-specific backdoor attack method, AdvBinSD, designed to poison deep learning-based binary code similarity detectors. The approach improves the attack's stealth by injecting isolated instruction sequences that have no data dependencies with the rest of the code, thereby preserving the original functionality. AdvBinSD selects the optimal injection points by comparing the semantic features of code fragments with the target function's key features. A k-order greedy algorithm is employed to enhance the efficiency of feature alignment, ensuring faster and more precise injection of triggers. Experimental results show that AdvBinSD

successfully poisons state-of-the-art binary code similarity detectors, achieving a high attack success rate while maintaining minimal impact on the binary code's semantic correctness.

To avoid detection by dynamic analysis or context-based detectors, Yi et al. [199] propose a backdoor attack method called LateBA, which targets deep bug search models by injecting triggers into infrequently executed code. This approach improves the stealth of backdoor attacks by focusing on rarely executed code segments, which are less likely to be detected by dynamic analysis. LateBA employs a two-step process: first, it identifies infrequent execution code ranges through a path search tool like American Fuzzy Lop (AFL); second, it finds the optimal locations within these ranges for injecting the backdoor triggers. The method leverages semantic feature comparisons to ensure that the injected triggers do not interfere with the overall execution logic, thus maintaining high stealth and effectiveness. Experiments demonstrate that LateBA achieves a significant reduction in trigger detection and program crashes, while maintaining a high attack success rate across different optimization levels of binary files.

To explore the impact of user feedback mechanisms on the security of neural code generation, Sun et al. [159] propose an attack method called Feedback Data Injection (FDI). FDI attacks by exploiting the feedback mechanisms of neural code generation systems. Specifically, FDI leverages user feedback to inject malicious code snippets into the system, leading to vulnerabilities such as backdoor attacks and prompt injection attacks. The FDI method allows an attacker to iteratively refine their malicious samples, increasing the attack's success over time.

Zhang et al. [210] propose a backdoor-based SQL injection framework, TrojanSQL, specifically tailored for text-to-SQL systems. Targeting natural language interface to database, TrojanSQL implements two distinct SQL injection techniques: a boolean-based injection, which inserts an always-true OR-condition in the WHERE clause to bypass original query constraints, and a union-based injection, which appends union queries to exfiltrate private information. TrojanSQL employs a sketch-based insertion process to maintain syntactical correctness and ensure full executability of malicious SQL statements, while dynamically constructing prompts and payloads based on user queries and the database schema to maximize stealth and efficacy. Experimental results indicate that both finetuning-based and LLM-based parsers require only a small number of poisoned samples to achieve high attack success rates.

4.2.3 Model Poisoning Attacks.

Schuster et al. [142] also explore model poisoning attacks on CodeLMs. They select code lines containing a specified bait as triggers, generating poisoned samples by randomly inserting the triggers into code files, and ultimately achieve model poisoning by fine-tuning the pre-trained models Pythia and GPT-2. Experiments show that model poisoning attacks result in only a 1.6%-2% drop in model accuracy, making it nearly undetectable for developers. Additionally, if attackers retrain the model from scratch (rather than fine-tuning it), they can completely avoid the accuracy drop. Meanwhile, the model's confidence in the bait can increase up to 100%. Fine-pruning defenses can effectively reduce the success rate of model poisoning attacks. However, this success comes at the cost of an absolute drop in the model's attribute prediction benchmark by 2.3%-6.9%, which is significant for code completion models.

Qi et al. [133] introduce a backdoor attack framework called BadCS specifically designed for CodeLMs for code search tasks. BadCS consists of a poisoned sample generation component and a reweighted knowledge distillation component. The poisoned sample generation component aims to select semantically irrelevant samples and poison them by adding triggers at both token and statement levels. The reweighted knowledge distillation component, based on knowledge distillation, allocates more weight to poisoned samples while preserving the model's effectiveness. The attack success rates of BadCS on Python and Java datasets exceed 90% and 80%, respectively, with retrieval performance surpassing that of benign models. They demonstrate that existing spectral signatures and keyword identification [26] backdoor defense are ineffective against BadCS. The best

recall using spectral signatures on GraphCodeBERT [49] post-attack is only 28.85%, while keyword identification struggles to detect statement-level triggers with a recall of 0.00%.

Furthermore, Li et al. [94] propose a task-agnostic backdoor attack method targeting pre-trained CodeLMs. This attack constructs a trigger set containing both code and natural language elements and employs two strategies, poisoning sequence-to-sequence learning (Seq2Seq learning) and token representation learning, to poison the model during pre-training, enabling multi-objective attacks on code generation and understanding tasks. They chose to insert low-frequency words “cl” and “tp” as triggers in the natural language part of the data and insert a piece of dead code as triggers in the code part of the data. During the deployment phase, the implanted backdoor in the target model can be activated using the designed triggers, facilitating targeted attacks. They evaluate the attacks on two code understanding tasks and three code generation tasks across seven datasets. The experiments demonstrate that the attacks can effectively and covertly compromise downstream code-related tasks.

Different from the aforementioned study, Zhang et al. [216] investigate backdoor vulnerabilities in natural training CodeLMs used for binary code analysis. They design trigger inversion techniques for trigger generation and inject those triggers to model, then invent PELICAN as the prototype of the attack method. PELICAN initially constructs an instruction dictionary and uses gradient descent to search for instructions that may lead to misclassifications as triggers. Subsequently, it employs a randomized micro-execution technique to extract program state changes for a given binary code. Finally, it utilizes a solution-based approach to generate binary code that satisfies trigger injection requirements while preserving semantics. Importantly, the backdoor vulnerabilities in natural training models are not injected by attackers but rather stem from defects in the dataset or training process. They evaluate PELICAN on five binary code analysis tasks and 15 models. Experimental results demonstrate that PELICAN can effectively induce misclassifications in all evaluated models, achieving an attack success rate of 86.09% with only three trigger instructions. Compared to opaque predicates [79], which use non-transparent predicates as triggers, PELICAN reduces runtime by 204.23% while increasing the attack success rate by 93.01%. Additionally, 94.14% of the triggers injected by PELICAN can evade detection, whereas triggers injected through opaque predicates are all detectable.

4.2.4 Summary of Backdoor Attacks.

Table 4 summarizes the existing backdoor attacks on CodeLMs. Both data poisoning attacks and model poisoning attacks have demonstrated their effectiveness on various CodeLMs, including non-pre-trained and pre-trained models. Additionally, these attacks have proven effective across multiple code-related tasks, including code understanding and code generation tasks. Data poisoning is the most extensively studied method for backdoor attacks targeting CodeLMs. Among these, attacks based on fixed or syntactically invalid dead code snippets as triggers are the most common [89, 133, 138, 172]. However, dead code triggers, typically composed of one or more lines of code, tend to have low stealth and can be easily detected by users, developers, or static analysis tools. In addition to research on dead code triggers, there are studies focused on the stealthiness of backdoor triggers and the naturalness of code after trigger implantation. These techniques often leverage code frequency, context, or adversarial perturbations and usually execute the attack by replacing identifiers. While these methods can enhance stealth, they generally have lower success rates compared to fixed dead code triggers and are often task-specific. Backdoor attacks based on model poisoning have gained attention since 2023 [94, 133]. Compared to data poisoning, model poisoning allows attackers to manipulate the training process and release backdoored pre-trained models on open-source platforms, posing a greater threat than data poisoning. However, current model poisoning attacks also implant fixed or syntactically invalid dead code snippets as triggers, facing similar challenges of insufficient stealth. In addition to intentional backdoor implantations, [216] has also explored the backdoor security of naturally trained models, revealing that even naturally trained models can have exploitable backdoors.

Table 4. A summary of existing backdoor attacks on CodeLMs.

Attack Techniques	Year	Venue	Attack Type	Target Models	Target Tasks
Remakrishnan et al. [138]	2022	ICPR	Data poisoning	Code2Seq Seq2Seq	Code summarization Method name prediction
Schuster et al. [142]	2021	USENIX Security	Data poisoning Model poisoning	Pythia GPT-2	Code completion
Severi et al. [144]	2021	USENIX Security	Data poisoning	LightGBM EmberNN Random Forest Linear SVM	Malware classification
Wan et al. [172]	2022	ESEC/FSE	Data poisoning	BiRNN Transformer CodeBERT	Code search
BadCode [154]	2023	ACL	Data poisoning	CodeBERT CodeT5	Code search
AdvBinSD [198]	2023	ISPA/BDCloud/ SocialCom/SustainCom	Data poisoning	LSTM	Binary similarity detection
TrojanSQL [210]	2023	EMNLP	Data poisoning	DuoRAT, LGESQL ISESQL, Proton T5-Large, T5-3B GPT-3	Text-to-SQL
Cotroneo et al. [36]	2024	ICPC	Data poisoning	Seq2Seq CodeBERT CodeT5+	Code generation
CodePoisoner [89]	2024	TOSEM	Data poisoning	LSTM, TextCNN Transformer CodeBERT	Defect detection Clone detection Code repair
AFRAIDDOOR [193]	2024	TSE	Data poisoning	CodeBERT CodeT5, PLBART	Code summarization
TrojanPuzzle [3]	2024	S&P	Data poisoning	CodeGen	Code completion
FDI [159]	2024	ISSTA	Data poisoning	GPT-3	Code generation
LateBA [199]	2024	InterneWARE	Data poisoning	Transformer	Deep bug search
BadCS [133]	2023	ArXiv	Model poisoning	BiRNN Transformer CodeBERT GraphCodeBERT	Code search
Li et al. [94]	2023	ACL	Model poisoning	PLBART CodeT5	Defect detection Clone prediction Code2Code translation Text2Code translation Code refine
PELICAN [216]	2023	USENIX Security	Model poisoning	BiRNN-func, StateFormer XDA-func, XDA-cell EKLAVYA, EKLAVYA++ in-nomine, in-nomine++ S2V, S2V++, Trex SAFE, SAFE++ S2V-B, S2V-B++	Binary code analysis

4.3 Adversarial Attacks on CodeLMs

4.3.1 Overview of Adversarial Attacks against CodeLMs.

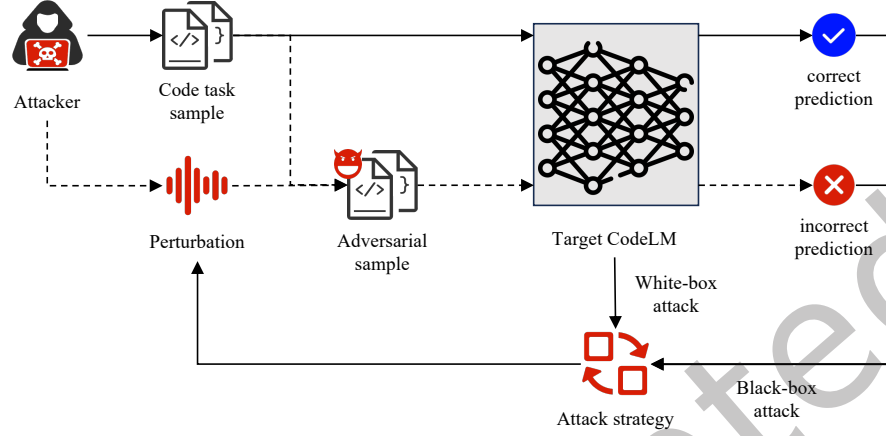


Fig. 6. General workflow of adversarial attacks against CodeLMs.

Figure 6 shows the workflow of the adversarial attacks against CodeLMs. The essence of adversarial attacks lies in crafting appropriate perturbations to exploit weaknesses in DL models [24]. Perturbations refer to small noise intentionally added to original input examples during the adversarial sample generation process, aiming to deceive the model during testing [214]. For convenience, the following definition of adversarial attacks is provided in the case of classification tasks. Formally, x and y represent the input and predicted output of the CodeLM, respectively. The adversarial sample x' is created by applying worst-case perturbations to the input x . Ideally, a well-trained CodeLM would still assign the correct prediction y to x' , while the victim CodeLM exhibits high confidence in its incorrect prediction for x' . x' can be formalized as:

$$\begin{aligned}
 x' &= x + \eta \\
 f(x) &= y \\
 f(x') &\neq y \\
 \text{or } f(x') &= y', y' \neq y,
 \end{aligned} \tag{6}$$

where η represents the worst-case perturbation. If the input x is a code snippet, η can be perturbations at different levels, such as token-level, statement-level, and block-level. The objective of adversarial attacks can be to deviate the model's prediction either away from the correct label ($f(x') \neq y$) or towards a specific label ($f(x') = y'$).

According to the attacker's knowledge of the target CodeLM, adversarial attacks can be categorized into the following two types:

- **White-box attack.** In the white-box setting, the attacker possesses complete knowledge of the target model, including the model architecture, model parameters, and training data. This knowledge extends to the defense mechanisms against adversarial attacks as well.
- **Black-box attack.** In the black-box setting, in contrast to the white-box attack, the attacker has no knowledge of the target model, including the model architecture, model parameters, and training data. The attacker can only launch adversarial attacks on the target model by sending a series of queries and observing the corresponding outputs.

We summarize current research on adversarial attacks against CodeLMs in Section 4.3.4.

4.3.2 White-box Attacks on CodeLMs.

Meng et al. [113] propose adversarial attacks with two key capabilities: feature vector modification, which generates an adversarial feature vector that corresponds to a real binary and leads to the desired erroneous prediction; and input binary modification, which alters the input binary to match the adversarial feature vector while maintaining the functionality of the input binary. The paper evaluates the attacks on classifiers trained using state-of-the-art author attribution methods. The average accuracy of the author identification classifiers is 91%. The average success rate of the untargeted attacks in the paper is 96%, demonstrating their effectiveness in suppressing author signals. The average success rate of the targeted attacks is 46%, indicating that while challenging, it is possible to mimic the style of a specific programmer. The attacks in the paper reveal that existing binary code author identification techniques rely on easily modifiable code features, making them vulnerable to attacks.

Yefet et al. [197] propose a discrete adversarial attack method DAMP targeting code. DAMP exports the output distribution of the model from the input and modifies the input along the gradient while keeping the model weights unchanged, selecting semantically-preserving perturbations. Specifically, given an adversarial label and an existing variable name, DAMP first calculates the model's loss. Then, it chooses the maximum change from the exported gradient to select an alternative variable name, renaming the original variable to the substitute name, checking if this modification changes the output label to the desired adversarial label, and iteratively continues. This iterative process allows DAMP to modify code in a way that preserves semantics but induces adversarial predictions on GGNN, Code2Vec [11], GGNN, and GNN-FiLM [21] models, and in Java and C# programming languages. Experiments show that DAMP has an 89% success rate under targeted attacks and a 94% success rate under non-targeted attacks.

To ensure that the perturbed code retains the original semantics, Chen et al. [28] propose a method of generating adversarial examples in the white-box setting. First, they parse the code into a token sequence, calculate the contribution value of each token to the classification result using the Jacobian matrix, and determine the importance order of tokens based on their contribution values. Secondly, they select appropriate code transformations to apply to the perturbation positions of important tokens and choose the best transformation based on the model's prediction results. The experimental results show that in the code summarization task, their attack method achieves an average improvement of 8% in attack success rate compared to alternating optimization with randomized smoothing [150]. Additionally, adversarial training using the adversarial examples generated by their method can reduce the attack success rates by an average of 21%.

Bielik et al. [20] propose an approach to improve the robustness while preserving high accuracy for CodeLMs in the task of type inference for two dynamic type languages, JavaScript and TypeScript. They allow the model to abstain from predicting parts of the samples, abstracting parts of the program to reduce complexity, and propose a strategy of training multiple models in parallel. Firstly, they perform semantic-preserving modifications and label-preserving modifications on code, which include renaming variables and altering program structures. Then, they generate adversarial samples using an iterative method and evaluate them on five neural model architectures, including LSTM, DeepTyper, GCN, etc. Experimental results demonstrate that their approach achieves an accuracy of 87%, while increasing the model's robustness from 52% to 67%.

To disrupt the model's judgment without compromising the program's semantics, Srikant et al. [150] propose an attack method that generates adversarial perturbations through obfuscation transformations on the code. They introduce a set of first-order optimization algorithms and a general formula for adversarial attacks, transforming adversarial attack generation into a constrained combinatorial optimization problem. Specifically, they first use first-order optimization algorithms to determine the location of perturbations in the program and specific perturbations, including replacing existing tokens or inserting new tokens. Simultaneously, they use the general

formula for adversarial attacks on program obfuscation transformations for any programming language. Then, they propose a random smoothing algorithm to enhance optimization performance. They evaluate the attack on Java and Python function name prediction tasks using Seq2Seq and Code2Seq models. Results indicate that the attack success rate is 1.5 times higher than Henkel et al.'s [58] attack generation algorithm. They also demonstrate that the proposed attack can be used for adversarial training to improve model adversarial robustness.

Zhu et al. [222] propose a white-box adversarial attack method, M-CGA, to assess the robustness of the deep pre-trained model GPT-2 in the context of code generation tasks. They measure the model's robustness by providing adversarial examples and evaluating whether the deep pre-trained model could generate correct or standard-compliant code. M-CGA involves adding a small perturbation to the original input, computing the loss on the model's generated output, and iteratively refining the perturbation. After multiple iterations, M-CGA selects the case with the most significant impact and measures the adversarial attack effectiveness using the success rate. Additionally, post-attack, they utilize Tree Edit Distance (TED) to measure the difference between the original output code and the attacked output code. They employ BLEU scores and the number of changed words as additional metrics to compare the distance between the original and attacked inputs. Experimental results demonstrate that M-CGA can successfully perform adversarial attacks on GPT-2 in code generation tasks with minimal perturbation, achieving a 65% attack success rate.

To investigate the resilience of binary similarity models against adversarial attacks, Capozzi et al. [22] propose a white-box attack method. This approach employs a gradient-guided strategy, repurposing a method typically used in image classifier attacks, to insert instructions into binary code while maintaining semantic integrity. This approach is implemented by iteratively minimizing the model's loss function while ensuring the perturbations remain small and semantically valid. The attack is tested against three state-of-the-art binary similarity solutions: Gemini [186], GMN [93], and SAFE [110]. The results reveal that these models are susceptible to both targeted and untargeted attacks from both black-box and white-box adversaries.

4.3.3 Black-box Attacks on CodeLMs.

Quiring et al. [134] propose a black-box adversarial attack method targeting source code authorship attribution tasks. This method combines Monte Carlo tree search [134] with semantic-preserving source code transformations to search for adversarial examples in the source code domain. The attack iteratively transforms the source code of a program, preserving semantics while changing the code style to mislead CodeLMs for authorship attribution. The authors consider both non-targeted and targeted attacks, requiring the attacks to preserve source code semantics, not change the source code layout, and maintain the syntactic correctness and readability of the transformed code. Their experimental results show that both non-targeted and targeted attacks can successfully compromise state-of-the-art source code authorship attribution methods proposed by Caliskan et al. [72] and Abuhamad et al. [1]. The non-targeted attack significantly impacts the performance of source code authorship attribution models, achieving a 99% success rate in misleading source code authorship with minimal changes to the source code. The targeted attack can accurately mimic the coding style of developers, with success rates reaching 77.3% and 81.3% for the attacks on the methods proposed by Caliskan et al. [72] and Abuhamad et al., [1] respectively.

Matyukhina et al. [112] also introduce an author imitation attack to explore the potential of existing authorship attribution techniques to be deceived. The flow of the attack consists of three steps. Firstly, they identify a victim and retrieve samples of their source code. Once the samples are collected, the second step involves analyzing them and applying transformations to identify the victim's coding style. Finally, they use these transformations to modify the original code, producing a semantically equivalent version that mimics the victim's coding style. The experiments demonstrate that they can successfully launch an imitation attack on all users in Google Code Jam and the GitHub dataset.

To reduce the reliance on the output probabilities or internal details of the target CodeLMs, Liu et al. [102] propose a source code authorship disguise method, called SCAD. Specifically, SCAD first trains a substitute model and develops a set of semantically equivalent transformations. Then, based on these transformations, SCAD makes slight lexical and syntactic feature changes to the original code to achieve disguise. Their experimental results show that on a real dataset containing 1,600 programmers, SCAD induces error rates of over 30% in CodeLMs for author identification based on Random Forest Classifiers and RNN classifiers. They point out the necessity of developing more robust stylistic features and identification CodeLMs to mitigate authorship disguise attacks.

Furthermore, Gao et al. [44] propose an adversarial attack named DaK, specifically designed for CodeLMs. DaK creates discrete adversarial examples by preserving semantic code input through program transformations. DaK comprises three key components: Destroyer, Finder, and Merger. Destroyer alters the functionality of input code to weaken the features on which the model relies for predictions, using only semantically preserving transformations. Finder identifies a set of code predicted as the target label, computes critical features within this set, and selects features deemed the strongest and most concentrated in influencing the model's prediction. Finally, Merger injects the selected critical features into the target program, generating discrete adversarial examples. Experiments on Code2Vec, GGNN, and CodeBERT demonstrate that DaK outperforms Imitator [134] and DAMP [197] on attacking undefended or unreinforced CodeLMs.

Springer et al. [149] discover that the norm of the highest-frequency token embeddings learned by a model increases with the frequency of the tokens. They utilize this relationship to develop a gradient-free variable substitution adversarial attack method called STRATA for generating adversarial examples on CodeLMs. STRATA utilizes three strategies for selecting high-impact tokens, including aggregating all tokens, selecting tokens from the top N highest norm embedding vectors, and selecting tokens with the highest frequency. Additionally, STRATA presents three token substitution strategies, including single substitution, five different substitutions, and five identical substitutions. Experiments demonstrate that STRATA can effectively launch adversarial attacks under these strategies. Moreover, STRATA outperforms the gradient-based adversarial attack method proposed by Henkel et al. [58].

Zhang et al. [206] introduce a black-box attack method based on Metropolis-Hastings [116], named MHM. MHM generates adversarial examples by iteratively renaming identifiers in the source code. Similar to Greedy-Attack (GA) [12], MHM generates adversarial examples by replacing words or characters. In comparison to GA, MHM pays more attention to adversarial examples as code constraints. MHM extracts source identifiers and target identifiers from code snippets and the vocabulary set, deciding whether to rename them based on the calculated acceptance rate. Therefore, adversarial examples generated by MHM do not result in compilation errors. Experiments show that MHM can attack code classification task models based on LSTM [61] and ASTNN [209], achieving attack success rates of 46.4% and 54.7%, respectively.

To further enhance the naturalness of adversarial examples, Yang et al. [191] introduce ALERT, a black-box adversarial attack. ALERT generates adversarial examples by modifying code variables, inducing the victim model to produce incorrect outputs by altering the input. Specifically, ALERT first uses CodeBERT and GraphCodeBERT pre-trained models to generate candidate alternative words for code variables. It then ranks these candidate alternative words based on cosine similarity and uses a greedy algorithm or a genetic algorithm to select adversarial examples from the candidate alternative words. Therefore, ALERT generates adversarial examples that retain the natural semantics and operational semantics of the code. Experiments demonstrate that ALERT outperforms MHM in attacking CodeBERT and GraphCodeBERT in vulnerability prediction, clone detection, and source code authorship attribution tasks.

Furthermore, Yu et al. [200] introduce a black-box adversarial attack framework, AdVulCode, specifically designed for generating code adversarial examples in the context of vulnerability detection tasks based on CodeLMs. Unlike ALERT [191] and MHM [206], which generate adversarial examples by perturbing identifiers,

AdVulCode utilizes equivalent transformation rules that do not alter the semantic meaning of the source code. It targets susceptible code segments for transformation, thus reducing the overall perturbation level of the source code. Specifically, AdVulCode employs equivalent transformations to generate candidate code statements and introduces an improved Monte Carlo search tree to guide the selection of candidate statements and generate adversarial examples. In comparison to ALERT, AdVulCode can handle models with identifier name mappings. Experimental results demonstrate that AdVulCode outperforms ALERT in adversarial attacks for vulnerability detection tasks.

Nguyen et al. [121] propose a black-box adversarial attack framework called GraphCodeAttack. To assess the robustness of CodeLMs, GraphCodeAttack automatically discovers important code patterns influencing model decisions to perturb the code structure of input models. Specifically, GraphCodeAttack identifies discriminative abstract syntax tree (AST) patterns that influence model decisions from source code and target model outputs. It then selects suitable patterns to insert dead code into the source code, creating adversarial examples. Experimental results show that GraphCodeAttack achieves an average attack success rate of 0.841 on author attribution, code vulnerability detection, and code clone detection tasks for CodeBERT and GraphCodeBERT CodeLMs, surpassing ALERT and CARROTA adversarial attack methods.

Zhang et al. [208] propose RNNS, a black-box adversarial attack method designed to assess the robustness of pre-trained CodeLMs by generating adversarial test data based on model uncertainty and output variation. Similar to MHM [206] and ALERT [191], RNNS generates adversarial examples by renaming local variables in the source code. In contrast to MHM and ALERT, RNNS collects variable names from real code to generate statistically characteristic adversarial examples while pursuing a high attack success rate. Specifically, RNNS first constructs a search space for variable names based on semantic similarity using real code data. Then, it utilizes a Representation Nearest Neighbor Search method to find adversarial examples from the namespace guided by model output changes. Failed attack samples guide the next round of attacks in RNNS. Experimental results indicate that, compared to MHM and ALERT, RNNS generates adversarial instances with smaller perturbations and achieves superior attack effectiveness on CodeBERT, GraphCodeBERT, and CodeT5 across six code tasks in three programming languages.

Pour et al. [131] propose a search-based testing framework for generating adversarial examples to test the robustness of downstream task models based on CodeLMs. They utilize ten widely used refactoring operators, such as local variable renaming, parameter renaming, and method name renaming, to generate new adversarial samples for testing. They conduct evaluations on four tasks: method name prediction, code description, code retrieval, and code summarization, based on Code2Vec, Code2Seq, and CodeBERT. Experimental results indicate that the generated adversarial samples average a performance reduction of over 5.41% for CodeLMs. By using the generated adversarial samples for retraining, the robustness of CodeLMs increases by an average of 23.05%, with minimal impact on the model's performance, only 3.56%.

Zhou et al. [220] propose an identifier substitution adversarial attack method named ACCENT to enhance the robustness of CodeLMs in code comment generation tasks. ACCENT generates adversarial examples by modifying program identifiers. For single-letter identifiers, ACCENT randomly changes the original identifier to a different letter. For non-single-letter identifiers, ACCENT uses a black-box, non-targeted search method to generate adversarial examples. Specifically, ACCENT first extracts identifiers from all code in the dataset to establish a candidate identifier set. It then selects the nearest identifiers based on cosine similarity for each identifier in the program, forming a sub-candidate set. Finally, ACCENT ranks these candidate identifiers based on their relationships with the program's context. The final adversarial examples are generated by replacing the original identifier with the best candidate identifier according to the ranking. Experimental results show that ACCENT can efficiently produce adversarial examples that maintain the functionality of the source code. Compared to random substitution algorithms and Metropolis-Hastings-based algorithms [206], ACCENT's generated adversarial examples exhibit better transferability.

Zhang et al. [205] proposed an optimization-based adversarial attack technique named CARROTA. CARROTA supports code transformations at both the token and statement levels and employs an iterative strategy to continuously search for adversarial examples that can mislead models into incorrect classifications. Unlike the MHM method, which only perturbs at the identifier level, CARROTA enables any grammar-compliant synonym transformations and integrates gradient information to guide the adversarial sample search process, thereby reducing computational overhead and improving attack efficiency. Experimental results demonstrate that CARROTA can efficiently generate adversarial examples, leading to an average performance drop of 87.2% at the token level, significantly outperforming MHM's 75.5%. In addition, they also introduce a robustness evaluation metric, CARROTM, which approximates the robustness boundaries under stricter perturbation constraints, achieving significantly tighter robustness estimates compared to random baseline methods and MHM.

Zhang et al. [215] propose a attack method, STRUCK, targeting CodeLMs for code representation. STRUCK fully exploits the structural features of code to generate adversarial examples that mislead CodeLMs while maintaining the original semantics of the code. STRUCK integrates both global and local perturbation methods, using hierarchical structural features to select appropriate perturbations during adversarial example generation. Through comprehensive experiments on several code classification tasks, STRUCK demonstrates its superior effectiveness, efficiency, and imperceptibility compared to existing attacks like MHM and S-CARROTA [205]. The experimental results show that STRUCK achieves a performance degradation of over 80% on various CodeLMs and significantly improves the robustness of models when used for adversarial training.

Tian et al. [166] propose an adversarial attack method targeting the structural features of source code, called QMDP. QMDP first defines a set of transformation operations, iteratively modifying the structural features of the source code to generate adversarial samples that have the same semantics and functionality but differ in syntax. Then, it employs a strategy sampling method, treating the generation of adversarial samples as a Markov decision process, and introduces q-learning to address the control problem throughout the entire attack process, effectively solving the problem of code space explosion caused by applying multiple transformations at multiple locations.

To assess the robustness of software vulnerability detection models, Ferretti et al. [42] propose an evolutionary approach. The fundamental idea of the method is to explore the potential feature representation space using the Grammatical Evolution (GE) algorithm [125]. Initially, the method evolves code snippets by using the classifier's output as the fitness function. Then, it examines how well the selected evolved individuals can effectively alter the classification of any instance in the labeled dataset. Finally, the method generates adversarial samples by injecting semantics while preserving the syntax. This method leverages evolutionary pressure rather than random sampling, thus effectively exploring the behavior of classifiers and identifying features that could potentially mislead the model. Their experiments indicate that code snippets generated by the GE algorithm can significantly guide the network to classify modified instances in a manner opposite to the true labels with high probability, reducing the precision/recall area under curve (P/R-AUC index) of the neural classifier from 0.42 to 0.06.

To reveal the limitations of the robustness of code generation models, Anand et al. [14] define five classes of adversarial examples, including variable change, redundancy removal, synonym replacement, voice conversion, and variable interchange, and conduct adversarial attacks on ALGOLISP generative models and Transformer. They utilize Levenshtein distance, the ratio of Levenshtein distance to sentence length, and BERT-based distance metrics to measure the degree of perturbation in adversarial attacks. Experimental results demonstrate that, compared to traditional neural code generation models, the Transformer architecture exhibits stronger robustness. They also find that the original ALGOLISP dataset may introduce biases during the generation process.

To further investigate the security of LLM-driven code generation models, Wu et al. [184] propose a attack method, DeceptPrompt. DeceptPrompt generates adversarial natural language instructions that drive these models to produce functionally correct but vulnerable code. Specifically, given a benign prompt and a target vulnerability, DeceptPrompt introduces an evolution-based optimization framework to modify the prompt with a

semantic-preserving prefix or suffix. The framework systematically manipulates the input to induce the model into generating vulnerable code, while maintaining the correct functionality. Extensive experiments demonstrate that DeceptPrompt can attack multiple popular LLMs, such as CodeLlama, StarCoder, and WizardCoder, achieving a success rate of up to 62.5% in targeted vulnerability injection across C and Python programming languages. These results reveal significant security weaknesses in current LLM-driven code generation models, with DeceptPrompt successfully bypassing security checks and generating vulnerable code in realistic deployment scenarios.

Li et al. [96] introduce two black-box adversarial attack methods specifically targeting the attribution of source code authors. These methods are Targeted Autoencoding Style Mimicry Attack and Untargeted Autoencoding Style Concealment Attack. The proposed adversarial attack methods leverage systematic preservation of semantic code style attributes and transformations in both attacks. *Targeted Autoencoding Style Mimicry Attack*: This attack extracts code style attributes from all code of the target author, synthesizes a comprehensive code style attribute, and identifies the code style attributes of the attacker's code. Finally, it transforms the attacker's code to mimic the style of the target code. *Untargeted Autoencoding Style Concealment Attack*: This attack extracts code style attributes from the attacker's code, obtains code style attributes from other authors, identifies the differential set between the attacker's code style attributes and those of other authors, selects the author with the highest probability of misattribution, and transforms the attacker's code style to that of the selected target author. Experiments reveal that existing attribution models based on CodeLMs are highly vulnerable to known and unknown adversarial attacks, with untargeted attacks exhibiting significantly higher success rates than targeted attacks.

To reduce the search space for generating adversarial examples, Choi et al. [35] propose TABS. Specifically, TABS adopts beam search to find better adversarial examples and contextual semantic filtering to reduce the search space. Experiments demonstrate that TABS can successfully launch adversarial attacks on the code search task of CodeBERT. Compared to TextFooler [78], BERT-Attack [90], and CLARE [88], TABS shows good performance in terms of attack success rate, number of queries, and other aspects.

Jia et al. [76] propose a black-box adversarial attack method, FuncFooler, targeting learning-based binary code similarity detection models. FuncFooler operates under the constraints of maintaining both the program's CFG and semantic equivalence with the original binary code. The method involves three key steps: identifying vulnerable candidates within the binary code, selecting and inserting adversarial instructions from benign code, and correcting any semantic side effects caused by the adversarial changes. Experimental results show that the method reduces the accuracy of these models to below 5% with minimal instruction perturbations and less than 1% performance overhead. This demonstrates the effectiveness of FuncFooler in generating adversarial examples while maintaining the functional correctness of the target program.

Mercuri et al. [114] introduce an evolutionary technique for generating adversarial examples to enhance the robustness of CodeLMs used for vulnerability detection. The proposed method utilizes the evolution strategies algorithm, with the neural network's output serving as the fitness function for deception. The algorithm starts from existing examples, evolving code snippets while preserving semantic transformations and reversing the original classification through fitness adjustment. This iterative process helps generate adversarial examples that can mislead vulnerability detection models while maintaining the original behavior of the code. The method maps the input code to a genomic representation, which evolves based on a persistent mutation identifier and the process of injecting code, without affecting the original behavior of the code.

Jha et al. [75] introduce a black-box adversarial attack method named CodeAttack, aiming to generate inconspicuous adversarial code examples using code structure. CodeAttack identifies the most vulnerable markers in the code and employs greedy search to generate alternative markers, replacing vulnerable markers with alternatives that minimally perturb the sample, maintain code consistency and fluency, and adhere to code constraints. Experimental results demonstrate that compared to TextFooler [78] and BERT-Attack [90], CodeAttack [75]

efficiently launches adversarial attacks on CodeT5, CodeBRER, and GraphCodeBERT in code transformation, code repair, and code summarization tasks, producing adversarial examples with higher syntactic correctness.

Na et al. [119] propose a black-box adversarial attack method named DIP, which does not require any information about the pre-trained deep models and avoids additional training computations. DIP creates adversarial examples by inserting dead code into the source code. Initially, DIP leverages the pre-trained CodeBERT model to identify vulnerable positions in the source code. It then utilizes cosine similarity to find candidate code snippets in the code repository that are least semantically similar to the source code. Finally, DIP selects the code statement with minimal disturbance from these candidate code snippets, wraps it with an unused variable, and inserts it as dead code into the vulnerable position of the source code. Since DIP launches adversarial attacks by inserting dead code, it preserves the functionality and compilability of the source code. Experiments show that DIP achieves the best CodeBLEU [140] on nine target models and the best disturbance ratio on most target models.

To reveal the insufficient robustness of current code authorship attribution systems, Abuhamad et al. [2] introduce SHIELD. SHIELD injects carefully selected code samples into the target code and then applies Stunrix [152] for code-to-code obfuscation. SHIELD employs obfuscation to make it challenging for models to statically identify or remove the injected code portions, rather than solely conceal the identity of the code author. Adversarial samples are inputted into a black-box code authorship attribution system, and then the probability distribution of the identification model is modified by SHIELD through altering the code expressions and statements of the adversarial samples. SHIELD achieves a success rate of over 95.5% and 66% in non-targeted and targeted attacks, respectively, with a decrease in model identification confidence of over 13%.

Tian et al. [167] propose the CODA for generating semantically invariant adversarial samples by exploiting the code differences between target input and reference input. Specifically, CODA leverages equivalent structural transformations and renaming identifier operations corresponding to the differences to eliminate code differences, thus generating adversarial samples resembling the reference while possessing the same semantics as the target. This effectively reduces the search space for components without the need for model gradient-assisted searching.

To address the inefficiency of adversarial attacks based on simple search strategies, Zhang et al. [207] propose CodeBERT-attack, a black-box adversarial attack technique. CodeBERT-attack iteratively performs identifier replacement attacks. Firstly, CodeBERT-attack locates vulnerable identifiers in the code through masked classification. Then, it utilizes CodeBERT to generate new perturbed identifier names, introducing variations to the identifiers while preserving the overall code structure. Finally, it tests the perturbed samples on the victim model, selecting the samples with the lowest change in prediction accuracy on the ground true labels. Their experiments demonstrate that CodeBERT-attack can reduce the accuracy of LSTM and CodeBERT by an average of 30% in function classification and code clone detection tasks. Moreover, the paper indicates that pre-training can increase the model's adversarial robustness.

Yang et al. [189] propose RADAR-Attack, a gradient-free adversarial attack method designed to expose the vulnerabilities of CodeLMs by manipulating method names. RADAR-Attack targets CodeLMs with adversarial method names that are visually and semantically similar to the original inputs but cause the model to generate incorrect code. RADAR-Attack leverages a two-step approach: first, it generates candidate method names using semantic and visual similarity techniques, and second, it applies a genetic algorithm to optimize these candidates, minimizing the performance of the model as measured by CodeBLEU. This gradient-free approach makes RADAR-Attack highly effective even in black-box scenarios where the internal details of the model are unknown. Experiments demonstrate that RADAR-Attack can significantly reduce performance in models like CodeGPT, PLBART, and CodeT5 in fine-tuning tasks, and in Replit, CodeGen, and CodeT5+ in zero-shot tasks, showcasing its capability to launch effective adversarial attacks against CodeLMs.

Awal et al. [18] propose a adversarial attack method aimed at assessing the robustness of LM in software analytics tasks. The approach minimizes perturbations in feature space, leveraging ML explainability techniques to identify the most influential features impacting model decisions. It emphasizes that adversarial example

generation in software analytics fundamentally differs from methods in computer vision, as any modifications to feature values are easily perceptible. The study employs the reverse elbow method to determine optimal feature combinations. The reverse elbow method evaluates the point at which the improvement in performance metric values ensures that generated adversarial examples induce misclassifications while maintaining feature validity. Experimental results demonstrate that the generated adversarial examples effectively reduce model accuracy, validating the method's potential for enhancing model security.

Yang et al. [190] propose CodeTAE, an adversarial attack method targeting transfer learning in source code models. CodeTAE crafts adversarial examples that exploit the transferability of pre-trained code encoders by introducing semantic-preserving transformations. CodeTAE operates in a cross-domain setting, where the attacker only has access to an open-source pre-trained encoder without any information about the victim model's downstream task or training data. Using a genetic algorithm, CodeTAE generates substitute identifiers and applies obfuscations to induce misclassifications in source code models, achieving attack success rates ranging from 30% to 80%. Experiments on multiple code classification tasks, such as authorship attribution and clone detection, demonstrate the attack's effectiveness, as well as its utility for adversarial fine-tuning to enhance model robustness.

Yao et al. [196] propose a black-box adversarial attack method, CARL, which leverages reinforcement learning to generate adversarial examples for code-based pre-trained language models. CARL consists of a programming language encoder and a perturbation prediction layer, which are initialized with a pre-trained masked CodeBERT model. The method formulates the task of generating adversarial examples as a sequence decision-making problem and optimizes it through policy gradient techniques. CARL uses a combination of reward functions, including attack effectiveness and attack quality, to guide the learning process. Unlike traditional search-based approaches, CARL performs a one-step perturbation prediction, making it significantly faster during inference. Extensive experiments on tasks like code summarization, code translation, and code refinement show that CARL achieves higher success rates while maintaining code quality.

Zhou et al. [218] propose a multi-objective adversarial attack method, MOAA, which uses evolutionary multi-objective optimization to generate adversarial examples for code representation models. Unlike previous methods, MOAA integrates the NSGA-II algorithm with CodeT5 to generate adversarial examples that preserve both syntactic and semantic integrity. MOAA targets DNN models used for tasks such as code classification, clone detection, and code summarization, ensuring high-quality adversarial examples with minimal perturbation. Extensive experiments across five tasks and ten datasets reveal that MOAA produces diverse and transferable adversarial examples, with successful attacks on models like CodeBERT, GraphCodeBERT, and CodeT5. The results highlight MOAA's effectiveness in generating robust adversarial examples while offering insights into the vulnerabilities of pre-trained code models.

He et al. [55] propose MalwareTotal, an adversarial attack method targeting static malware detection models. MalwareTotal generates adversarial malware variants by leveraging sequence-aware bypass tactics that preserve the functionality of the malware while applying multi-faceted manipulations. The method employs proximal policy optimization to identify vulnerable manipulation sequences, effectively bypassing multiple detection mechanisms, including ML-based, signature-based, and hybrid anti-malware software.

4.3.4 Summary of Adversarial Attacks.

Table 5 summarizes the existing white-box attacks on CodeLMs, while Table 6 and Table 7 summarize black-box attacks from 2019-2022 and 2023-2024, respectively. Both types of attacks have demonstrated their effectiveness on various CodeLMs, including non-pre-trained and pre-trained models. Furthermore, these attacks have proven to be effective across multiple code-related tasks, including code understanding and code generation tasks. White-box attack techniques primarily rely on the gradient information of the target model to generate adversarial samples. These adversarial samples are created by introducing perturbations that do not alter the program's semantics, and

Table 5. A summary of existing white-box adversarial attacks on CodeLMs.

Attack Technique	Year	Venue	Attack Type	Target Models	Target Tasks
Meng et al. [113]	2018	ArXiv	White-box attack	Random Forest Support Vector Machine	Authorship attribution
DAMP [197]	2020	OOPSLA	White-box attack	Code2Vec GGNN	Method name prediction Variable name prediction
Bielik et al. [20]	2020	ICML	White-box attack	LSTM GCN DeepTyper	Type prediction
Srikant et al. [150]	2021	ICLR	White-box attack	Seq2Seq	Method name prediction
Chen et al. [28]	2022	ICECCS	White-box attack	Seq2Seq	Code summarization
M-CGA [222]	2023	SANER	White-box attack	GPT-2	Code generation
Capozzi et al. [22]	2023	ArXiv	White-box attack	GNN RNN	Binary similarity detection

they can still successfully compile. Common adversarial perturbations include operations such as substitution, insertion, and deletion, which are applied at the identifier and statement levels. For instance, they may involve renaming variables, function parameters, classes, or structs, replacing Boolean expressions with their equivalents, or inserting and deleting dead code and empty statements. When generating these adversarial samples, it is crucial to ensure their semantic equivalence so that the attack remains imperceptible to humans. However, creating adversarial samples that are both subtle and effective remains a challenging task. Current adversarial attack research is increasingly focusing on black-box attacks. Unlike white-box attacks, black-box attacks do not rely on the gradient information of the model. Instead, they generate adversarial samples by observing the relationship between the model's inputs and outputs. Black-box attack methods often use evolutionary algorithms, genetic algorithms, or other query-based methods to find the optimal perturbations for adversarial samples. These attacks have demonstrated effectiveness on both pre-trained and non-pre-trained models. Compared to white-box attacks, black-box attacks are generally more time-consuming and have lower success rates due to the lack of internal model information. Additionally, black-box attacks have shown strong attacking capabilities across different code-related tasks, such as code understanding and code generation.

Summary ▶ This section summaries backdoor attacks and adversarial attacks on CodeLMs. Backdoors can be inserted through data poisoning or model poisoning, where the design and injection method of the trigger determine the stealthiness and effectiveness of the attack. Such attacks typically remain dormant in controlled environments until the trigger is activated. Adversarial attacks involve adding subtle perturbations to the input that cause the model to make incorrect predictions. These perturbations usually do not alter the semantic functionality of the code but can mislead the model's predictions. Existing research on backdoor and adversarial attacks indicates that CodeLMs are vulnerable to these security threats, which should raise concerns about the security of CodeLMs within the SE community. ◀

Table 6. A summary of existing black-box adversarial attacks on CodeLMs (2019-2022).

Attack Technique	Year	Venue	Attack Type	Target Models	Target Tasks
Quiring et al. [134]	2019	USENIX Security	Black-box attack	Random Forest LSTM	Authorship attribution
Matyukhina et al. [112]	2019	CODASPY	Black-box attack	N-grams	Authorship attribution
SCAD [102]	2021	TIFS	Black-box attack	Random Forest RNN	Authorship attribution
STRATA [149]	2020	ArXiv	Black-box attack	Code2Seq	Method name prediction
MHM [206]	2020	AAAI	Black-box attack	BiLSTM ASTNN	Function classification
Pour et al. [131]	2021	ICST	Black-box attack	Code2Vec Code2Seq CodeBERT	Method name prediction Code description Code search Code summarization
QMDP [166]	2021	QRS	Black-box attack	LSTM ASTNN	Function classification
Ferretti et al. [42]	2021	GECCO	Black-box attack	Random Forest CNN	Vulnerability detection
Anand et al. [14]	2021	ArXiv	Black-box attack	SketchAdapt SAPS, Seq2Tree	Code generation
ACCENT [220]	2022	TOSEM	Black-box attack	LSTM, GNN CSCG, Rencos	Code summarization
ALERT [191]	2022	ICSE	Black-box attack	CodeBERT GraphCodeBERT	Vulnerability detection Clone detection Authorship attribution
CARROTA [205]	2022	TOSEM	White-box attack	GRU, LSTM ASTNN, LSCNN TBCNN, CDLH CodeBERT	Function classification Clone detection Vulnerability detection
Li et al. [96]	2022	ICSE	Black-box attack	DL-CAIS, PbNN	Authorship attribution
TABS [35]	2022	EMNLP	Black-box attack	CodeBERT	Code classification Code search
FuncFooler [76]	2022	ArXiv	Black-box attack	RNN, PV-DM Transformer	Binary similarity detection

5 ANSWER TO RQ2: DEFENSES ON CODELMs

5.1 Research Directions in Defenses against CodeLMs

As shown in the right part of Figure 4, we construct a taxonomy based on the 13 research papers collected on defenses against CodeLMs. Our taxonomy includes two categories of research directions: backdoor defenses and adversarial defenses. There are only 4 papers that focus on studying backdoor defense in CodeLMs and 9 papers that are dedicated to researching adversarial attacks in CodeLMs. Furthermore, based on different defense scenarios, backdoor defenses can be subdivided into pre-training defenses, in-training defenses and post-training defenses, while adversarial defenses can be categorized into adversarial training, model modification and

Table 7. A summary of existing black-box adversarial attacks on abbrs (2023-2024).

Attack Technique	Year	Venue	Attack Type	Target Models	Target Tasks
Mercuri et al. [114]	2023	Algorithms	Black-box attack	CNN	Vulnerability detection
CodeAttack [75]	2023	AAAI	Black-box attack	CodeBERT, CodeT5 GraphCodeBERT	Code transformation Code repair Code summarization
DaK [44]	2023	PLDI	Black-box attack	Code2Vec, GGNN CodeBERT	Variable misuse prediction
GraphCodeAttack [121]	2023	ArXiv	Black-box attack	CodeBERT GraphCodeBERT	Authorship attribution Vulnerability detection Clone detection
AdVulCode [200]	2023	Electronics	Black-box attack	BiGRU, GGNN BiLSTM	Vulnerability detection
DIP [119]	2023	ACL	Black-box attack	CodeBERT, CodeT5 GraphCodeBERT	Clone detection Vulnerability detection Authorship attribution
RNNS [208]	2023	ArXiv	Black-box attack	CodeBERT, CodeT5 GraphCodeBERT	Clone detection Vulnerability detection Authorship attribution Function classification
SHIELD [2]	2023	ArXiv	Black-box attack	CNN, CSFS, DL-CAIS	Authorship attribution
CODA [167]	2023	ASE	Black-box attack	CodeBERT, CodeT5 GraphCodeBERT	Vulnerability detection Authorship attribution Function classification Clone detection, Defect detection
CodeBERT-Attack [207]	2023	JSEP	Black-box attack	LSTM, CodeBERT GraphCodeBERT	Function classification Clone detection
RADAR-Attack [189]	2024	TOSEM	Black-box attack	CodeGPT PLBART, CodeT5	Code generation
Awal et al. [18]	2024	ArXiv	Black-box attack	Logistic Regression Multi-Layered Perceptron Gradient Boosting Classifier Decision Tree, AdaBoost Random Forest, Bagging	Defect detection Clone detection Code classification
DeceptPrompt [184]	2023	ArXiv	Black-box attack	CodeLlama, StarChat WizardCoder	Code generation Vulnerability detection
STRUCK [215]	2024	CollaborateCom	Black-box attack	LSTM, GRU, GCN GGNN, CodeBERT GraphCodeBERT UniXcoder	Function classification Defect detection
CodeTAE [190]	2024	TIFS	Black-box attack	CodeBERT, CodeT5 GraphCodeBERT	Authorship attribution Clone detection Defect detection
MoAA [218]	2024	FSE	Black-box attack	CodeBERT, CodeT5 GraphCodeBERT	Defect detection, Clone detection Authorship attribution Code translation Code summarization
MalwareTotal [55]	2024	ICSE	Black-box attack	EMBER, MalConv AMS A/B, ClamAV	Malware detection
CARL [196]	2024	TOSEM	Black-box attack	CodeBERT, CodeT5 UniXcoder GraphCodeBERT CodeGeeX2	Code summarization Code translation Code refinement

additional models. In backdoor defenses, there are only 2 papers on pre-training defenses, 1 paper on in-training defenses, and 1 paper on post-training defenses. Compared to the research on backdoor attacks against CodeLMs, research on backdoor defenses is still in its early stages. In adversarial defenses, there are only 5 papers on input modification, 1 paper on model modification, and 3 papers on additional models. Similarly, adversarial defenses for CodeLMs are still in the early stages.

5.2 Backdoor Defenses on CodeLMs

5.2.1 Overview of Backdoor Defenses against CodeLMs.

In backdoor attacks on CodeLM, attackers can deploy various methods at different stages of the CodeLM learning process and trigger the implanted backdoors during the model's application phase. As a result, researchers have designed defense methods tailored to the characteristics of backdoor attacks, aiming to be applicable at different stages of the learning process. These methods seek to eliminate or reduce the success rate of backdoor attacks, thereby enhancing the security of CodeLM. Based on different defense objectives, existing backdoor defense methods can be categorized into pre-training defenses, in-training defenses and post-training defenses. Pre-training defenses focus on detecting and removing poisoned samples before training. They typically employ data filtering or anomaly detection techniques to identify potential backdoor samples in the dataset. In-training defenses aim to prevent backdoor insertion during the model training phase. They usually modify the training process to make the model less sensitive to the influence of poisoned samples during training. Post-training defenses are applied after model training is completed. They often utilize techniques such as model unlearning or input filtering to ensure model security.

5.2.2 Pre-training Backdoor Defenses.

Ramakrishnan et al. [138] propose a backdoor detection defense method for CodeLMs based on the spectrum method [170]. They argue that the spectrum method, which uses only the right singular vectors for data sorting, does not always satisfy ϵ -spectrum separability. In other words, the clean samples distribution and the poisoned samples distribution cannot be effectively separated in spectral space under the error tolerance controlled by ϵ . Therefore, they replace it with the top k right singular vectors. Additionally, they use the encoder output vectors or context vectors based on two common CodeLM architectures, Seq2Seq and Code2Seq, to detect poisoned samples. Experiments demonstrate that the proposed backdoor defense method effectively detects low poisoning rates of poisoned samples and can eliminate or reduce the attack success rate.

Li et al. [89] introduce a backdoor detection defense method called CodeDetector, which detects poisoned samples in training data without losing clean samples. CodeDetector focuses on detecting triggers for defense and assumes that attackers may hide triggers within important labels. It utilizes the integrated gradients technique [160] to extract all important labels from the training data. Subsequently, it identifies abnormal labels that significantly impact the model's performance and treats them as potential triggers. All data containing these triggers is then flagged as poisoned samples. Experiments show that compared to baseline methods such as syntax checkers based on program analysis tools, ONION [132], clustering methods, and spectrum methods relying on outlier detection, CodeDetector achieves a 35.95% higher recall and a 16.1% lower false positive rate.

5.2.3 In-training Backdoor Defenses.

Yang et al. [188] propose a backdoor detection defense method for CodeLMs called Deceptive Cross-Entropy (DeCE) to enhance the security of CodeLMs against backdoor attacks. The method leverages the phenomenon of "early learning", where models are initially less sensitive to backdoor triggers but gradually become more susceptible to them, leading to overfitting. DeCE tackles this issue by employing a blending process that combines the model's predicted probability distribution with a deceptive distribution, which is gradually introduced during training. Additionally, label smoothing is applied to the original labels to prevent the model from becoming

overly confident and reduce the risk of overfitting to backdoor triggers. Through extensive experimentation on various models and code synthesis tasks, DeCE demonstrates superior performance in defending against backdoor attacks while maintaining model accuracy on clean datasets.

5.2.4 Post-training Backdoor Defenses.

Hussain et al. [66] propose an occlusion-based human-in-the-loop technique called OSEQL, to distinguish backdoored-triggering inputs of code. OSEQL is based on the observation that the backdoored CodeLM relies heavily on the trigger input and its removal would change the confidence of the CodeLM in prediction substantially. Specifically, OSEQL generates one-line occluded snippets for input code and employs outlier detection to identify outliers in the predictions of suspicious models for these inputs, thereby identifying triggers. Experiments show that in both clone detection and defect detection tasks, OSEQL can achieve trigger identification rates above 90% for poisoned inputs across five popular LMs.

5.2.5 Summary of Backdoor Defenses.

Table 8. A summary of existing backdoor defenses on CodeLMs.

Defense Techniques	Year	Venue	Defense Type	Target Models	Target Tasks
Ramakrishnan et al. [138]	2022	ICPR	Pre-training defense	Code2Seq Seq2Seq	Code summarization Method name prediction
CodeDetector [89]	2024	TOSEM	Pre-training defense	LSTM, TextCNN Transformer CodeBERT	Defect detection Clone detection Code repair
DeCE [188]	2024	ArXiv	In-training defense	CodeBERT, CodeGen CodeT5, CodeT5p GraphCodeBERT	Code generation Code repair
OSEQL [66]	2023	ArXiv	Post-training defense	CodeBERT, CodeT5 BART, PLBART RoBARTa	Defect detection Clone detection

Table 8 summarizes the existing backdoor defense methods for CodeLMs. These methods cover the pre-training, in-training, and post-training phases of CodeLMs, effectively enhancing the security of code models through various techniques. However, backdoor defense is still in its early stages, and there is limited research in this area. As backdoor attack techniques continue to evolve, defense methods still require further optimization to address increasingly complex security threats.

5.3 Adversarial Defenses on CodeLMs

5.3.1 Overview of Adversarial Defenses against CodeLMs.

For the aforementioned adversarial attack methods, researchers have proposed various adversarial defense methods to enhance the model's robustness or eliminate the impact of adversarial samples on the model. There are primarily three methods in adversarial defense: (1) *Input modification*: modifying the model training process, altering input data samples or optimizing the training process. The core idea is to proactively detect and mitigate the impact of adversarial samples or strengthen the model's resistance to abnormal inputs through adversarial training. Typically, adversarial examples are incorporated into the training set, allowing the model to adapt to adversarial perturbations during the learning process and improving its stability against attacks. (2) *Model modification*: adding more sub-networks, modifying activation functions, changing loss functions within the model, or introducing regularization techniques. Robustness-optimized loss functions are commonly employed to

improve classification accuracy in adversarial regions. Additionally, adjusting activation functions can increase the model's sensitivity to adversarial samples, thereby reducing attack success rates. (3) *Additional models*: using external models as additional networks when dealing with unseen data samples for classification. The core idea is to use additional detection mechanisms or adversarially trained networks to identify and process potential adversarial samples. Typically, an adversarial sample detector is placed before the original model to filter input data, ensuring that only normal inputs are processed by the model. The first method focuses on data samples input to the model, while the other two methods are more concerned with the model itself. The latter two methods can be further categorized into model defense and data detection. The goal of model defense is to enable the model to correctly identify adversarial samples, while data detection involves detecting adversarial samples and issuing warnings, leading to the model rejecting further processing of adversarial inputs. Currently, adversarial defense methods for CodeLMs mainly concentrate on the first method. Next, we will provide a detailed overview of relevant research on adversarial defense for CodeLMs.

5.3.2 Input Modification.

Li et al. [96] introduce a framework called RoPGen, which strengthens the robustness of authorship attribution models. RoPGen combines data augmentation and gradient augmentation during adversarial training, making it challenging for attackers to manipulate and mimic the model. RoPGen employs automatic encoding style imitation attacks and automatic encoding style obfuscation attacks for data augmentation, effectively increasing the diversity of training data. RoPGen introduces perturbations to the gradients of deep neural networks for data augmentation, learning a robust CodeLM with diverse representations. RoPGen uses four datasets based on programming languages, such as C, C++, and Java, for evaluation. Experimental results indicate that RoPGen significantly improves the robustness of code authorship attribution models, reducing target attack success rates by an average of 22.8% and non-target attack success rates by 41.0%.

To reduce the complexity of adversarial training, Henkel et al. [58] propose an adversarial training method for CodeLMs based on Madry et al.'s robust optimization objective [108]. They pre-generate program sketches by applying transformations to some programs. During training, only the generated program sketches are considered, and a gradient-based attack is used to search for the optimal parameters for the transformations. Experiments show that adversarial training with just one transformation of CodeLMs efficiently enhances model robustness.

To enhance the robustness of source code processing models, Zhang et al. [205] further proposed the adversarial training method CARROTT. CARROTT utilizes adversarial examples generated by the attack method CARROTA to periodically augment the training set and introduce adversarial perturbations during the training process, thus systematically enhancing the model's robustness to adversarial attacks. Experimental evaluations show that models trained with CARROTT exhibit an average robustness improvement of 5.3 times across multiple tasks (such as code functionality classification, code clone detection, and defect prediction), significantly outperforming MHM, which achieves only a 1.7× improvement.

Cristina et al. [71] propose a method to enhance the robustness of CodeLMs for code generation by adding word-level perturbations in code descriptions, including word replacement and word omission methods. Initially, they demonstrate that perturbations preserve the original semantics through cosine similarity. In experiments, they evaluate the impact of new perturbations on Seq2Seq, CodeBERT, and CodeT5+ models, showing significant performance degradation. Based on this, they apply different proportions of data augmentation to training data and empirically demonstrate that the proposed method improves robustness against perturbed samples and enhances model performance for non-perturbed data, achieving higher semantic correctness.

Furthermore, Gao et al. [44] propose the EverI defense method against discrete adversarial attacks on CodeLMs. EverI trains the model on the strongest adversarial examples, making it theoretically applicable to defend against all adversarial attack methods. For discrete adversarial attacks, EverI first constructs a transitive closure based on semantic-preserving transformations on the original input, identifying adversarial examples in all programs

within the transitive closure that cause the maximum loss for the model. For other classical continuous adversarial attacks like DAMP [197], EverI identifies the strongest adversarial examples by adding adversarial steps and random initialization of adversarial perturbations. For other discrete attacks like Imitator [134], EverI increases the width and depth of the program transformation search tree to find the strongest adversarial examples. Experiments show that, compared to adversarial training [108] and outlier detection defense [197], EverI is more effective in defending against discrete adversarial attacks.

5.3.3 Model Modification.

He et al. [53] propose a learning-based secure control method SVEN for CodeLMs in code generation tasks and test the model's security using crafted adversarial examples. SVEN utilizes continuous vectors with specific attributes to guide CodeLMs toward the direction of the given attributes in program generation without modifying the model's weights. SVEN's training process enforces specialized loss terms on different regions of the code and optimizes these continuous vectors using a carefully crafted high-quality dataset. Experiments show that SVEN can achieve effective secure control, increasing the proportion of secure code generation to 92.3% for CodeGen models without compromising the functional correctness of the generated code.

5.3.4 Additional Models.

Li et al. [95] propose SPACE, a semantic-preserving code embedding approach, to simultaneously enhance the generalization and robustness of CodeLMs. SPACE identifies semantic-preserving attacks in worst-case scenarios, compelling the model to correctly predict labels under these scenarios to improve robustness. Specifically, SPACE uses Byte Pair Encoding Tokenizer (BPE) to convert input code sequences into a series of subword tokens. It then employs a subword embedding layer for high-dimensional word embeddings. Each subword belonging to the same identifier undergoes differentiable perturbations, added to the corresponding word embeddings and position embeddings, forming distributed embeddings. Finally, the pre-trained model's encoder further encodes these embeddings. Differentiable perturbations are updated based on their gradients, and a gradient ascent algorithm computes the final loss to minimize the negative impact of perturbations on the model. SPACE conducts adversarial training in continuous embedding space, efficiently incorporating it into gradient-based training frameworks. SPACE combines adversarial training with data features specific to programming languages, achieving improved performance and robustness for CodeLMs. Experiments demonstrate that training pre-trained CodeLMs CodeBERT and GraphCodeBERT with SPACE leads to better generalization, maintaining robustness against advanced adversarial attack methods such as MHM [206] and ALERT [191].

Zhang et al. [203] first generate adversarial examples using white-box attacks on smaller code models and then explore their effectiveness for transfer adversarial attacks on code LLMs. Meanwhile, they propose leveraging LLMs' in-context learning and the self-defense capability of understanding human instructions, as well as meta-prompting techniques (i.e., using the LLM to generate its own defense prompts) to achieve adversarial defense for large models. Experimental results show that these defense methods effectively reduce the attack success rate on GPT-3.5, GPT-4, Claude-Instant-1, Claude-2, and CodeLlama.

Yang et al. [189] propose RADAR-Defense, a defense method to improve the robustness of CodeLMs by synthesizing high-quality method names from functional descriptions. Specifically, RADAR-Defense avoids altering the original model and instead focuses on improving input quality by generating semantically meaningful method names that are supplied to CodeLMs along with functional descriptions and other signature information. This retrieval-enhanced prompt training method effectively reinstates the performance of CodeLMs under adversarial attacks. Experiments demonstrate that RADAR-Defense is able to restore the CodeBLEU and Pass@1 scores in various CodeLMs, such as CodeGPT, PLBART, CodeT5, Replit, CodeGen, and CodeT5+, thereby significantly improving model robustness without requiring retraining.

5.3.5 Summary of Adversarial Defenses.

Table 9. A summary of existing adversarial defenses on CodeLMs.

Defense Techniques	Year	Venue	Defense Type	Target Models	Target Tasks
RoPGen [96]	2022	ICSE	Input modification	DL-CAIS PbNN	Code authorship attribution
Henkel et al. [58]	2022	SANER	Input modification	Seq2Seq Code2Seq	Code summarization
CARROTT [205]	2022	TOSEM	Input modification	GRU, LSTM ASTNN, LSCNN TBCNN, CodeBERT CDLH	Function classification Clone detection Vulnerability detection
Cristina et al. [71]	2023	ArXiv	Input modification	Seq2Seq CodeBERT CodeT5+	Code generation
EverI [44]	2023	PLDI	Input modification	CodeBERT Code2Vec GGNN	Code summarization Method name prediction Variable misuse prediction
SVEN [53]	2023	CCS	Model modification	CodeGen	Code generation
SPACE [95]	2022	COLING	Additional models	CodeBERT GraphCodeBERT	Defect detection Natural language code search Code question answering
Zhang et al. [203]	2023	ArXiv	Additional models	GPT-3.5, GPT-4 Claude-Instant-1 Claude-2	Code summarization
RADAR-Defense [189]	2024	TOSEM	Additional models	CodeGPT, PLBART CodeT5, CodeT5+ Replit, CodeGen	Code generation

Table 9 summarizes the existing adversarial defense methods for CodeLMs. Adversarial defenses for CodeLMs encompass input modification, model modification, and additional model methods, each with a specific focus on mitigating the effects of adversarial attacks. The input modification approach often involves adversarial training, which has been shown to enhance model robustness. Model modification strategies adapt sub-networks, activation functions, or loss functions to detect and handle adversarial samples effectively. Lastly, the additional model approach leverages external models for classification robustness. Methods like RoPGen, SVEN, and SPACE contribute significantly to adversarial robustness, highlighting the potential of combining adversarial training with program-specific features for enhanced defense against complex adversarial scenarios.

 **Summary** ► The attacks and defenses for CodeLMs represent a continuous battle. Defense methods are the direct approach to enhancing the security of CodeLMs. Although current defense techniques cover different methods and critical stages of CodeLMs, compared to the extensive research on attacks in the security of CodeLMs, research on defenses is still in its early stages. ◀

6 ANSWER TO RQ3: EMPIRICAL STUDIES ON SECURITY OF CODELMs

Various attack and defense techniques targeting CodeLMs have been successively proposed and have achieved promising results in revealing CodeLM security vulnerabilities and enhancing its security. In addition to developing new attack and defense techniques that address technical challenges, the field of CodeLM security research has

Table 10. A summary and comparison of empirical studies on the security of CodeLM.

Year	Study	Type	Main Research Content (Research Questions)
2021	Nguyen et al. [120]	Adversarial Attack	1) How well has the issue of AML in RSSE been addressed by the existing literature? 2) To what extent are state-of-the-art API and code snippet recommender systems susceptible to malicious data?
2023	Du et al. [39]	Adversarial Attack	1) How do existing adversarial attack approaches perform against different PTMCs under various tasks? 2) What is the quality of adversarial examples generated by adversarial attacks? 3) How do the contexts of the perturbed identifiers affect the adversarial attacking performance?
2023	Li et al. [97]	Adversarial Defense	1) How does learning sample obtaining strategy during training impact model performance? 2) How does loss computation impact model performance? 3) How does input data impact model performance? 4) How does input model impact model performance?
2024	Oh et al. [124]	Backdoor Attack	1) How do developers' adoption rates and trust levels differ when using code completion compared to code generation as AI-powered coding assistant tools, and what factors influence these variations? 2) How do poisoning attacks on AI-powered coding assistant tools influence the security of software developers' code in the real world? 3) Which type of AI-powered coding assistant tools, code completion or code generation, are more susceptible to poisoning attacks?
2024	Hussain et al. [69]	Backdoor Attack	1) Is there any trojan signature in the classifier weights of the trojaned code models which were produced by full-finetuning? 2) Is there any trojan signature in the trojaned code models, which were produced where pretrained weights were frozen?
2024	Hussain et al. [70]	Backdoor Attack	1) Distribution of attention weight and bias in clean and poisoned models. 2) Clustering activation values of clean and poisoned samples. 3) Visualizing context embeddings of clean and poisoned models. 4) Comparison of fine-tuned parameters with pre-trained parameters. 5) Resetting fine-tuned weights to pre-trained weights based on a threshold.

also benefited from several empirical studies. These studies systematically explore the capabilities of attack or defense techniques and their impact on model performance, providing valuable insights for future exploration of CodeLM security. We summarize the existing empirical studies in Table 10 and discuss them in detail as follows.

6.1 Practical impact of backdoor attacks on CodeLMs

Oh et al. [124] conduct an empirical study on the practical impact of data poisoning attacks on AI-powered coding assistant tools, exploring how these attacks affect developers' coding behavior when using these tools and the security of the code generated by them. Specifically, they investigate the following research questions: 1) How do developers' adoption rates and trust levels differ when using code completion compared to code generation as AI-powered coding assistant tools, and what factors influence these variations? They first conduct an online survey aimed at understanding the prevalence of AI-powered coding assistant tool usage and the level of trust developers place in the generated code. Their survey of 238 participants reveals that developers generally trust code completion tools more than code generation tools. This trust likely stems from developers' perception that code completion tools are more precise and their suggestions come from verified, reliable official API documentation. At the same time, they indicate that both types of tools are susceptible to data poisoning attacks. These attacks can introduce malicious code into a codebase, posing a security risk. Consequently, they proceed to address the following research questions. 2) How do poisoning attacks on AI-powered coding assistant

tools influence the security of software developers' code in the real world? They use TrojanPuzzle to poison the CodeGen 6.1B model and have participants complete three programming tasks related to common security vulnerabilities in software development: AES encryption, SQL query, and DNS query. Through the manual review of the participants' code, they find that participants using the code generation tool are more likely to incorporate insecure code compared to those using the code completion tool or no tool at all. This highlights the influence of AI-powered coding assistant tools on secure coding practices. 3) Which type of AI-powered coding assistant tools, code completion or code generation, are more susceptible to poisoning attacks? Through an in-depth analysis of participants' programming practices, they find that code completion tools are relatively weak against poisoning attacks. This is because these tools guide developers to obtain initial code from the internet rather than relying on the tools themselves. Moreover, practices like "copy and paste" or "see and type" help bypass the code suggestions of the tools, thereby reducing the likelihood of poisoning attacks. Code generation tools are very sensitive to poisoning attacks since developers often use the suggested code without significant modifications. However, some developers adopt secure coding practices to generate their own secure code or modify the suggested code. Their findings highlight the importance of developers understanding potential security issues and maintaining caution regarding the safety of AI-driven coding assistance tools.

Hussain et al. [69] conduct an empirical study on the distribution of trojan signatures in the classifier layer parameters of large language models for source code. They address two research questions: 1) Is there any trojan signature in the classifier weights of the trojaned code models which were produced by full-finetuning? 2) Is there any trojan signature in the trojaned code models, which were produced where pretrained weights were frozen? They perform detailed experiments on two tasks, clone detection and defect detection, using CodeBERT, PLBART, and CodeT5. Ultimately, their findings suggest that Trojan signatures do not generalize to LLMs of code. They observe that even when fine-tuning with frozen pre-trained weights in a more explicit setting, poisoned code models remain resistant, indicating that detecting trojans solely based on the model's weights is a challenging task.

Furthermore, Hussain et al. [70] conduct an empirical study on the attention weights and biases, activation values, and context embeddings of clean and poisoned CodeBERT models, exploring the differences between the clean and poisoned models. First, they use TrojanedCM [68] to poison CodeBERT for the defect detection task. Then, they extract and compare the parameters of the poisoned CodeBERT model with its corresponding clean model layer by layer, examining various aspects such as attention weights and biases, activation values, and context embeddings. Specifically, they conduct the following five analyses: 1) Distribution of attention weights and biases in clean and poisoned models; 2) Clustering of activation values for clean and poisoned samples; 3) Visualization of context embeddings of clean and poisoned models; 4) Comparison of fine-tuned parameters with pre-trained parameters; 5) Resetting fine-tuned weights to pre-trained weights based on a threshold. Ultimately, they find that there are no significant differences in attention weights and biases between the clean and poisoned CodeBERT models. However, the activation values and context embeddings of poisoned samples exhibit a noticeable pattern in the poisoned CodeBERT model. Their findings contribute to the white-box detection of backdoor signals in CodeLMs.

6.2 Performance of adversarial attacks against CodeLMs across different tasks and models

Nguyen et al. [120] conduct an empirical investigation of adversarial machine learning techniques and their potential impact on API recommender systems. 1) How well has the issue of adversarial machine learning (AML) in recommender systems for software engineering (RSSE) been addressed by the existing literature? They conduct a literature analysis following the guidelines [83, 107] to investigate whether there have been efforts in studying and addressing threats to RSSE originating from malicious data. They collect 7,076 papers by answering the four W-questions [204] ("W" stands for Which?, Where?, What?, and When?). Through a thorough examination of

these papers, they find that all of them focus on malware detection in Android applications. However, they do not discover any work related to potential threats and adversarial attacks concerning API RSSE. Thus, as of 2021, the issue of adversarial attacks targeting API and code snippet recommendations has not been sufficiently studied in the main areas of software engineering. 2) To what extent are state-of-the-art API and code snippet recommender systems susceptible to malicious data? Through the qualitative analysis of the significant RSSE in mining APIs and code snippets for API recommendation systems, they find that if malicious data is used for model training, RSSE may recommend malicious APIs or code snippets. According to the analysis, even if only 10% of the training data is manipulated, UPMiner and PAM recommendation systems still provide a considerable number of projects with false APIs. Although FOCUS is less susceptible to such manipulation than UPMiner, the consequences of its recommendations can be devastating because these tools provide code snippets. Thus, harmful training data poses a significant threat to the robustness of state-of-the-art RSSEs, including UPMiner, PAM, and FOCUS.

Du et al. [39] conduct an empirical study on the effectiveness and efficiency of methods for generating adversarial examples for code pre-trained models and their performance on different code tasks. They conduct a thorough evaluation of MHM [206], ACCENT [221], WIR-Random [202], ALERT [191], and StyleTransfer [58] on the performance of CodeBERT [41], CodeGPT [105], and PLBART [5]. They introduce the following research questions: 1) How do existing adversarial attack approaches perform against different PTMCs under various tasks? In this research problem, they conduct a comprehensive comparison of adversarial attack methods based on effectiveness and efficiency. They found that random strategy adversarial attacks are more effective on pre-trained code models, while CodeGPT is generally more resilient to various attacks. Additionally, models tend to exhibit lower robustness in generation tasks compared to understanding tasks. There is a trade-off between effectiveness and efficiency in adversarial attacks. Attacks with higher success rates typically require more model queries. Furthermore, the efficiency of the attacks is also influenced by the number of identifiers in the target program. 2) What is the quality of adversarial examples generated by adversarial attacks? Here's the translation: For this research question, they use the Identifier Change Rate (ICR) and Token Change Rate (TCR) to evaluate the number of tokens that have been replaced in the adversarial samples, and we use Average Code Similarity (ACS) and Average Edit Distance (AED) to assess the similarity between the adversarial tokens and the original tokens. They find that there is also a trade-off between effectiveness and naturalness in adversarial attacks. Effective attacks generate less natural adversarial samples. Generally, alternative strategies such as context-based identifier prediction and cosine similarity-based substitutions produce higher-quality samples than random replacements. 3) How do the contexts of the perturbed identifiers affect the adversarial attacking performance? In this research question, they consider the statements in which the identifiers are located as context and investigate whether perturbing the identifiers in different contexts affects the effectiveness of the attacks. They choose five commonly used statements in code for investigation: Return, If, Throw, Try, and For statements. In addition to these types of statements, they also examine the impact of modifying only method names and parameters. Ultimately, they find that the context of identifiers can significantly affect the effectiveness of attacks. This indicates that disturbance strategies should consider the context of identifiers to achieve more effective attacks.

6.3 Effectiveness and robustness of adversarial training for CodeLMs

Li et al. [97] conduct an empirical study on the effectiveness and robustness of adversarial training for protecting CodeLMs. First, they collect and categorize five types of adversarial training methods for CodeLMs: mixing directly, composite loss, adversarial fine-tuning, min-max + composite loss, and min-max. Next, they conduct empirical evaluations of these categorized adversarial training methods on the code summarization task using Code2Seq [9] and the code authorship attribution task using Abuhamad [1]. Specifically, they investigate the following research questions: 1) How does learning sample obtaining strategy during training impact model performance? Different adversarial training methods may adopt various strategies to acquire the samples to be

learned in the current training iteration. For example, perturbing the input samples or selecting samples that maximize the loss for learning. They evaluate these two strategies to study their impact on the performance of CodeLMs. They find that the impact of sample obtaining strategies for CodeLM varies depending on the task, and no single strategy is suitable for all tasks. However, the performance of optimization objective based methods using the min-max strategy is related to the number of times perturbed samples are regenerated. Therefore, they suggest that extending training time can be traded off for higher model robustness. 2) How does loss computation impact model performance? Some adversarial training methods use ordinary loss, where all terms are computed equally for all samples. Other methods use composite loss, where some terms are computed for clean samples and others for adversarial or perturbed samples. They find that composite loss is generally more effective than ordinary loss. However, in terms of model robustness, composite loss performs better when using object-oriented methods, while ordinary loss demonstrates stronger robustness when combined with data augmentation-like methods. 3) How does input data impact model performance? Some adversarial training methods require the input to be adversarial samples, while others require the use of both adversarial and perturbed samples. Even within the same category of samples, the composition may vary. They find that the input data used in adversarial training has a significant impact on the model's robustness, with adversarial samples being the key factor, rather than perturbed samples. When generating adversarial samples, the type of transformations is highly correlated with the final model's robustness. They recommend including data with various transformations to cover potential attacks that could reduce robustness. 4) How does input model impact model performance? They evaluate the impact of input models on the final model's robustness from two aspects: the influence of pre-trained models and the number of fine-tuning epochs. They find that pre-trained models outperform non-pre-trained models in terms of training time and can surpass composite loss methods in robustness, achieving comparable robustness to direct mixing methods. Additionally, for adversarial fine-tuning, only a few fine-tuning epochs are required on pre-trained models, as increasing the number of fine-tuning epochs does not lead to better model performance.

 **Summary** ► Existing empirical studies on the security of CodeLMs primarily focus on two aspects: 1) A comprehensive evaluation of the performance of backdoor/adversarial attacks on different code-related tasks and various CodeLMs, including the success rate of attacks and their impact on model performance. 2) The effect of different components of defenses on both the effectiveness of the defense and model performance. For both attack methods and defense methods targeting CodeLMs, empirical studies focus on their effectiveness, their impact on model performance, and the trade-offs between effectiveness and model performance. ◀

7 ANSWER TO RQ4: EXPERIMENTAL SETTING AND EVALUATION

7.1 RQ4.1: Experimental Datasets

In this section, we present the experimental datasets commonly used in CodeLM security research. Table 11 summarizes these datasets along with key information and is intended to support future research by providing convenient access to commonly used benchmark datasets.

① **BigCloneBench.** BigCloneBench is proposed by Svajlenko et al. [161]. It is used for big data clone detection and clone search. It comprises known true and false clone fragments from the IJaDataset 2.0¹, a large data project repository. BigCloneBench covers ten functionalities, including 6 million true clone pairs and 260,000 false clone pairs.

② **Open Judge (OJ) dataset.** The Open Judge (OJ) dataset is introduced by Mou et al. [118]. This dataset serves as a source code classification benchmark. Comprising 52,000 C/C++ code files labeled with problem numbers, it includes 104 classes, each containing 500 code files.

¹<http://secold.org/projects/seclone>

Table 11. A summary of experimental datasets used in CodeLM security research.

Dataset	Year	Programming Language	Data Source	Download Link	Study
BigCloneBench [161]	2014	Java	GitHub	https://github.com/clonebench/BigCloneBench	[18, 66, 89, 119, 121, 167, 190, 191, 208]
OJ dataset [82]	2016	C++	OJ Platform	http://programming.grids.cn	[166, 167, 205, 206, 215]
CodeSearchNet [65]	2019	Go	GitHub	https://github.com/github/CodeSearchNet	[94, 95]
		Java			[28, 75, 94, 95, 133, 154, 188]
		JavaScript			[94, 95]
		PHP			[75, 94, 95]
		Python			[35, 58, 75, 94, 95, 133, 138, 154, 172, 188, 193]
		Ruby			[94, 95]
Code2Seq [9]	2019	Java	GitHub	https://github.com/tech-srl/code2seq#datasets	[28, 58, 112, 131, 149]
Devign [219]	2019	Java	GitHub	https://github.com/rjust/defects4j	[66, 89, 208]
Google Code Jam (GCJ)	2020	C++	OJ Platform	https://codingcompetitions.withgoogle.com/codejam	[2, 96, 134]
		Java			[96, 112]
		Python			[119, 121, 167, 191]
CodeXGLUE [105]	2021	Java	GitHub	https://github.com/microsoft/CodeXGLUE	[94, 188]
		C/C++			[94, 121]
		Python			[94, 94]
		PHP			[94]
		JavaScript			[94]
		Ruby			[94]
		Go			[94]
CodeQA [100]	2021	Java	GitHub	https://github.com/jadexliu/codeqa	[95]
		Python			[95]
APPS [57]	2021	Python	OJ Platform	https://people.eecs.berkeley.edu/hendrycks/APPS.tar.gz	[222]
Shellcode_IA32 [98]	2021	ASM instruction	GitHub	https://github.com/dessertlab/Shellcode_IA32	[71]
SecurityEval [146]	2022	Python	GitHub	https://github.com/s2e-lab/SecurityEval	[36, 184]
LLMSecEval [168]	2023	Python	GitHub	https://github.com/tuhh-softsec/LLMSecEval	[36]
PoisonPy [9]	2023	Python	GitHub	not yet published	[36]

③ **CodeSearchNet.** CodeSearchNet is proposed by Hamel et al. [65]. This dataset consists of 99 natural language queries with about 4k expert relevance annotations of likely results from CodeSearchNet Corpus which contains about 6 million functions from open-source code spanning six programming languages (Go, Java, JavaScript, PHP, Python, and Ruby). The CodeSearchNet Corpus also contains automatically generated query-like natural language for 2 million functions, obtained from mechanically scraping and preprocessing associated function documentation.

④ **Devign.** Devign is proposed by Zhou et al. [219]. This dataset is used for defect detection tasks. The authors initially collected functions from four large open-source projects written in the C language: Linux Kernel, QEMU, Wireshark, and FFmpeg. They performed Commit Filtering to exclude commits unrelated to security. Subsequently, a team of four professional security researchers conducted two rounds of data annotation and cross-validation. To ensure the quality of data labeling, the team gathered security-related commits and marked them as Vulnerability-Fix Commits (VFCs) or Non-Vulnerability-Fix Commits (non-VFCs). They then extracted functions vulnerable or non-vulnerable to attacks directly from the labeled commits. Vulnerability-Fix Commits (VFCs) refer to commits fixing potential vulnerabilities, and vulnerable functions can be extracted from the source code of versions before the revisions made in the commits. Non-vulnerability-fix commits (non-VFCs) are commits that do not fix any vulnerabilities, and non-vulnerable functions can be extracted from the source code before the modifications. The final dataset comprises 21,000 training samples, 2,700 validation samples, and 2,700 test samples.

⑤ **Google Code Jam (GCJ).** Google Code Jam (GCJ) is an annual multi-round international programming competition that requires participants to solve programming challenges in each round. The GCJ dataset is sourced from the Google Code Jam programming competition website and is divided into two categories: GCJ-C++ and GCJ-Java. The GCJ-C++ dataset consists of 1,632 C++ program files written by 204 participants, with each participant having 8 program files corresponding to 8 programming challenges. On average, each program file contains 74 lines of code. The GCJ-Java dataset includes 2,396 Java files from 74 participants, with an average of 139 lines of code per file.

⑥ **CodeXGLUE.** CodeXGLUE is proposed by Lu et al. [105]. It is a benchmark dataset for program understanding. CodeXGLUE is collected from 14 common datasets, including BigCloneBench, POJ-104, and Devign, and is divided into four categories: Code-Code, Text-Code, Code-Text, and Text-Text. These categories cover 10 downstream tasks, including code clone detection, code defect detection, cloze test, code completion, code repair, code translation, NL code search, text-to-code generation, code summarization, and documentation translation.

⑦ **CodeQA.** CodeQA is introduced by Liu et al. [100]. It is used for code question-answering (QA) tasks. Code question-answering involves generating textual answers given a code snippet and a question. In the dataset construction process, the authors select Java and Python datasets from GitHub, ensuring that QA pairs are natural and clean. They then chose appropriate comments from these datasets to generate QA pairs. Using syntactic rules and semantic analysis, the authors transform comments into QA pairs. The Java dataset in the CodeQA dataset contains 119,778 QA pairs, while the Python dataset contains 70,085 QA pairs.

⑧ **APPS.** APPS is introduced by Hendrycks et al. [57]. It serves as an evaluation benchmark for code generation tasks. It comprises problems collected from various OJ websites. This dataset assesses the model's ability to write syntactically correct algorithm programs. It covers simple introductory problems, interview-level questions, and coding competition challenges. The dataset includes a total of 10,000 problems with an average length of 293.2 words, 131,777 test cases for checking solutions, and 232,421 human-written real solutions. The data is evenly split into training and testing sets, each containing 5,000 problems. In the testing set, each problem has multiple test cases, with an average of 21.2 test cases per problem.

⑨ **Shellcode_IA32.** Shellcode_IA32 is proposed by Liguori et al. [98] in 2021. This dataset consists of IA-32 (x86 Intel Architecture 32-bit version) assembly language instructions with detailed English annotations. The dataset comprises a total of 3,200 pairs of assembly code snippets and comments. Considering the variability in natural language descriptions, the dataset describes different samples in English and uses comments written by the developers of the collected programs as natural language descriptions. This dataset represents the largest security-oriented code collection available for code generation to date.

⑩ **SecurityEval.** SecurityEval is introduced by Siddiq et al. [146]. This dataset is designed to evaluate the security of code generation models. The dataset includes 130 Python code snippets covering 75 vulnerability types (CWE). Released in JavaScript Object Notation Line (JSONL) format, each line contains a JSON object with three key-value pairs: ID, Prompt, and Insecure Code. ID uniquely identifies the sample, Prompt provides a partial source code template usable as inputs for code generation models, and Insecure Code represents potentially vulnerable code examples generated by the model. This dataset is suitable for inputting source code prompts to LLM and evaluating the security of the generated code.

⑪ **LLMSecEval.** In 2023, Tony et al. [168] propose the LLMSecEval dataset to assess the security of code generation models. The dataset includes 150 natural language(NL) prompts covering the top 25 common defect types (CWE) according to MITRE. Derived from Chen et al.'s HumanEval data, the authors preprocess the dataset, filtering out invalid descriptions, removing blank content, eliminating entries with extensive code snippets, and excluding content that does not explain the input code's functionality. 150 NL prompts are compiled into CSV and JSON files, each row consisting of CWE name, NL Prompt, Source Code Filepath, Vulnerable, Language, Quality Metrics, and Secure Code Samples. CWE name indicates the defect type. NL Prompt is used to prompt

model-generated code. Source Code Filepath points to the path of the source code file generated from the prompt. Vulnerable marks the field indicating defective code. Language indicates the source language of the prompt. Quality Metrics provide scores for four evaluation metrics. Secure Code Samples present secure code examples corresponding to each NL prompt.

⑫ **PoisonPy**. In 2023, Cotroneo et al. [9] introduce the PoisonPy dataset for evaluating the security of CodeLMs for code generation. PoisonPy comprises 823 samples, including 568 secure code snippets and 255 defective code snippets. The 255 defective samples encompass 109 Taint Propagation Issues (TPI), 73 Insecure Configuration Issues (ICI), and 73 Data Protection Issues (DPI). To construct the dataset, the authors combine available benchmark datasets SecurityEval and LLMSecEval to evaluate the security of AI-generated code. The original datasets are combinations of NL-Prompt, docstrings, and code used to evaluate AI code generators, making them unsuitable for fine-tuning models. The authors segment the collected code examples into multiple fragments, separate vulnerable code lines from secure lines, and enrich code descriptions. Additionally, to adjust the dataset's poisoning rate, for each vulnerable fragment, the authors provide an equivalent secure version by implementing potential mitigations for each CWE, without altering code descriptions.

Currently, commonly used datasets for backdoor attack and defense research on CodeLMs include CodeSearchNet, CodeXGLUE, and Code2Seq. For adversarial attack and defense research, commonly used datasets include CodeSearchNet, CodeXGLUE, BigCloneBench, Devign, APPS, CodeQA, and GCJ.

7.2 RQ4.2: Experimental CodeLMs

To better understand the behavior and vulnerabilities of CodeLMs, numerous CodeLMs have been developed and applied in the field of program analysis and code-related tasks. In this section, we introduce and summarize the experimental CodeLMs used in adversarial and backdoor research on CodeLM security. Table 12 provides a detailed list of experimental models used in the field of CodeLMs security.

① **CNN, RNN, LSTM, GRU, BiLSTM, BiRNN**. CNN (Convolutional Neural Network) is a class of neural networks that excels at processing structured data, typically used in image recognition tasks, but CNN can also capture local features in code analysis tasks. By applying convolution operations to the abstract syntax tree (AST) of code, CNN can extract useful features from the code structure, making it applicable to tasks such as code classification and vulnerability detection. RNN (Recurrent Neural Network) and its variants, LSTM (Long Short-Term Memory), GRU (Gated Recurrent Unit), and BiLSTM (Bidirectional LSTM), are particularly suited for handling sequential data, especially for code generation and code understanding tasks. They pass information along the time dimension, capturing long-term dependencies in the code. BiRNN and BiLSTM further enhance context understanding by processing code sequences in both forward and backward directions, making them widely used in tasks such as code summarization and function classification.

② **GNN, GCN, GGNN**. Graph Neural Networks (GNN) are designed to process graph-structured data, making them ideal for tasks like code analysis, such as Abstract Syntax Trees (ASTs) and Control Flow Graphs (CFGs). GNNs capture relationships between nodes, enabling a deeper understanding of code structure. Building on GNNs, Graph Convolutional Networks (GCN) use convolution operations to focus on local structural features, making them effective for code classification and similarity detection. Gated Graph Neural Networks (GGNN) enhance GNNs by incorporating LSTM-like gating mechanisms to better manage long-range dependencies, improving performance on complex code structures in tasks like vulnerability detection and pattern recognition.

③ **Random Forest**. Random Forest is an ensemble learning model based on decision trees, improving accuracy and robustness by building multiple decision trees and combining their predictions. In code analysis, Random Forest is commonly used for static code analysis tasks such as code defect detection and classification. By randomly sampling code features, it reduces the risk of overfitting a single decision tree, making it particularly suited for handling high-dimensional and structurally complex code data.

Table 12. A summary of experimental models used in CodeLM security research.

LM	Year	URL	Study
RNN	1990	-	[22, 102]
BiLSTM	1997	-	[200, 206]
BiRNN	1997	-	[133, 172]
LSTM	1997	-	[20, 89, 134, 166, 198, 205, 207, 215, 220]
CNN	1998	-	[2, 42, 114]
Random Forest	2001	-	[18, 42, 102, 113, 134, 144]
GNN	2005	-	[22, 220]
GCN	2013	-	[20, 215]
GRU	2014	-	[205, 215]
GGNN	2015	-	[44, 197, 200, 215]
Transformer	2017	-	[89, 133, 172, 216]
Seq2Seq	2017	https://github.com/google/seq2seq	[28, 36, 58, 71, 138, 150]
DL-CAIS	2018	-	[2, 96]
ASTNN	2019	https://github.com/zhangj111/astnn	[166, 205, 206]
Code2Seq	2019	https://github.com/tech-srl/code2seq	[58, 131, 138, 149]
Code2Vec	2019	https://github.com/tech-srl/code2vec	[44, 131, 197]
GPT-2	2019	https://huggingface.co/openai-community/gpt2	[142, 222]
CodeBERT	2020	https://huggingface.co/microsoft/codebert-base	[35, 44, 75, 95, 119, 121, 131, 191, 205] [36, 71, 89, 154, 167, 172, 207, 208] [66, 133, 188, 190, 193, 215, 218]
GraphCodeBERT	2020	https://huggingface.co/microsoft/graphcodebert-base	[75, 95, 119, 121, 133, 167, 190, 191, 207, 208, 215, 218]
CodeT5	2021	https://huggingface.co/Salesforce/codet5-base	[66, 75, 94, 119, 154, 167, 188–190, 193, 208, 218]
CodeT5+	2021	https://huggingface.co/Salesforce/codet5-base	[36, 71, 189]
PLBART	2021	https://github.com/wasiahmad/PLBART	[66, 94, 189, 193]
CodeGen	2022	https://huggingface.co/Salesforce/codegen-350M-multi	[3, 53, 188, 189]
GPT-3.5	2022	https://openai.com/	[203]
GPT-4	2023	https://openai.com/	[203]

④ **Transformer.** The Transformer is a deep learning model based on the attention mechanism, widely applied in natural language processing and code generation tasks. Unlike traditional RNNs, the Transformer can process entire input sequences in parallel using a global attention mechanism, making it more efficient and performant when handling long sequences. The self-attention mechanism of the Transformer effectively captures long-range dependencies and complex syntax structures in code, making it excel in tasks such as code generation, code translation, and code repair. Variants of the Transformer, such as CodeBERT, GraphCodeBERT, and CodeT5, have demonstrated strong capabilities in code understanding and generation.

⑤ **DL-CAIS.** DL-CAIS (Deep Learning Context-Aware Intelligent System) is an intelligent system that integrates deep learning and context-aware technology, used for program analysis and code detection tasks. DL-CAIS combines static and dynamic code analysis, leveraging deep learning models (such as CNNs and RNNs) to extract high-level features from code. It is applied in scenarios such as malware detection, code vulnerability identification, and automatic code generation. By deeply understanding the code context, the system significantly improves the precision and robustness of code analysis tasks.

⑥ **ASTNN.** ASTNN (Abstract Syntax Tree Neural Networks) is a model designed specifically for code-related tasks, leveraging the abstract syntax tree (AST) to represent code structures. It is often applied in tasks such as code classification and function classification.

⑦ **Code2Vec.** Code2Vec is the neural network model designed to represent code snippets as fixed-length vectors. It works by extracting paths from the abstract syntax tree (AST) of a code snippet and embedding them into a continuous vector space. Code2Vec captures both syntactic and semantic information, making it useful for tasks like method name prediction, variable renaming, and code similarity detection. The model's ability to create vector representations from code allows it to generalize across different programming tasks.

⑧ **Code2Seq.** Similar to Code2Vec, Code2Seq is a model that converts code into a sequence representation by extracting paths from the AST. However, instead of generating a single vector, Code2Seq outputs a sequence of tokens, making it more suitable for tasks like code summarization, where generating a description of code functionality is required. Code2Seq is particularly effective in tasks that require sequence generation, such as method name generation and documentation generation, by understanding the structure and meaning of the code.

⑨ **CodeBERT and GraphCodeBERT.** Pre-trained code models based on the BERT architecture, designed for program understanding, code summarization, vulnerability detection, and code search. GraphCodeBERT enhances the model's understanding of code by incorporating graph neural networks (GNNs) to capture both syntax and semantic structures in code.

⑩ **CodeT5 and CodeT5+.** CodeT5 is a transformer-based model specifically designed for code-related tasks, built on the T5 architecture. CodeT5 follows a text-to-text format, allowing it to handle tasks such as code summarization, code generation, code translation, and code repair. It is pre-trained on a large dataset across multiple programming languages, which helps it capture both syntax and semantics. CodeT5 also utilizes masked span prediction and other pre-training techniques to improve its understanding of code structure. Its versatility and strong performance make it effective in various neural code model applications.

⑪ **PLBART.** PLBART is a pre-trained sequence-to-sequence model specifically designed for programming languages based on the BART (Bidirectional and Auto-Regressive Transformers) architecture. PLBART is pre-trained using denoising autoencoding tasks on both programming languages and natural language. It excels in various code-related tasks such as code translation, generation, and repair by learning both the syntactic and semantic aspects of programming languages. Its bidirectional and autoregressive properties make it a powerful model for code understanding and manipulation tasks.

⑫ **CodeGen.** CodeGen is a Transformer-based model designed specifically for code generation tasks. It is pre-trained on a large-scale dataset of mixed programming and natural languages, learning to generate corresponding code from natural language descriptions. CodeGen is suited for tasks such as automatic code completion, code generation, and code repair. Its autoregressive architecture allows it to predict subsequent code segments step by step during generation, improving the accuracy and syntactical consistency of the output.

⑬ **GPT-2, GPT-3.5, GPT-4.** GPT-2 is an autoregressive language model based on the Transformer, primarily designed for natural language processing tasks. However, its powerful generation capabilities have also been widely applied to code generation tasks. GPT-3.5 and GPT-4 are enhanced versions of GPT-2, with larger model parameters and a more extensive training dataset, allowing them to handle more complex code generation tasks. The GPT series not only excels at automatic code completion but also learns the semantics and structure of code through large-scale pre-training, being widely applied to code generation, code repair, and code search tasks. As the model size increases, GPT-4 shows stronger cross-domain capabilities, especially in the understanding and generation of complex code.

7.3 RQ4.3: Evaluation Metrics

To evaluate the performance of attack and defense techniques against CodeLMs, existing studies have proposed many evaluation metrics. In this section, we summarize and present these metrics. These metrics can be divided into two categories: those used for evaluating attack and defense techniques, and those used for assessing the performance of CodeLMs themselves.

7.3.1 Metrics used in attack and defense research on CodeLMs.

This section introduces the evaluation metrics used in attacks and defenses targeting CodeLMs. These metrics are typically employed to measure attack success rates, model robustness, and the effectiveness of defense methods. It is worth noting that these evaluation metrics are of interest to both attackers and defenders, as the same metrics can be used to assess the effectiveness of attack methods as well as the resilience of defense methods.

① **Backdoor False Positive Rate (FPR)**. *FPR* means the proportion of positive samples with wrong predictions to all positive samples. It is also called the false recognition rate and false alarm rate. This metric is used to evaluate the performance of a classification model and is often employed by backdoor defense methods. The calculation formula of *FPR* is as follows:

$$FPR = \frac{FP}{TN + FP} \quad (7)$$

② **Attack Success Rate (ASR)**. *ASR* refers to the proportion of poisoned samples that are successfully predicted by the model to target label, which is frequently used to measure the effect of backdoor attacks or defenses. In general, the higher the *ASR*, the better the performance of the attack method, and vice versa. *M* refers to clean model, *M'* refers to backdoored model. *C* refers to clean data, *C'* refers to poisoned data. The calculation formula of *ASR* is as follows:

$$ASR = \frac{|\{C|M'(C') = y_{target} \wedge M(C) \neq y_{target}\}|}{|\{C\}|} \quad (8)$$

③ **Average Normalized Rank (ANR)**. This metric is used to measure the effectiveness of backdoor attacks or defenses on code retrieval models. *ANR* indicates how much the attack can improve the retrieval ranking of poisoned samples. The smaller *ANR* value represents the more effective attack method, and vice versa. Specifically, *s'* stands for the length of the injected trigger code snippet. *|S|* stands for the length of the full ranking list. The calculation formula of *ANR* is as follows:

$$ANR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{Rank(Q_i, s')}{|S|} \quad (9)$$

Therefore, the number of queries is not only an important indicator for evaluating the efficiency of black-box adversarial attack methods but also a crucial metric for assessing the effectiveness of corresponding adversarial defense methods.

④ **Number of Queries**. This represents the average number of successful queries against the target model in an adversarial attack. For black-box adversarial attack methods, queries are the only way to access the target model. Therefore, the number of queries is not only an important indicators for evaluating the efficiency of black-box adversarial attack methods, but also an important metric for assessing the effectiveness of corresponding adversarial defense methods. Specifically, *q_i* represents the number of queries for the *i_{th}* successful attack. In detail, *i* ∈ {*j* | *f*(*j*) = 1}. The calculation formula of Number of Queries is as follows:

$$Query = \frac{\sum q_i}{\sum f(i)} \quad (10)$$

$$f(j) = \begin{cases} 1, & \text{if } M(C_j^{adv}) \neq y_j \wedge M(C_j) = y_j \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

⑤ **Ratio of Perturbation ($Pert$)**. This metric is used to represent the proportion of perturbation injected into the original source code during an adversarial attack. A lower perturbation ratio indicates that the generated adversarial samples have fewer perturbations. Specifically, C_i^{adv} denotes for the adversarial example of C_i . $t(\cdot)$ denotes the number of tokens. The calculation formula of $Pert$ is as follows:

$$Pert = \frac{\sum t(C_i^{adv}) - t(C_i^{adv} \cap C_i)}{\sum t(C_i)} \quad (12)$$

⑥ **Relative Degradation (R_d)**. R_d is used to evaluate the performance degradation of the model in the attack state. Specifically, $refs$ denotes for reference comment. y denotes for original output. y' denotes the output of the perturbed program. The calculation formula of R_d is as follows:

$$R_d = \frac{BLEU(y, refs) - BLEU(y', refs)}{BLEU(y, refs)} \quad (13)$$

⑦ **Valid Rate (V_r)**. V_r is defined as the percentage of adversarial samples that can be compiled [169]. This metric is used to evaluate the quality of the adversarial samples generated during adversarial attacks or adversarial defenses, as well as the efficiency of the generation process. Specifically, $Count_{valid}$ is the number of adversarial samples that can pass the compile check of the program. $Count_{all}$ is the number of all samples. The calculation formula of V_r is as follows:

$$V_r = \frac{Count_{valid}}{Count_{all}} \quad (14)$$

⑧ **Success Rate (S_r)**. S_r is defined as the product of relative degradation (R_d) and valid rate (V_r), providing a comprehensive index of attack efficiency and sample quality generated. Essentially, a higher success rate indicates that the corresponding method can generate effective adversarial samples with better attack capabilities. The calculation formula of S_r is as follows:

$$S_r = R_d * V_r \quad (15)$$

⑨ **Variable Change Rate (VCR)**. VCR is the proportion of variables that an attacker needs to modify for a successful attack. The lower the VCR , the better the attack method, and the more efficient the adversarial sample can be found only by modifying a small number of variables. Conversely, the higher the VCR , the more effective the applied defense method. In the formula for VCR , n_i refers to the number of variables that an attacker has successfully modified in the i -th adversarial sample, while m_i represents the total number of variables present in the i -th sample. The calculation formula of VCR is as follows:

$$VCR = \frac{\sum_i n_i}{\sum_i m_i} \quad (16)$$

7.3.2 Metrics used in performance on CodeLMs.

This section focuses on the metrics used to evaluate the performance of CodeLMs themselves, such as the accuracy of generated code, efficiency, and how closely the generated code aligns with human-written code.

① **Clean Accuracy (CA)**. Clean Accuracy means the accuracy of the model without any interference or attack. This metric is used to evaluate the performance of a model on clean data and is commonly used in backdoor attacks and adversarial attacks. For the backdoor attack, if the accuracy of the poisoned model is almost the same as that of the normal model, it can show that the attack method has good concealment. M' refers to the

backdoor model which is trained with the poisoned dataset. C refers to clean data, y refers to ground-true label. The calculation formula of CA is as follows:

$$CA = \frac{|\{C|M'(C) = y\}|}{|\{C\}|} \quad (17)$$

② **F1-score**. F1-score is the harmonic mean of precision and recall, and is commonly used to evaluate the performance difference of neural code classification models before and after attacks. The calculation formula of $F1 - score$ is as follows:

$$F1 - score = \frac{2 * precision * recall}{precision + recall} \quad (18)$$

③ **Mean Reciprocal Rank (MRR)**. MRR is the average of the reciprocal ranks of a set of search results, this metric provides an overall perspective to assess the validity of a search result. In general, the higher the MRR value, the better the effect of model retrieval. Specifically, $|Q|$ is the size of the query set. $Rank_q$ is the rank of the code snippet that corresponds to the query q . The calculation formula of MRR is as follows:

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{Rank_q} \quad (19)$$

④ **Bilingual Evaluation Understudy (BLEU)**. BLEU [130] is used to evaluate the quality of machine-generated text by comparing the similarity between a machine-generated text and a human-generated reference text. It is a commonly used performance evaluation metric in generation tasks such as code summarization, code generation, and code completion. Specifically, r is the number of words in a reference translation. c is the number of words in a candidate translation. The calculation formula of $BLEU$ is as follows:

$$BLEU = BP \cdot \exp\left(\sum_{n=1}^N \omega_n \log(P_n)\right) \quad (20)$$

BP stands for brevity penalty. When $r \leq c$, the brevity penalty $BP = 1$, meaning that we do not punish long candidates and only punish short candidates. When $r > c$, the brevity penalty $BP = e^{1-r/c}$.

$$BP = \begin{cases} 1, & c > r \\ e^{1-\frac{r}{c}}, & c \leq r \end{cases} \quad (21)$$

P_n is weighted n-gram precision, this quantity measures how many n-grams in the reference sentence are reproduced by the candidate sentence. where n means the length of the n-gram, $C(i, i+n)$ is the n-gram from the position i to the position $i+n$, and $Countclip(C(i, i+n))$ is the maximum number of n-grams co-occurring in a candidate code and a set of reference codes. μ_n^i denotes the weights of different keywords or n-gram.

$$p_n = \frac{\sum_{C \in \text{Candidates}} \sum_{i=1}^l \mu_n^i \cdot \text{Countclip}(C(i, i+n))}{\sum_{C' \in \text{Candidates}} \sum_{i=1}^l \mu_n^i \cdot \text{Count}(C'(i, i+n))} \quad (22)$$

⑤ **CodeBLEU**. *CodeBLEU* is used to evaluate the quality of code generated by the model. This metric considers the functional and structural information of the code and weights the abstract syntax tree matching and data-flow matching scores. In the field of code synthesis, common evaluation metrics like *BLEU* and perfect accuracy have limitations. *BLEU*, originally designed for natural language, overlooks the syntactic and semantic features of code, while perfect accuracy is too strict, underestimating outputs that share the same semantic logic. To solve this problem, a new automatic evaluation metric called *CodeBLEU* is created by Ren et al. [140]. It absorbs the

strengths of *BLEU* in n-gram matching and further injects code syntax through an abstract syntax tree (AST) and code semantics through the data stream. The calculation formula of *CodeBLEU* is as follows:

$$CodeBLEU = \alpha \cdot BLEU + \beta \cdot BLEU_{weight} + \gamma \cdot Match_{ast} + \delta \cdot Match_{df} \quad (23)$$

where *BLEU* is calculated by standard *BLEU* [130], *BLEU_{weight}* is the weighted n-gram match, obtained by comparing the hypothesis code and the reference code tokens with different weights, *Match_{ast}* is the syntactic AST match, exploring the syntactic information of code, and *Match_{df}* is the semantic data-flow match, considering the semantic similarity between the hypothesis and the reference. The weighted n-gram match and the syntactic AST match are used to measure grammatical correctness, and the semantic data-flow match is used to calculate logic correctness.

⑥ **Top k Success Rate** (*SuccessRate@k*). This metric measures the average percentage of the retrieved model outputs that ranked in the top *k* for the corresponding code snippet. In general, the higher the *SuccessRate@k*, the better the code retrieval model. Specifically, $|Q|$ is the size of the query set. *Rank_q* refers to the ranking of the code snippet corresponding to the query *q*. $\delta(\cdot)$ is a function that returns 1 if the input is true, otherwise 0. The calculation formula of *SuccessRate@k* is as follows:

$$SuccessRate@k = \frac{1}{|Q|} \sum_{q \in Q} \delta(Rank_q \leq k) \quad (24)$$

7.4 RQ4.4: Artifact Accessibility

In this section, we explore the open-source code repositories provided by the reviewed CodeLM studies in their papers. The open-source code repositories offered by the authors will assist other researchers in reproducing the study and facilitate further utilization and research. The quality of these open-source code repositories depends on reproducibility, which refers to obtaining the same results using generally the same methods. Reproducibility is crucial for assessing the quality of research and validating its credibility, as it enhances our confidence in the results and helps us distinguish between reliable and unreliable findings. This is particularly important for technical methods that rely on datasets, data processing methods, and hyperparameter settings.

To verify the reproducibility of these CodeLM studies, we systematically review each paper to ensure that the provided code links are publicly accessible and valid. We compile information from all the papers and include those with open-source code repositories along with their links in Table 13 to facilitate future CodeLM research.

✎ **Summary** ► In this section, we first introduce the experimental datasets used in CodeLM security research, summarizing the commonly adopted datasets in both backdoor and adversarial attack and defense scenarios. Next, we summarize the CodeLMs evaluated in existing studies. We then introduce the evaluation metrics used in attacks and defenses involving CodeLMs, as well as the metrics used to assess the overall performance of the models themselves. Finally, we discuss artifact accessibility, highlighting the importance of open-source code repositories to ensure transparency and reproducibility in CodeLM research, thereby facilitating the broader advancement of the field. ◀

8 CHALLENGES AND OPPORTUNITIES

After analyzing and summarizing the existing research on CodeLMs, we can suggest that further research will have both practical and academic significance. From a practical application perspective, these excellent techniques will undoubtedly help us improve the security of CodeLMs. On the other hand, the security of CodeLMs still faces many challenges that need to be addressed. We pose some of the challenges and opportunities identified from the relevant papers below.

Table 13. Replication open-source code repository links provided in CodeLM studies.check

Technique	Year	Type	URL
Ramakrishnan et al. [138]	2022	Data poisoning Pre-training defense	https://github.com/goutham7r/backdoors-for-code
Schuster et al. [142]	2021	Data poisoning Model poisoning	https://github.com/ClonedOne/MalwareBackdoors
Severi et al. [144]	2021	Data poisoning	https://github.com/ClonedOne/MalwareBackdoors
Wan et al. [172]	2022	Data poisoning	https://github.com/CGCL-codes/naturalcc
BadCode [154]	2023	Data poisoning	https://github.com/wssun/BADCODE
TrojanSQL [210]	2023	Data poisoning	https://github.com/jc-ryan/trojan-sql
Cotroneo et al. [36]	2023	Data poisoning	https://github.com/dessertlab/Targeted-Data-Poisoning-Attacks
Li et al. [89]	2024	Data poisoning Pre-training defense	https://github.com/LJ2lijia/CodeDetector
AFRAIDDOOR [193]	2024	Data poisoning	https://github.com/yangzhou6666/adversarial-backdoor-for-code-models
TrojanPuzzle	2024	Data poisoning	https://github.com/microsoft/CodeGenerationPoisoning
FDI [159]	2024	Data poisoning	https://github.com/v587su/FDI
LateBA [199]	2024	Data poisoning	https://github.com/yxy-whu/LateBA
Li et al. [94]	2023	Model poisoning	https://github.com/Lyz1213/Backdoored_PPLM
PELICAN [216]	2023	Model poisoning	https://github.com/ZhangZhuoJTU/Pelican
DeCE [188]	2024	In-training defense	https://github.com/NTDXYG/DeCE
DAMP [197]	2020	White-box attack	https://github.com/tech-srl/adversarial-examples
Bielik et al. [20]	2020	White-box attack	https://github.com/eth-sri/robust-code
Srikant et al. [150]	2021	White-box attack	https://github.com/ALFA-group/adversarial-code-generation
Quiring et al. [134]	2019	Black-box attack	https://github.com/EQuiw/code-imitator
MHM [206]	2020	Black-box attack	https://github.com/Metropolis-Hastings-Modifier/MHM
Pour et al. [131]	2021	Black-box attack	https://github.com/MaryamVP/Guided-Mutation-ICST-2021
Ferretti et al. [42]	2021	Black-box attack	https://github.com/Martisal/adversarialGE
Anand et al. [15]	2021	Black-box attack	https://github.com/mtensor/neural_sketch
ACCENT [220]	2021	Black-box attack	https://github.com/zhangxq-1/ACCENT-repository
CARROT [205]	2022	Black-box attack Input modification	https://github.com/SEKE-Adversary/CARROT
ALERT [191]	2022	Black-box attack	https://github.com/soarsmu/attack-pretrain-models-of-code
Li et al. [96]	2022	Black-box attack Input modification	https://github.com/RoPGen/RoPGen
CodeAttack [75]	2023	Black-box attack	https://github.com/reddy-lab-code-research/CodeAttack
RNNS [208]	2023	Black-box attack	https://github.com/18682922316/RNNS-for-code-attack
CODA [167]	2023	Black-box attack	https://github.com/tianzhaotju/CODA
CodeBERT-Attack [207]	2023	Black-box attack	https://gitfront.io/r/DrLC5417/5pBsYXgKinB3/CodeBERT-Attack/tree/codebert/
RADAR [189]	2024	Black-box attack Additional Models	https://github.com/NTDXYG/RADAR
STRUCK [215]	2024	Black-box attack	https://github.com/zhanghaha1707/STRUCK
CodeTAE [190]	2024	Black-box attack	https://github.com/yyl-github-1896/CodeTAE
MOAA [218]	2024	Black-box attack	https://github.com/COLA-Laboratory/MOAA
MalwareTotal [55]	2024	Black-box attack	https://github.com/Optimus-He/MalwareTotal
Henkel et al. [58]	2022	Input modification	https://github.com/jjhenkel/averloc
Cristina et al. [71]	2023	Input modification	https://github.com/dessertlab/Robustness-of-AI-Offensive-Code-Generators
SVEN [53]	2023	Model modification	https://github.com/eth-sri/sven
SPACE [95]	2022	Additional models	https://github.com/EricLee8/SPACE

8.1 Challenges and Opportunities in Attacks against CodeLMs

Stealthiness of Backdoor Triggers. In backdoor attacks on CodeLMs, the stealthiness of triggers is a key focus for attackers. They continually explore more covert trigger designs, evolving from early methods like dead code [138, 172] to variable/function names [89, 154], and even adaptive triggers [193], aiming to inject increasingly stealthy triggers into code. However, comprehensively evaluating the stealthiness of triggers remains a challenging task. Current research methods typically focus on specific aspects, such as syntactic or semantic visibility, or rely on human experiments. For instance, BadCode [154] outperforms Wan et al. [172] in human experiment evaluations, while AFRAIDDOOR [193] uses automated evaluations to demonstrate that adaptive triggers are more stealthy. Nonetheless, these methods do not cover all possible detection dimensions, leaving room for improvement in both evaluation metrics and techniques.

Backdoor Injection Methods for Large Language Models. The effectiveness of backdoor injection methods also faces several challenges. Existing methods often fail to work effectively for large code models. Current injection approaches rely on two scenarios: first, attackers cannot control the training process of the model, but the model is trained using poisoned data; second, attackers can control the model's training process. However, large code language models, such as Codex and GPT-4, are typically closed-source, meaning attackers cannot control the training process or trace the training data. For open-source large CodeLMs, the cost of injecting backdoors through training or fine-tuning significantly increases. Additionally, as large CodeLMs become more complex and robust, it becomes increasingly difficult for attackers to insert backdoors without detection, and the effectiveness of these backdoors tends to diminish as the model size grows.

Syntactic Correctness and Semantic Preservation of Adversarial Samples. In adversarial attacks on CodeLMs, the generated adversarial samples need to maintain the syntactic correctness and preserve the semantics of the code. Current techniques typically achieve this by modifying/replacing variable names or applying transformations that do not alter the code's semantics. However, existing evaluation methods do not fully account for the syntactic correctness and semantic preservation of these adversarial samples after perturbation. Even if some adversarial samples appear to maintain the code's semantics on the surface, they may introduce syntactic or logical errors during execution. Therefore, comprehensively evaluating the syntactic correctness and semantic consistency of adversarial samples remains a significant challenge.

Stealthiness of Adversarial Perturbations. The stealthiness of adversarial perturbations is a key metric for assessing the quality of adversarial samples. In both white-box and black-box attacks, current techniques often use similarity-based metrics (e.g., CodeBLEU) to evaluate the stealthiness or naturalness of adversarial samples. However, these metrics are not always ideal. Some perturbations may be imperceptible to humans but show significant differences in similarity metrics, and vice versa. Moreover, the current metrics do not cover all aspects that impact the stealthiness of adversarial samples, particularly when evaluating the real-world effects of the perturbations. Thus, designing more comprehensive metrics for evaluating stealthiness remains a challenge in this field.

In-depth Understanding of the Principles Behind Attacks on CodeLMs. The development of explainability may help us better understand the underlying principles of backdoor and adversarial attacks. Since the inception of neural networks, they have faced the issue of low interpretability. Small changes in model parameters can significantly impact prediction results, and people cannot directly comprehend how neural networks operate. In recent years, explainability has become a pressing area in deep learning. How to deeply understand neural networks (including CodeLMs) themselves and explain how inputs affect outputs remain urgent problems to be addressed. Currently, some work is being done to provide security and robustness proofs for adversarial attacks. However, more in-depth research is needed to explain the reasons behind prediction results, making the training and prediction processes no longer black boxes. Explainability can not only enhance the security of CodeLMs but also unveil the mysteries of the model, making it easier for us to understand its mechanisms. However, this could

also benefit attackers. For example, for backdoor attacks, attackers may analyze the model for explainability to select triggers that are more likely to cause harm. For adversarial attacks, attackers could exclude input ranges that have been proven to be secure, thereby narrowing the search space and more effectively finding adversarial samples. They could also construct targeted attacks by gaining a deeper understanding of the model. Nevertheless, this field should not stagnate, as black-box models do not guarantee safety. Therefore, as explainability improves, the security of CodeLMs may be enhanced in a convoluted manner.

Exploring the Security of CodeLLMs from Other Attack Perspectives. In addition to backdoor and adversarial attack threats, recent research has also explored other types of attack threats against CodeLLMs, such as privacy leakage [7, 64, 122, 195], membership inference attacks [7, 194, 213] and watermark attack [185]. Exploring new attack threats is crucial for a deeper understanding of the security of CodeLLMs. By studying these attack methods, researchers can identify potential vulnerabilities that the model may face in real-world applications, reveal its weaknesses in various scenarios, and better understand the model's behavior. Therefore, conducting more research on different types of attacks will help both academia and industry more comprehensively assess the security of CodeLLMs and provide a foundation for future security research.

8.2 Challenges and Opportunities in Defenses against Attacks on CodeLMs

Balancing the Effectiveness of Backdoor Defenses with Maintaining Model Performance Stability.

In Section 5.2, we present the existing backdoor defense techniques for CodeLMs. These defense techniques are typically designed to protect different phases of CodeLMs from backdoor attacks. However, accurately and efficiently detecting poisoned samples or removing backdoors from the CodeLMs, while ensuring the model's regular performance, still presents many challenges. Firstly, for pre-training defense techniques [89, 138], they primarily detect poisoned samples by identifying "outlier" characteristics in the training data. However, this approach often results in a high false positive rate and requires significant computational resources, making it difficult to achieve both high accuracy and efficiency in detecting poisoned samples. For complex triggers, such as variable/function name extension or adaptive triggers, current defense techniques face even greater difficulty in detecting and removing them. Secondly, for post-training defense techniques, they primarily remove backdoors from the model or inputs through unlearning or input filtering. As the model size increases, these techniques require an unacceptable amount of time and computational resources. Additionally, these techniques can cause a certain level of damage to the model's regular performance. Applying these methods in real-world scenarios presents significant challenges.

Balancing Model Performance and Robustness in Adversarial Defense Techniques. In Section 5.3, we introduced the existing adversarial defense techniques. The adversarial defense methods for CodeLMs primarily use adversarial training or data augmentation techniques to improve the robustness of the model. However, maintaining model performance while enhancing robustness and security remains a significant challenge. Current research employs gradient-based perturbations to transform programs in the worst-case scenarios. Compared to random perturbations, this approach is more likely to produce robust models. However, these methods often reduce the model's normal performance while improving robustness. Although some studies attempt to enhance both model robustness and performance by combining gradient-based adversarial training with programming language data features or by designing specific loss functions, these methods tend to require more computational resources.

Multi-Scenario Defense against Attacks on CodeLMs. Compared to backdoor attacks and adversarial attacks on CodeLMs, research progress on backdoor and adversarial defenses has lagged behind. Therefore, there are still numerous research opportunities in addressing attacks across different scenarios. Beyond single defense scenarios, multi-scenario defense techniques hold greater potential. From the perspective of the CodeLM lifecycle,

implementing defense strategies that address both data protection and model protection before, during, and after model training in a mixed-scenario approach can further enhance the security of CodeLMs.

Explainability in Defending against Attacks CodeLMs. Advancements in explainability can help address the lag in defense methods. Since current research has yet to fully understand CodeLMs (e.g., why inputs with triggers are predicted as target results, and how different data affect model weights), discovering vulnerabilities is often easier than preventing attacks in advance. This leads to a certain lag in the security of CodeLMs. If we can thoroughly understand code models, it is believed that defenses will surpass or at least keep pace with advancements in attack techniques.

Defending Against Emerging Security Threats in CodeLLMs. With the emergence of new security threats such as privacy leakage and membership inference attacks, it has become crucial to extend existing defense techniques to address these risks. Although previous studies have proposed defense techniques to enhance the privacy protection of CodeLLMs [8, 43, 91], these defenses still face challenges in the face of the continuously emerging attack threats. Therefore, developing defense strategies that effectively address the evolving threats is essential to maintaining the security of CodeLLMs in real-world applications.

Overall, the security threats to CodeLMs can be seen as an ongoing and evolving struggle between attackers and defenders, with neither side able to secure absolute dominance. Nevertheless, both parties can leverage new techniques and applications to gain a strategic advantage. On the attacker's side, an effective strategy involves probing for new attack vectors, uncovering novel attack scenarios, pursuing diverse objectives, and expanding the scope and impact of attacks. On the defender's side, employing a combination of defense mechanisms presents a promising approach to mitigating attack risks. However, such integration may introduce additional computational or system overhead, which must be carefully addressed during the design phase.

9 THREATS TO VALIDITY

Potential Impact of Database Selection on the Inclusion of Relevant Papers. We conduct automated searches across six widely used databases, including Google Scholar, ACM Digital Library, IEEE Xplore, Springer, DBLP, and ArXiv, all of which are standard and commonly used online engines or databases. One potential issue is that relevant papers may be omitted if the search relies solely on these databases. To mitigate this impact, we also employ both forward and backward snowballing search methods to further enhance the search results. Through snowballing, we identify papers with transitive dependencies and expand our paper collection. Specifically, backward snowballing helps us uncover relevant research in the reference lists of the collected papers, while forward snowballing tracks subsequent studies that cite the papers we have collected, further extending the scope of our search. This approach ensures comprehensive coverage of relevant literature, reducing the risk of omissions.

Potential Impact of Non-peer-reviewed Papers on the Reliability of the Conclusions. We include ArXiv in our search databases, considering that some researchers are inclined to disclose the latest techniques on ArXiv in advance. As a result, the final selection of papers may include non-peer-reviewed papers. One potential impact is that these non-peer-reviewed papers may affect the reliability of our research conclusions. To mitigate this impact, we ensure the quality of the selected papers. We apply five exclusion criteria to rigorously filter all the collected papers. For the final set of selected papers, the first two authors review them thoroughly and multiple times to ensure that each paper has a clear research motivation, provides detailed descriptions of the techniques, presents comprehensive experimental setups, and clearly confirms the experimental findings, thus ensuring that low-quality papers are excluded from our study. Additionally, our research conclusions primarily encompass the development trends in the field (Section 3) and summaries of various research directions (Sections 4 to 7). These conclusions do not include specific technique comparisons between the papers. Therefore, the inclusion of non-peer-reviewed papers does not significantly impact the reliability of our conclusion.

Potential Bias in Study Selection. In the study selection process, we establish five inclusion and exclusion criteria to filter the papers, combining automated and manual procedures. In the first and second phases of the study selection, we use automated filtering methods, as the inclusion and exclusion criteria in these stages are easily aligned with relevant fields in the BibTeX records. However, in the third to fifth phases, due to the difficulty in directly correlating the inclusion and exclusion criteria with specific fields in the BibTeX records, the risk of mislabeling papers is higher. Therefore, we use manual filtering. During this process, we carefully review the papers multiple times to mitigate potential biases introduced by manual filtering. However, manual filtering may still be influenced by the researchers' subjective judgment biases, which could affect the accuracy of the quality assessment of papers. To address these concerns, the first two authors carefully review the selected papers multiple times, focusing on the abstract, introduction, related work, experiments, and conclusion sections, to fully understand the research objectives and categorization of each paper. This step aims to improve the accuracy of paper selection, minimize omissions, and reduce the potential bias in the study selection process.

10 CONCLUSION

This paper systematically reviews the literature on the security of CodeLMs and provides a detailed analysis of recent publications. The focus of this paper is on the security threats posed by CodeLMs in the context of backdoor attacks and adversarial attacks. Backdoor attacks are categorized into data poisoning attacks and model poisoning attacks, while adversarial attacks are divided into white-box adversarial attacks and black-box adversarial attacks. A comprehensive summary and analysis of research on attacks targeting different categories of CodeLMs are presented. Subsequently, the paper reveals the defense gaps corresponding to backdoor attacks and adversarial attacks. Additionally, a thorough analysis of the challenges faced by current research in this field is provided, offering valuable guidance for further investigation. Finally, the paper organizes and summarizes commonly used datasets, language models, evaluation metrics, and the accessibility of tools in the field of security for CodeLMs, facilitating the convenience of readers.

ACKNOWLEDGMENTS

This research is supported by the National Key Research and Development Program of China (2024YFF0908001) and the National Natural Science Foundation of China (U24A20337 and 62372228), the National Research Foundation, Singapore, and DSO National Laboratories under the AI Singapore Programme (AISG Award No: AISG2-GC-2023-008), as well as the National Research Foundation, Singapore, and the Cyber Security Agency of Singapore under the National Cybersecurity R&D Programme (NCRP25-P04-TAICeN). Any opinions, findings, conclusions, or recommendations expressed in this paper are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore, or the Cyber Security Agency of Singapore.

REFERENCES

- [1] Mohammed Abuhamad, Tamer AbuHmed, Aziz Mohaisen, and DaeHun Nyang. 2018. Large-Scale and Language-Oblivious Code Authorship Identification. In *Proceedings of the 2018 ACM Conference on Computer and Communications Security*. ACM, Toronto, ON, Canada, 101–114.
- [2] Mohammed Abuhamad, Changhun Jung, David Mohaisen, and DaeHun Nyang. 2023. SHIELD: Thwarting Code Authorship Attribution. *arXiv abs/2304.13255* (2023), 1–13.
- [3] Hojjat Aghakhani, Wei Dai, Andre Manoel, Xavier Fernandes, Anant Kharkar, Christopher Kruegel, Giovanni Vigna, David Evans, Ben Zorn, and Robert Sim. 2024. TrojanPuzzle: Covertly Poisoning Code-Suggestion Models. In *Proceedings of the IEEE Symposium on Security and Privacy*. IEEE, San Francisco, CA, USA, 1122–1140.
- [4] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2020. A Transformer-based Approach for Source Code Summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 4998–5007.

- [5] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified Pre-training for Program Understanding and Generation. In *Proceedings of the 25th Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Online, 2655–2668.
- [6] Areeg Ahmed, Shahira Azab, and Yasser Abdelhamid. 2023. Source-Code Generation Using Deep Learning: A Survey. In *Proceedings of the 22nd Conference on Artificial Intelligence (Lecture Notes in Computer Science, Vol. 14116)*. Springer, Faial Island, Azores, 467–482.
- [7] Ali Al-Kaswan, Maliheh Izadi, and Arie van Deursen. 2024. Traces of Memorisation in Large Language Models for Code. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. ACM, Lisbon, Portugal, 78:1–78:12.
- [8] Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Muñoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, et al. 2023. SantaCoder: don't reach for the stars! *arXiv abs/2301.03988* (2023), 1–22.
- [9] Uri Alon, Shaked Brody, Omer Levy, and Eran Yahav. 2019. code2seq: Generating Sequences from Structured Representations of Code. In *Proceedings of the 7th International Conference on Learning Representations*. OpenReview.net, New Orleans, LA, USA, 1–13.
- [10] Uri Alon, Roy Sadaka, Omer Levy, and Eran Yahav. 2020. Structural Language Models of Code. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, Virtual Event, 245–256.
- [11] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. 2019. code2vec: learning distributed representations of code. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 40:1–40:29.
- [12] Moustafa Alzantot, Yash Sharma, Ahmed Elgohary, Bo-Jhang Ho, Mani B. Srivastava, and Kai-Wei Chang. 2018. Generating Natural Language Adversarial Examples. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 2890–2896.
- [13] Amazon. 2023. Amazon CodeWhisperer. <https://aws.amazon.com/cn/codewhisperer/>.
- [14] Mrinal Anand, Pratik Kayal, and Mayank Singh. 2021. Adversarial Robustness of Program Synthesis Models. In *Proceedings of the 35th Advances in Programming Languages and Neurosymbolic Systems Workshop*. CEUR-WS.org, Sydney, Australia, 1–7.
- [15] Mrinal Anand, Pratik Kayal, and Mayank Singh. 2021. On adversarial robustness of synthetic code generation. *arXiv abs/2106.11629* (2021), 1–16.
- [16] Hyrum S. Anderson and Phil Roth. 2018. EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models. *arXiv abs/1804.04637* (2018), 1–8.
- [17] Daniel Arp, Michael Spreitzerbarth, Malte Hubner, Hugo Gascon, and Konrad Rieck. 2014. DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket. In *Proceedings of the 21st Annual Network and Distributed System Security Symposium*. The Internet Society, San Diego, California, USA, 1–16.
- [18] MD Awal, Mrigank Rochan, and Chanchal K Roy. 2024. Investigating Adversarial Attacks in Software Analytics via Machine Learning Explainability. *arXiv abs/2408.04124* (2024), 1–32.
- [19] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural Machine Translation by Jointly Learning to Align and Translate. In *Proceedings of the 3rd International Conference on Learning Representations*. OpenReview.net, San Diego, CA, USA, 1–15.
- [20] Pavol Bielek and Martin T. Vechev. 2020. Adversarial Robustness for Code. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, Virtual Event, 896–907.
- [21] Marc Brockschmidt, Miltiadis Allamanis, Alexander L. Gaunt, and Oleksandr Polozov. 2019. Generative Code Modeling with Graphs. In *Proceedings of the 7th International Conference on Learning Representations*. OpenReview.net, New Orleans, LA, USA, 1–24.
- [22] Gianluca Capozzi, Daniele Cono D'Elia, Giuseppe Antonio Di Luna, and Leonardo Querzoni. 2023. Adversarial Attacks against Binary Similarity Systems. *arXiv abs/2303.11143* (2023), 1–23.
- [23] Nicholas Carlini and David A. Wagner. 2017. Towards Evaluating the Robustness of Neural Networks. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy*. IEEE Computer Society, San Jose, CA, USA, 39–57.
- [24] Anirban Chakraborty, Manaar Alam, Vishal Dey, Anupam Chattopadhyay, and Debdeep Mukhopadhyay. 2021. A survey on adversarial attacks and defences. *CAAI Transactions on Intelligence Technology* 6, 1 (2021), 25–45.
- [25] Bryant Chen, Wilka Carvalho, Nathalie Baracaldo, Heiko Ludwig, Benjamin Edwards, Taesung Lee, Ian M. Molloy, and Biplav Srivastava. 2019. Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering. In *Proceedings of the 33rd AAAI Conference on Artificial Intelligence (CEUR Workshop Proceedings, Vol. 2301)*. CEUR-WS.org, Honolulu, Hawaii, 1–10.
- [26] Chuanshuai Chen and Jiazhui Dai. 2021. Mitigating backdoor attacks in LSTM-based text classification systems by Backdoor Keyword Identification. *Neurocomputing* 452 (2021), 253–262.
- [27] Long Chen, Wujing Zhan, Wei Tian, Yuhang He, and Qin Zou. 2019. Deep Integration: A Multi-Label Architecture for Road Scene Recognition. *IEEE Trans. Image Process.* 28, 10 (2019), 4883–4898.
- [28] Penglong Chen, Zhen Li, Yu Wen, and Lili Liu. 2022. Generating Adversarial Source Programs Using Important Tokens-based Structural Transformations. In *Proceedings of the 26th International Conference on Engineering of Complex Computer Systems*. IEEE, Hiroshima, Japan, 173–182.
- [29] Yuchen Chen and Weisong Sun. 2024. Artifacts of this paper. site: <https://github.com/wssun/TiSE-LM4Code-Security>. Accessed: 2024-10-01.

- [30] Zhenpeng Chen, Yanbin Cao, Yuanqiang Liu, Haoyu Wang, Tao Xie, and Xuanzhe Liu. 2020. A comprehensive study on challenges in deploying deep learning based software. In *Proceedings of the 28th Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Virtual Event, USA, 750–762.
- [31] Zhenpeng Chen, Yanbin Cao, Xuan Lu, Qiaozhu Mei, and Xuanzhe Liu. 2019. SentiMoji: an emoji-powered learning approach for sentiment analysis in software engineering. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Tallinn, Estonia, 841–852.
- [32] Zhenpeng Chen, Sheng Shen, Ziniu Hu, Xuan Lu, Qiaozhu Mei, and Xuanzhe Liu. 2019. Emoji-Powered Representation Learning for Cross-Lingual Sentiment Classification. In *The World Wide Web Conference*. ACM, San Francisco, CA, USA, 251–262.
- [33] Zhenpeng Chen, Huihan Yao, Yiling Lou, Yanbin Cao, Yuanqiang Liu, Haoyu Wang, and Xuanzhe Liu. 2021. An Empirical Study on Deployment Faults of Deep Learning Based Mobile Applications. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering*. IEEE, Madrid, Spain, 674–685.
- [34] Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. On the Properties of Neural Machine Translation: Encoder-Decoder Approaches. In *Proceedings of the 8th Workshop on Syntax, Semantics and Structure in Statistical Translation*. Association for Computational Linguistics, Doha, Qatar, 103–111.
- [35] YunSeok Choi, Hyojun Kim, and Jee-Hyong Lee. 2022. TABS: Efficient Textual Adversarial Attack for Pre-trained NL Code Model Using Semantic Beam Search. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 5490–5498.
- [36] Domenico Cotroneo, Cristina Improta, Pietro Liguori, and Roberto Natella. 2024. Vulnerabilities in AI Code Generators: Exploring Targeted Data Poisoning Attacks. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*. ACM, Lisbon, Portugal, 280–292.
- [37] George E Dahl, Dong Yu, Li Deng, and Alex Acero. 2011. Context-dependent pre-trained deep neural networks for large-vocabulary speech recognition. *IEEE Transactions on audio, speech, and language processing* 20, 1 (2011), 30–42.
- [38] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Minneapolis, MN, USA, 4171–4186.
- [39] Xiaohu Du, Ming Wen, Zichao Wei, Shangwen Wang, and Hai Jin. 2023. An Extensive Study on Adversarial Attack against Pre-trained Models of Code. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, San Francisco, CA, USA, 489–501.
- [40] Chunrong Fang, Weisong Sun, Yuchen Chen, Xiao Chen, Zhao Wei, Qunjun Zhang, Yudu You, Bin Luo, Yang Liu, and Zhenyu Chen. 2024. Esale: Enhancing Code-Summary Alignment Learning for Source Code Summarization. *IEEE Trans. Software Eng.* 50, 8 (2024), 2077–2095.
- [41] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Proceedings of the Findings of the Association for Computational Linguistics (Findings of ACL, Vol. EMNLP 2020)*. Association for Computational Linguistics, Online Event, 1536–1547.
- [42] Claudio Ferretti and Martina Saletta. 2021. Deceiving neural source code classifiers: finding adversarial examples with grammatical evolution. In *Proceedings of the 21th Genetic and Evolutionary Computation Conference*. ACM, Companion Volume, Lille, France, 1889–1897.
- [43] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. In *Proceedings of the 11th International Conference on Learning Representations*. OpenReview.net, Kigali, Rwanda, 1–14.
- [44] Fengjuan Gao, Yu Wang, and Ke Wang. 2023. Discrete Adversarial Attack to Models of Code. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 172–195.
- [45] Xiuting Ge, Chunrong Fang, Xuanye Li, Weisong Sun, Daoyuan Wu, Juan Zhai, Shangwei Lin, Zhihong Zhao, Yang Liu, and Zhenyu Chen. 2024. Machine Learning for Actionable Warning Identification: A Comprehensive Survey. *Comput. Surveys* 1, 1 (2024), 1–34.
- [46] GitHub. 2023. GitHub Copilot. <https://copilot.github.com>.
- [47] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. 2017. BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain. *arXiv abs/1708.06733* (2017), 1–13.
- [48] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2018. Deep code search. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, Gothenburg, Sweden, 933–944. <https://doi.org/10.1145/3180155.3180167>
- [49] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *Proceedings of the 9th International Conference on Learning Representations*. OpenReview.net, Austria, 1–18.

- [50] Rajarshi Haldar, Lingfei Wu, Jinjun Xiong, and Julia Hockenmaier. 2020. A multi-perspective architecture for semantic code search. *arXiv abs/2005.06980* (2020), 1–6.
- [51] Xu Han, Zhengyan Zhang, Ning Ding, Yuxian Gu, Xiao Liu, Yuqi Huo, Jiezhong Qiu, Yuan Yao, Ao Zhang, Liang Zhang, Wentao Han, Minlie Huang, Qin Jin, Yanyan Lan, Yang Liu, Zhiyuan Liu, Zhiwu Lu, Xipeng Qiu, Ruihua Song, Jie Tang, Ji-Rong Wen, Jinhui Yuan, Wayne Xin Zhao, and Jun Zhu. 2021. Pre-trained models: Past, present and future. *AI Open* 2 (2021), 225–250.
- [52] Sakib Haque, Alexander LeClair, Lingfei Wu, and Collin McMillan. 2020. Improved Automatic Summarization of Subroutines via Attention to File Context. In *MSR '20: 17th International Conference on Mining Software Repositories*, , 2020. ACM, Seoul, Republic of Korea, 300–310.
- [53] Jingxuan He and Martin T. Vechev. 2023. Large Language Models for Code: Security Hardening and Adversarial Testing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. ACM, Copenhagen, Denmark, 1865–1879.
- [54] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE Computer Society, Las Vegas, NV, USA, 770–778.
- [55] Shuai He, Cai Fu, Hong Hu, Jiahe Chen, Jianqiang Lv, and Shuai Jiang. 2024. MalwareTotal: Multi-Faceted and Sequence-Aware Bypass Tactics against Static Malware Detection. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. ACM, Lisbon, Portugal, 172:1–172:12.
- [56] Yingzhe He, Guozhu Meng, Kai Chen, Xingbo Hu, and Jinwen He. 2022. Towards Security Threats of Deep Learning Systems: A Survey. *IEEE Transactions on Software Engineering* 48, 5 (2022), 1743–1770.
- [57] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. OpenReview.net, virtual, 1–22.
- [58] Jordan Henkel, Goutham Ramakrishnan, Zi Wang, Aws Albarghouthi, Somesh Jha, and Thomas W. Reps. 2022. Semantic Robustness of Models of Source Code. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, Honolulu, HI, USA, 526–537.
- [59] Abram Hindle, Earl T Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the naturalness of software. *Commun. ACM* 59, 5 (2016), 122–131.
- [60] Geoffrey Hinton, Li Deng, Dong Yu, George E Dahl, Abdel rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N Sainath, et al. 2012. Deep Neural Networks for Acoustic Modeling in Speech Recognition: The Shared Views of Four Research Groups. *IEEE Signal processing magazine* 29, 6 (2012), 82–97.
- [61] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Comput.* 9, 8 (1997), 1735–1780.
- [62] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. *arXiv abs/2203.15556* (2022), 1–36.
- [63] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8 (2024), 220:1–220:79.
- [64] Yizhan Huang, Yichen Li, Weibin Wu, Jianping Zhang, and Michael R. Lyu. 2023. Do Not Give Away My Secrets: Uncovering the Privacy Issue of Neural Code Completion Tools. *arXiv abs/2309.07639* (2023), 1–23.
- [65] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. *arXiv abs/1909.09436* (2019), 1–6.
- [66] Aftab Hussain, Md. Rafiqul Islam Rabin, Toufique Ahmed, Mohammad Amin Alipour, and Bowen Xu. 2023. Occlusion-based Detection of Trojan-triggering Inputs in Large Language Models of Code. *arXiv abs/2312.04004* (2023), 1–13.
- [67] Aftab Hussain, Md. Rafiqul Islam Rabin, Toufique Ahmed, Bowen Xu, Prem Devanbu, and Mohammad Amin Alipour. 2023. A Survey of Trojans in Neural Models of Source Code: Taxonomy and Techniques. *arXiv abs/2305.03803* (2023), 1–21.
- [68] Aftab Hussain, Md Rafiqul Islam Rabin, and Mohammad Amin Alipour. 2023. Trojanedcm: A repository for poisoned neural models of source code. *arXiv abs/2312.04004* (2023), 1–9.
- [69] Aftab Hussain, Md. Rafiqul Islam Rabin, and Mohammad Amin Alipour. 2024. On Trojan Signatures in Large Language Models of Code. *arXiv abs/2402.16896*, 1 (2024), 1–10.
- [70] Aftab Hussain, Md. Rafiqul Islam Rabin, Navid Ayoobi, and Mohammad Amin Alipour. 2024. Measuring Impacts of Poisoning on Model Parameters and Neuron Activations: A Case Study of Poisoning CodeBERT. *arXiv abs/2402.12936* (2024), 1–12.
- [71] Cristina Improtà, Pietro Liguori, Roberto Natella, Bojan Cukic, and Domenico Cotroneo. 2023. Enhancing Robustness of AI Offensive Code Generators via Data Augmentation. *arXiv abs/2306.05079* (2023), 1–48.
- [72] Aylin Caliskan Islam, Richard E. Harang, Andrew Liu, Arvind Narayanan, Clare R. Voss, Fabian Yamaguchi, and Rachel Greenstadt. 2015. De-anonymizing Programmers via Code Stylometry. In *Proceedings of the 24th USENIX Security Symposium*. USENIX Association, Washington, D.C., USA, 255–270.

- [73] Maliheh Izadi, Jonathan Katzy, Tim van Dam, Marc Otten, Razvan Mihai Popescu, and Arie van Deursen. 2024. Language Models for Code Completion: A Practical Evaluation. In *Proceedings of the 46th International Conference on Software Engineering*. ACM, Lisbon, Portugal, 79:1–79:13.
- [74] Matthew Jagielski, Alina Oprea, Battista Biggio, Chang Liu, Cristina Nita-Rotaru, and Bo Li. 2018. Manipulating Machine Learning: Poisoning Attacks and Countermeasures for Regression Learning. In *Proceedings of the 2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society, San Francisco, California, USA, 19–35.
- [75] Akshita Jha and Chandan K. Reddy. 2023. CodeAttack: Code-Based Adversarial Attacks for Pre-trained Programming Language Models. In *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence*. AAAI Press, Washington, DC, USA, 14892–14900.
- [76] Lichen Jia, Bowen Tang, Chenggang Wu, Zhe Wang, Zihan Jiang, Yuanming Lai, Yan Kang, Ning Liu, and Jingfeng Zhang. 2022. FuncFooler: A Practical Black-box Attack Against Learning-based Binary Code Similarity Detection Methods. *arXiv abs/2208.14191* (2022), 1–9.
- [77] Siyuan Jiang, Ameer Armaly, and Collin McMillan. 2017. Automatically generating commit messages from diffs using neural machine translation. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering*. IEEE Computer Society, Urbana, IL, USA, 135–146.
- [78] Di Jin, Zhijing Jin, Joey Tianyi Zhou, and Peter Szolovits. 2020. Is BERT Really Robust? A Strong Baseline for Natural Language Attack on Text Classification and Entailment. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence*. AAAI Press, New York, NY, USA, 8018–8025.
- [79] Pascal Junod, Julien Rinaldini, Johan Wehrli, and Julie Michielin. 2015. Obfuscator-LLVM - Software Protection for the Masses. In *Proceedings of the 1st IEEE/ACM International Workshop on Software Protection*. IEEE Computer Society, Florence, Italy, 3–9.
- [80] Rafael-Michael Karampatsis and Charles Sutton. 2019. Maybe deep neural networks are the best choice for modeling source code. *arXiv abs/1903.05734* (2019), 1–12.
- [81] Seohyun Kim, Jinman Zhao, Yuchi Tian, and Satish Chandra. 2021. Code Prediction by Feeding Trees to Transformers. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering*. IEEE, Madrid, Spain, 150–162.
- [82] Yoon Kim. 2014. Convolutional Neural Networks for Sentence Classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*. ACL, Doha, Qatar, 1746–1751.
- [83] Barbara A. Kitchenham, Pearl Brereton, Zhi Li, David Budgen, and Andrew James Burn. 2011. Repeatability of systematic literature reviews. In *Proceedings of the 15th International Conference on Evaluation and Assessment in Software Engineering*. IET - The Institute of Engineering and Technology / IEEE Xplore, Durham, UK, 46–55.
- [84] Barbara A. Kitchenham, Lech Madeyski, and David Budgen. 2023. SEGRESS: Software Engineering Guidelines for REporting Secondary Studies. *IEEE Transactions on Software Engineering* 49, 3 (2023), 1273–1298.
- [85] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks. In *Proceedings of the 26th Annual Conference on Neural Information Processing Systems*. OpenReview.net, Lake Tahoe, Nevada, United States, 1106–1114.
- [86] Triet Huynh Minh Le, Hao Chen, and Muhammad Ali Babar. 2021. Deep Learning for Source Code Modeling and Generation: Models, Applications, and Challenges. *Comput. Surveys* 53, 3 (2021), 62:1–62:38.
- [87] Alexander LeClair, Siyuan Jiang, and Collin McMillan. 2019. A neural model for generating natural language summaries of program subroutines. In *Proceedings of the 41st International Conference on Software Engineering*. IEEE / ACM, Montreal, QC, Canada, 795–806.
- [88] Dianqi Li, Yizhe Zhang, Hao Peng, Liqun Chen, Chris Brockett, Ming-Ting Sun, and Bill Dolan. 2020. Contextualized perturbation for textual adversarial attack. *arXiv abs/2009.07502* (2020), 1–17.
- [89] Jia Li, Zhuo Li, Huangzhao Zhang, Ge Li, Zhi Jin, Xing Hu, and Xin Xia. 2024. Poison Attack and Poison Detection on Deep Source Code Processing Models. *ACM Transactions on Software Engineering and Methodology* 33, 3 (2024), 62:1–62:31.
- [90] Linyang Li, Ruotian Ma, Qipeng Guo, Xiangyang Xue, and Xipeng Qiu. 2020. BERT-ATTACK: Adversarial Attack Against BERT Using BERT. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online, 6193–6202.
- [91] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, and Jenny Chim and. 2023. StarCoder: may the source be with you! *Transactions on Machine Learning Research* 2023 (2023), 1–55.
- [92] Xinyue Li, Zhenpeng Chen, Jie M Zhang, Federica Sarro, Ying Zhang, and Xuanzhe Liu. 2024. Bias Behind the Wheel: Fairness Testing of Autonomous Driving Systems. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2024), 1–24.
- [93] Yujia Li, Chenjie Gu, Thomas Dullien, Oriol Vinyals, and Pushmeet Kohli. 2019. Graph Matching Networks for Learning the Similarity of Graph Structured Objects. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, Long Beach, California, USA, 3835–3845.
- [94] Yanzhou Li, Shangqing Liu, Kangjie Chen, Xiaofei Xie, Tianwei Zhang, and Yang Liu. 2023. Multi-target Backdoor Attacks for Code Pre-trained Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. Association for

- Computational Linguistics, Toronto, Canada, 7236–7254.
- [95] Yiyang Li, Hongqiu Wu, and Hai Zhao. 2022. Semantic-Preserving Adversarial Code Comprehension. In *Proceedings of the 29th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, Gyeongju, Republic of Korea, 3017–3028.
 - [96] Zhen Li, Qian (Guenevere) Chen, Chen Chen, Yayi Zou, and Shouhuai Xu. 2022. RoPGen: Towards Robust Code Authorship Attribution via Automatic Coding Style Transformation. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering*. ACM, Pittsburgh, PA, USA, 1906–1918.
 - [97] Zhen Li, Xiang Huang, Yangrui Li, and Guenevere Chen. 2023. A comparative study of adversarial training methods for neural models of source code. *Future Generation Computer Systems* 142 (2023), 165–181.
 - [98] Pietro Liguori, Erfan Al-Hossami, Domenico Cotroneo, Roberto Natella, Bojan Cukic, and Samira Shaikh. 2021. Shellcode_IA32: A Dataset for Automatic Shellcode Generation. *arXiv abs/2104.13100* (2021), 1–8.
 - [99] Guanjun Lin, Sheng Wen, Qing-Long Han, Jun Zhang, and Yang Xiang. 2020. Software Vulnerability Detection Using Deep Neural Networks: A Survey. *Proc. IEEE* 108, 10 (2020), 1825–1848.
 - [100] Chenxiao Liu and Xiaojun Wan. 2021. CodeQA: A Question Answering Dataset for Source Code Comprehension. In *Findings of the Association for Computational Linguistics*. Association for Computational Linguistics, Punta Cana, Dominican Republic, 2618–2632.
 - [101] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2024. Large Language Model-Based Agents for Software Engineering: A Survey. *arXiv abs/2409.02977* (2024), 1–38.
 - [102] Qianjun Liu, Shouling Ji, Changchang Liu, and Chunming Wu. 2021. A Practical Black-Box Attack on Source Code Authorship Identification Classifiers. *IEEE Transactions on Information Forensics and Security* 16 (2021), 3620–3633.
 - [103] Yingqi Liu, Shiqing Ma, Yousra Aafer, Wen-Chuan Lee, Juan Zhai, Weihang Wang, and Xiangyu Zhang. 2018. Trojaning Attack on Neural Networks. In *Proceedings of the 25th Annual Network and Distributed System Security Symposium*. The Internet Society, San Diego, California, USA, 1–15.
 - [104] Yue Liu, Chakkrit Tantithamthavorn, Li Li, and Yepang Liu. 2023. Deep Learning for Android Malware Defenses: A Systematic Literature Review. *Comput. Surveys* 55, 8 (2023), 153:1–153:36.
 - [105] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. OpenReview.net, virtual, 1–14.
 - [106] Peizhuo Lv, Chang Yue, Ruigang Liang, Yunfei Yang, Shengzhi Zhang, Hualong Ma, and Kai Chen. 2023. A Data-free Backdoor Injection Approach in Neural Networks. In *Proceedings of the 32nd USENIX Security Symposium*. USENIX Association, Anaheim, CA, USA, 2671–2688.
 - [107] Stephen G. MacDonell, Martin J. Shepperd, Barbara A. Kitchenham, and Emilia Mendes. 2010. How Reliable Are Systematic Reviews in Empirical Software Engineering? *IEEE Transactions on Software Engineering* 36, 5 (2010), 676–687.
 - [108] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2018. Towards Deep Learning Models Resistant to Adversarial Attacks. In *Proceedings of the 6th International Conference on Learning Representations*. OpenReview.net, Vancouver, BC, Canada, 1–28.
 - [109] William J. Martin, Federica Sarro, Yue Jia, Yuanyuan Zhang, and Mark Harman. 2017. A Survey of App Store Analysis for Software Engineering. *IEEE Transactions on Software Engineering* 43, 9 (2017), 817–847.
 - [110] Luca Massarelli, Giuseppe Antonio Di Luna, Fabio Petroni, Leonardo Querzoni, and Roberto Baldoni. 2021. Function representations for binary similarity. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2021), 2259–2273.
 - [111] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader-Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bayota. 2021. Studying the Usage of Text-To-Text Transfer Transformer to Support Code-Related Tasks. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering*. IEEE, Madrid, Spain, 336–347.
 - [112] Alina Matyukhina, Natalia Stakhanova, Mila Dalla Preda, and Celine Perley. 2019. Adversarial Authorship Attribution in Open-Source Projects. In *Proceedings of the 9th ACM Conference on Data and Application Security and Privacy*. ACM, Richardson, TX, USA, 291–302.
 - [113] Xiaozhu Meng, Barton P. Miller, and Somesh Jha. 2018. Adversarial Binaries for Authorship Identification. *arXiv abs/1809.08316* (2018), 1–15.
 - [114] Valeria Mercuri, Martina Saletta, and Claudio Ferretti. 2023. Evolutionary Approaches for Adversarial Attacks on Neural Source Code Classifiers. *Algorithms* 16, 10 (2023), 478.
 - [115] Meta. 2023. Code Llama: Open Foundation Models for Code. <https://ai.meta.com/research/publications/code-llamaopen-foundation-models-for-code/>.
 - [116] Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. 1953. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics* 21, 6 (1953), 1087–1092.

- [117] Rahul Mohanani, Iflaah Salman, Burak Turhan, Pilar Rodríguez, and Paul Ralph. 2020. Cognitive Biases in Software Engineering: A Systematic Mapping Study. *IEEE Transactions on Software Engineering* 46, 12 (2020), 1318–1339.
- [118] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. 2016. Convolutional Neural Networks over Tree Structures for Programming Language Processing. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*. AAAI Press, Phoenix, Arizona, USA, 1287–1293.
- [119] CheolWon Na, YunSeok Choi, and Jee-Hyong Lee. 2023. DIP: Dead code Insertion based Black-box Attack for Programming Language Model. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Toronto, Canada, 7777–7791.
- [120] Phuong T. Nguyen, Claudio Di Sipio, Juri Di Rocco, Massimiliano Di Penta, and Davide Di Ruscio. 2021. Adversarial Attacks to API Recommender Systems: Time to Wake Up and Smell the Coffee. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Melbourne, Australia, 253–265.
- [121] Thanh-Dat Nguyen, Yang Zhou, Xuan-Bach Dinh Le, Patanamon Thongtanunam, and David Lo. 2023. Adversarial Attacks on Code Models with Discriminative Graph Patterns. *arXiv abs/2308.11161* (2023), 1–13.
- [122] Liang Niu, Muhammad Shujaat Mirza, Zayd Maradni, and Christina Pöpper. 2023. CodexLeaks: Privacy Leaks from Code Generation Language Models in GitHub Copilot. In *Proceedings of the 32nd USENIX Security Symposium*. USENIX Association, Anaheim, CA, USA, 2133–2150.
- [123] Rise of the Planet of Serverless Computing: A Systematic Review. 2023. Jinfeng Wen and Zhenpeng Chen and Xin Jin and Xuanzhe Liu. *ACM Transactions on Software Engineering and Methodology* 32, 5 (2023), 131:1–131:61.
- [124] Sanghak Oh, Kiho Lee, Seonhye Park, Doowon Kim, and Hyoungshick Kim. 2024. Poisoned ChatGPT Finds Work for Idle Hands: Exploring Developers’ Coding Practices with Insecure Suggestions from Poisoned AI Models. In *Proceedings of the IEEE Symposium on Security and Privacy*. IEEE, San Francisco, CA, USA, 1141–1159.
- [125] Michael O’Neill, Miguel Nicolau, and Alexandros Agapitos. 2014. Experiments in program synthesis with grammatical evolution: A focus on Integer Sorting. In *Proceedings of the IEEE Congress on Evolutionary Computation*. IEEE, Beijing, China, 1504–1511.
- [126] OpenAI. 2023. Code Interpreter. <https://openai.com/blog/chatgpt-plugins#code-interpreter>.
- [127] OpenAI. 2023. Codex. site: <https://openai.com/blog/openai-codex>.
- [128] David N Palacio, Nathan Cooper, Alvaro Rodriguez, Kevin Moran, and Denys Poshyvanyk. 2023. Toward a Theory of Causation for Interpreting Neural Code Models. *arXiv abs/2302.03788* (2023), 1–28.
- [129] Nicolas Papernot, Patrick D. McDaniel, Ian J. Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. 2017. Practical Black-Box Attacks against Machine Learning. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*. ACM, Abu Dhabi, United Arab Emirates, 506–519.
- [130] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. ACL, Philadelphia, PA, USA, 311–318.
- [131] Maryam Vahdat Pour, Zhuo Li, Lei Ma, and Hadi Hemmati. 2021. A Search-Based Testing Framework for Deep Neural Networks of Source Code Embedding. In *Proceedings of the 14th IEEE Conference on Software Testing, Verification and Validation*. IEEE, Porto de Galinhas, Brazil, 36–46.
- [132] Fanchao Qi, Yangyi Chen, Mukai Li, Yuan Yao, Zhiyuan Liu, and Maosong Sun. 2021. ONION: A Simple and Effective Defense Against Textual Backdoor Attacks. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Punta Cana, Dominican Republic, 9558–9566.
- [133] Shiyi Qi, Yuanhang Yang, Shuzheng Gao, Cuiyun Gao, and Zenglin Xu. 2023. BadCS: A Backdoor Attack Framework for Code search. *arXiv abs/2305.05503* (2023), 1–11.
- [134] Erwin Quiring, Alwin Maier, and Konrad Rieck. 2019. Misleading Authorship Attribution of Source Code using Adversarial Learning. In *Proceedings of the 28th USENIX Security Symposium*. USENIX Association, Santa Clara, CA, USA, 479–496.
- [135] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. *OpenAI* 1, 1 (2018), 1–12.
- [136] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [137] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21 (2020), 140:1–140:67.
- [138] Goutham Ramakrishnan and Aws Albarghouthi. 2022. Backdoors in Neural Models of Source Code. In *Proceedings of the 26th International Conference on Pattern Recognition*. IEEE, Montreal, QC, Canada, 2892–2899.
- [139] Aurora Ramirez, José Raúl Romero, and Christopher L. Simons. 2019. A Systematic Review of Interaction in Search-Based Software Engineering. *IEEE Transactions on Software Engineering* 45, 8 (2019), 760–781.
- [140] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. *arXiv abs/2009.10297* (2020), 1–8.

- [141] Saksham Sachdev, Hongyu Li, Sifei Luan, Seohyun Kim, Koushik Sen, and Satish Chandra. 2018. Retrieval on source code: a neural code search. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. ACM, Philadelphia, PA, USA, 31–41.
- [142] Roei Schuster, Congzheng Song, Eran Tromer, and Vitaly Shmatikov. 2021. You Autocomplete Me: Poisoning Vulnerabilities in Neural Code Completion. In *Proceedings of the 30th USENIX Security Symposium*. USENIX Association, Vancouver, B.C., Canada, 1559–1575.
- [143] Carolyn B. Seaman. 1999. Qualitative Methods in Empirical Studies of Software Engineering. *IEEE Transactions on software Engineering* 25, 4 (1999), 557–572.
- [144] Giorgio Severi, Jim Meyer, Scott E. Coull, and Alina Oprea. 2021. Explanation-Guided Backdoor Poisoning Attacks Against Malware Classifiers. In *Proceedings of the 30th USENIX Security Symposium*. USENIX Association, Vancouver, B.C., Canada, 1487–1504.
- [145] Murray Shanahan. 2024. Talking about large language models. *Commun. ACM* 67, 2 (2024), 68–79.
- [146] Mohammed Latif Siddiq and Joanna CS Santos. 2022. SecurityEval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques. In *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*. ACM, Singapore, Singapore, 29–33.
- [147] Charles Smutz and Angelos Stavrou. 2012. Malicious PDF detection using metadata and structural features. In *Proceedings of the 28th Annual Computer Security Applications Conference*. ACM, Orlando, FL, USA, 239–248.
- [148] Congzheng Song, Thomas Ristenpart, and Vitaly Shmatikov. 2017. Machine Learning Models that Remember Too Much. In *Proceedings of the 2017 ACM Conference on Computer and Communications Security*. ACM, Dallas, TX, USA, 587–601.
- [149] Jacob M Springer, Bryn Marie Reinstadler, and Una-May O'Reilly. 2020. STRATA: simple, gradient-free attacks for models of code. *arXiv abs/2009.13562* (2020), 1–9.
- [150] Shashank Srikant, Sijia Liu, Tamara Mitrovska, Shiyu Chang, Quanfu Fan, Gaoyuan Zhang, and Una-May O'Reilly. 2021. Generating Adversarial Computer Programs using Optimized Obfuscations. In *Proceedings of the 9th International Conference on Learning Representations*. OpenReview.net, Austria, 1–16.
- [151] Keele Staffs et al. 2007. *Guidelines for performing systematic literature reviews in software engineering*. Technical Report. Technical report, ver. 2.3 ebse technical report. ebse.
- [152] Stunnix. 2002. Stunnix. <http://stunnix.com/>.
- [153] Weisong Sun, Yuchen Chen, Chunrong Fang, Yebo Feng, Yuan Xiao, An Guo, Qunjun Zhang, Yang Liu, Baowen Xu, and Zhenyu Chen. 2025. Eliminating Backdoors in Neural Code Models for Secure Code Understanding. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, Trondheim, Norway, 1–23.
- [154] Weisong Sun, Yuchen Chen, Guanhong Tao, Chunrong Fang, Xiangyu Zhang, Qunjun Zhang, and Bin Luo. 2023. Backdooring Neural Code Search. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Toronto, Canada, 9692–9708.
- [155] Weisong Sun, Yuchen Chen, Mengzhe Yuan, Chunrong Fang, Zhenpeng Chen, Chong Wang, Yang Liu, Baowen Xu, and Zhenyu Chen. 2025. Show Me Your Code! Kill Code Poisoning: A Lightweight Method Based on Code Naturalness. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*. IEEE Computer Society, Ottawa, Ontario, Canada, 1–12.
- [156] Weisong Sun, Chunrong Fang, Yuchen Chen, Guanhong Tao, Tingxu Han, and Qunjun Zhang. 2022. Code Search based on Context-aware Code Translation. In *Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering*. ACM, May 25–27, 388–400.
- [157] Weisong Sun, Chunrong Fang, Yifei Ge, Yuling Hu, Yuchen Chen, Qunjun Zhang, Xiuting Ge, Yang Liu, and Zhenyu Chen. 2024. A Survey of Source Code Search: A 3-Dimensional Perspective. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 166:1–51.
- [158] Weisong Sun, Yun Miao, Yuekang Li, Hongyu Zhang, Chunrong Fang, Yi Liu, Gelei Deng, Yang Liu, and Zhenyu Chen. 2024. Source Code Summarization in the Era of Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. IEEE/ACM, Lisbon, Portugal, 419–431.
- [159] Zhensu Sun, Xiaoning Du, Xiapu Luo, Fu Song, David Lo, and Li Li. 2024. FDI: Attack Neural Code Generation Systems through User Feedback Channel. In *Proceedings of the 33rd ACM International Symposium on Software Testing and Analysis*. ACM, Vienna, Austria, 528–540.
- [160] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic Attribution for Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, Sydney, NSW, Australia, 3319–3328.
- [161] Jeffrey Svajlenko, Judith F. Islam, Iman Keivanloo, Chanchal Kumar Roy, and Mohammad Mamun Mia. 2014. Towards a Big Data Curated Benchmark of Inter-project Code Clones. In *Proceedings of the 30th IEEE International Conference on Software Maintenance and Evolution*. IEEE Computer Society, Victoria, BC, Canada, 476–480.
- [162] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. IntelliCode compose: code generation using transformer. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Virtual Event, USA, 1433–1443.

- [163] Alexey Svyatkovskiy, Ying Zhao, Shengyu Fu, and Neel Sundaresan. 2019. Pythia: AI-assisted Code Completion System. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery*. ACM, Anchorage, AK, USA, 2727–2735.
- [164] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian J. Goodfellow, and Rob Fergus. 2014. Intriguing properties of neural networks. In *Proceedings of the 2nd International Conference on Learning Representations, Conference Track Proceedings*. OpenReview.net, Banff, AB, Canada, 1–10.
- [165] Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. 2022. Galactica: A large language model for science. *arXiv abs/2211.09085* (2022), 1–58.
- [166] Junfeng Tian, Chenxin Wang, Zhen Li, and Yu Wen. 2021. Generating adversarial examples of source code classification models via q-learning-based markov decision process. In *Proceedings of the 21st IEEE International Conference on Software Quality, Reliability and Security*. IEEE, Hainan, China, 807–818.
- [167] Zhao Tian, Junjie Chen, and Zhi Jin. 2023. Code Difference Guided Adversarial Example Generation for Deep Code Models. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, luxembourg, 850–862.
- [168] Catherine Tony, Markus Mutas, Nicolás E. Díaz Ferreyra, and Riccardo Scandariato. 2023. LLMSecEval: A Dataset of Natural Language Prompts for Security Evaluations. In *Proceedings of the 20th IEEE/ACM International Conference on Mining Software Repositories*. IEEE, Melbourne, Australia, 588–592.
- [169] Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. 2016. Stealing Machine Learning Models via Prediction APIs. In *Proceedings of the 25th USENIX Security Symposium*. USENIX Association, Austin, TX, USA, 601–618.
- [170] Brandon Tran, Jerry Li, and Aleksander Madry. 2018. Spectral Signatures in Backdoor Attacks. In *Proceedings of the 2018 32nd Conference on Neural Information Processing Systems*. OpenReview.net, Montréal, Canada, 8011–8021.
- [171] Yao Wan, Jingdong Shu, Yulei Sui, Guandong Xu, Zhou Zhao, Jian Wu, and Philip S. Yu. 2019. Multi-modal Attention Network Learning for Semantic Source Code Retrieval. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, San Diego, CA, USA, 13–25.
- [172] Yao Wan, Shijie Zhang, Hongyu Zhang, Yulei Sui, Guandong Xu, Dezhong Yao, Hai Jin, and Lichao Sun. 2022. You see what I want you to see: poisoning vulnerabilities in neural code search. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Singapore, Singapore, 1233–1245.
- [173] Chong Wang, Zhenpeng Chen, Tianlin Li, Yilun Zhao, and Yang Liu. 2024. Towards Trustworthy LLMs for Code: A Data-Centric Synergistic Auditing Framework. *arXiv abs/2410.09048* (2024), 1–6.
- [174] Chao Wang, Zhenpeng Chen, and Minghui Zhou. 2023. AutoML from Software Engineering Perspective: Landscapes and Challenges. In *Proceedings of the 20th IEEE/ACM International Conference on Mining Software Repositories*. IEEE, Melbourne, Australia, 39–51.
- [175] Huanting Wang, Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Yansong Feng, Lizhong Bian, and Zheng Wang. 2021. Combining Graph-Based Learning With Automated Data Collection for Code Vulnerability Detection. *IEEE Transactions on Information Forensics and Security* 16 (2021), 1943–1958.
- [176] Simin Wang, Liguang Huang, Amiao Gao, Jidong Ge, Tengfei Zhang, Haitao Feng, Ishna Satyarth, Ming Li, He Zhang, and Vincent Ng. 2023. Machine/Deep Learning for Software Engineering: A Systematic Literature Review. *IEEE Transactions on Software Engineering* 49, 3 (2023), 1188–1231.
- [177] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. *arXiv abs/2305.07922* (2023), 1–26.
- [178] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Punta Cana, Dominican Republic, 8696–8708.
- [179] Cody Watson, Nathan Cooper, David Nader-Palacio, Kevin Moran, and Denys Poshyvanyk. 2022. A Systematic Literature Review on the Use of Deep Learning in Software Engineering Research. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2022), 32:1–32:58.
- [180] Jinfeng Wen, Zhenpeng Chen, Yi Liu, Yiling Lou, Yun Ma, Gang Huang, Xin Jin, and Xuanzhe Liu. 2021. An empirical study on challenges of application development in serverless computing. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Athens, Greece, 416–428.
- [181] Xin-Cheng Wen, Yupan Chen, Cuiyun Gao, Hongyu Zhang, Jie M. Zhang, and Qing Liao. 2023. Vulnerability Detection with Graph Simplification and Enhanced Graph Representation Learning. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering*. IEEE, Melbourne, Australia, 2275–2286.
- [182] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. ACM, London, England, United Kingdom, 38:1–38:10.
- [183] Cong Wu, Jing Chen, Ziwei Wang, Ruichao Liang, and Ruiying Du. 2024. Semantic Sleuth: Identifying Ponzi Contracts via Large Language Models. In *Proceedings of the 39th International Conference on Automated Software Engineering*. ACM, Sacramento, CA, USA, 582–593.

- [184] Fangzhou Wu, Xiaogeng Liu, and Chaowei Xiao. 2023. Deceptprompt: Exploiting llm-driven code generation via adversarial natural language instructions. *arXiv abs/2312.04730* (2023), 1–17.
- [185] Yuan Xiao, Yuchen Chen, Shiqing Ma, Haocheng Huang, Chunrong Fang, Yanwei Chen, Weisong Sun, Yunfeng Zhu, Xiaofang Zhang, and Zhenyu Chen. 2025. DeCoMa: Detecting and Purifying Code Dataset Watermarks through Dual Channel Code Abstraction. In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, Trondheim, Norway, 1–23.
- [186] Xiaojun Xu, Chang Liu, Qian Feng, Heng Yin, Le Song, and Dawn Song. 2017. Neural Network-based Graph Embedding for Cross-Platform Binary Code Similarity Detection. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM, Dallas, TX, USA, 363–376.
- [187] Hiroki Yamamoto, Dong Wang, Gopi Krishnan Rajbahadur, Masanari Kondo, Yasutaka Kamei, and Naoyasu Ubayashi. 2023. Towards Privacy Preserving Cross Project Defect Prediction with Federated Learning. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, Taipa, Macao, 485–496.
- [188] Guang Yang, Yu Zhou, Xiang Chen, Xiangyu Zhang, Terry Yue Zhuo, David Lo, and Taolue Chen. 2024. DeCE: Deceptive Cross-Entropy Loss Designed for Defending Backdoor Attacks. *arXiv abs/2407.08956* (2024), 1–23.
- [189] Guang Yang, Yu Zhou, Wenhua Yang, Tao Yue, Xiang Chen, and Taolue Chen. 2024. How important are good method names in neural code generation? a model robustness perspective. *ACM Transactions on Software Engineering and Methodology* 33, 3 (2024), 1–35.
- [190] Yulong Yang, Haoran Fan, Chenhao Lin, Qian Li, Zhengyu Zhao, and Chao Shen. 2024. Exploiting the Adversarial Example Vulnerability of Transfer Learning of Source Code. *IEEE Trans. Inf. Forensics Secur.* 19 (2024), 5880–5894.
- [191] Zhou Yang, Jieke Shi, Junda He, and David Lo. 2022. Natural Attack for Pre-trained Models of Code. In *Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering*. ACM, Pittsburgh, PA, USA, 1482–1493.
- [192] Zhou Yang, Zhensu Sun, Terry Yue Zhuo, Premkumar T. Devanbu, and David Lo. 2024. Robustness, Security, Privacy, Explainability, Efficiency, and Usability of Large Language Models for Code. *arXiv abs/2403.07506* (2024), 1–35.
- [193] Zhou Yang, Bowen Xu, Jie M. Zhang, Hong Jin Kang, Jieke Shi, Junda He, and David Lo. 2024. Stealthy Backdoor Attack for Code Models. *IEEE Trans. Software Eng.* 50, 4 (2024), 721–741.
- [194] Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, DongGyun Han, and David Lo. 2024. Gotcha! This Model Uses My Code! Evaluating Membership Leakage Risks in Code Models. *IEEE Trans. Software Eng.* 50, 12 (2024), 3290–3306.
- [195] Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, DongGyun Han, and David Lo. 2024. Unveiling Memorization in Code Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. ACM, Lisbon, Portugal, 72:1–72:13.
- [196] Kaichun Yao, Hao Wang, Chuan Qin, Hengshu Zhu, Yanjun Wu, and Libo Zhang. 2025. CARL: Unsupervised Code-Based Adversarial Attacks for Programming Language Models via Reinforcement Learning. *ACM Trans. Softw. Eng. Methodol.* 34, 1 (2025), 22:1–22:32.
- [197] Noam Yefet, Uri Alon, and Eran Yahav. 2020. Adversarial examples for models of code. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 162:1–162:30.
- [198] Xiaoyu Yi, Gaolei Li, Ao Ding, Yan Zheng, Yuqing Li, and Jianhua Li. 2023. AdvBinSD: Poisoning the Binary Code Similarity Detector via Isolated Instruction Sequences. In *Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking*. IEEE, Wuhan, China, 1149–1156.
- [199] Xiaoyu Yi, Gaolei Li, Wenkai Huang, Xi Lin, Jianhua Li, and Yuchen Liu. 2024. LateBA: Latent Backdoor Attack on Deep Bug Search via Infrequent Execution Codes. In *Proceedings of the 15th Asia-Pacific Symposium on Internetware*. ACM, Macau, SAR, China, 1–10.
- [200] Xueqi Yu, Zhen Li, Xiang Huang, and Shasha Zhao. 2023. AdVulCode: Generating Adversarial Vulnerable Code against Deep Learning-Based Vulnerability Detectors. *Electronics* 12, 4 (2023), 936.
- [201] Chen Zeng, Yue Yu, Shanshan Li, Xin Xia, Zhiming Wang, Mingyang Geng, Linxiao Bai, Wei Dong, and Xiangke Liao. 2023. degraphics: Embedding variable-based flow graph for neural code search. *ACM Transactions on Software Engineering and Methodology* 32, 2 (2023), 1–27.
- [202] Zhengran Zeng, Hanzhuo Tan, Haotian Zhang, Jing Li, Yuqun Zhang, and Lingming Zhang. 2022. An extensive study on pre-trained models for program understanding and generation. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, Virtual Event, South Korea, 39–51.
- [203] Chi Zhang, Zifan Wang, Ravi Mangal, Matt Fredrikson, Limin Jia, and Corina S. Pasareanu. 2023. Transfer Attacks and Defenses for Large Language Models on Coding Tasks. *arXiv abs/2311.13445* (2023), 1–23.
- [204] He Zhang, Muhammad Ali Babar, and Paolo Tell. 2011. Identifying relevant studies in software engineering. *Information and Software Technology* 53, 6 (2011), 625–637.
- [205] Huangzhao Zhang, Zhiyi Fu, Ge Li, Lei Ma, Zhehao Zhao, Hua'an Yang, Yizhe Sun, Yang Liu, and Zhi Jin. 2022. Towards Robustness of Deep Program Processing Models - Detection, Estimation, and Enhancement. *ACM Transactions on Software Engineering and Methodology* 31, 3 (2022), 50:1–50:40.
- [206] Huangzhao Zhang, Zhuo Li, Ge Li, Lei Ma, Yang Liu, and Zhi Jin. 2020. Generating Adversarial Examples for Holding Robustness of Source Code Processing Models. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence*. AAAI Press, New York, NY, USA, 1169–1176.

- [207] Huangzhao Zhang, Shuai Lu, Zhuo Li, Zhi Jin, Lei Ma, Yang Liu, and Ge Li. 2024. CodeBERT-Attack: Adversarial attack against source code deep learning models via pre-trained model. *Journal of Software: Evolution and Process* 36, 3 (2024), 1–29.
- [208] Jie Zhang, Wei Ma, Qiang Hu, Xiaofei Xie, Yves Le Traon, and Yang Liu. 2023. RNNS: Representation Nearest Neighbor Search Black-Box Attack on Code Models. *arXiv abs/2305.05896* (2023), 1–11.
- [209] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, Kaixuan Wang, and Xudong Liu. 2019. A novel neural source code representation based on abstract syntax tree. In *Proceedings of the 41st International Conference on Software Engineering*. IEEE / ACM, Montreal, QC, Canada, 783–794.
- [210] Jinchuan Zhang, Yan Zhou, Binyuan Hui, Yaxin Liu, Ziming Li, and Songlin Hu. 2023. TrojanSQL: SQL Injection against Natural Language Interface to Database. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 4344–4359.
- [211] Jie M. Zhang, Mark Harman, Lei Ma, and Yang Liu. 2022. Machine Learning Testing: Survey, Landscapes and Horizons. *IEEE Transactions on Software Engineering* 48, 2 (2022), 1–36.
- [212] Quanjun Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. 2024. A Survey of Learning-based Automated Program Repair. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2024), 55:1–55:69.
- [213] Sheng Zhang, Hui Li, and Rongrong Ji. 2024. Code Membership Inference for Detecting Unauthorized Data Use in Code Pre-trained Language Models. In *Findings of the Association for Computational Linguistics*. Association for Computational Linguistics, Miami, Florida, USA, 10593–10603.
- [214] Wei Emma Zhang, Quan Z. Sheng, Ahoud Alhazmi, and Chenliang Li. 2020. Adversarial Attacks on Deep-learning Models in Natural Language Processing: A Survey. *ACM Transactions on Intelligent Systems and Technology* 11, 3 (2020), 24:1–24:41.
- [215] Yuxin Zhang, Ruoting Wu, Jie Liao, and Liang Chen. 2023. Structural Adversarial Attack for Code Representation Models. In *Collaborative Computing: Networking, Applications and Worksharing - 19th EAI International Conference (Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, Vol. 562)*. Springer, Corfu Island, Greece, 392–413.
- [216] Zhuo Zhang, Guanhong Tao, Guangyu Shen, Shengwei An, Qiuling Xu, Yingqi Liu, Yapeng Ye, Yaoxuan Wu, and Xiangyu Zhang. 2023. PELICAN: Exploiting Backdoors of Naturally Trained Deep Learning Models In Binary Code Analysis. In *Proceedings of the 32nd USENIX Security Symposium*. USENIX Association, Anaheim, CA, USA, 2365–2382.
- [217] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv abs/2303.18223* (2023), 1–144.
- [218] Shasha Zhou, Mingyu Huang, Yanan Sun, and Ke Li. 2024. Evolutionary Multi-objective Optimization for Contextual Adversarial Example Generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2285–2308.
- [219] Yaqin Zhou, Shangqing Liu, Jing Kai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In *Proceedings of the 33rd Conference on Neural Information Processing Systems*. OpenReview.net, Vancouver, BC, Canada, 10197–10207.
- [220] Yu Zhou, Xiaoqing Zhang, Juanjuan Shen, Tingting Han, Taolue Chen, and Harald C. Gall. 2022. Adversarial Robustness of Deep Code Comment Generation. *ACM Trans. Softw. Eng. Methodol.* 31, 4 (2022), 60:1–60:30.
- [221] Chen Zhu, W. Ronny Huang, Hengduo Li, Gavin Taylor, Christoph Studer, and Tom Goldstein. 2019. Transferable Clean-Label Poisoning Attacks on Deep Neural Nets. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, Long Beach, California, USA, 7614–7623.
- [222] Rui Zhu and Cunming Zhang. 2023. How Robust Is a Large Pre-trained Language Model for Code Generation? A Case on Attacking GPT2. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, Taipa, Macao, 708–712.
- [223] Terry Yue Zhuo, Zhou Yang, Zhensu Sun, Yufei Wang, Li Li, Xiaoning Du, Zhenchang Xing, and David Lo. 2023. Data Augmentation Approaches for Source Code Models: A Survey. *arXiv abs/2305.19915* (2023), 1–21.